

**RANCANG BANGUN SUBSISTEM OTOMATISASI PERANGKAT
IOT BERBASIS *WEBSITE*
(STUDI KASUS : *GREENHOUSE* KEBUN RAYA ITERA)**

TUGAS AKHIR

**MUHAMMAD YUSUF
122140193**



**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI SUMATERA
LAMPUNG SELATAN
2025**

LEMBAR PENGESAHAN

Saya menyatakan bahwa Tugas Akhir berjudul “RANCANG BANGUN SUBSISTEM OTOMATISASI PERANGKAT IOT BERBASIS *WEBSITE* (STUDI KASUS : GREENHOUSE KEBUN RAYA ITERA)” merupakan hasil karya saya sendiri dan belum pernah diajukan, baik sebagian maupun seluruhnya, di Institut Teknologi Sumatera atau institusi pendidikan lain oleh saya maupun pihak lain.

Lampung Selatan, 9 Januari 2025

Penulis,

Muhammad Yusuf
NIM.122140193

Diperiksa dan disetujui oleh,
Pembimbing

1. Muhammad Habib Algifari, S.Kom., M.T.I.
NIP. 199105252022031002
2. Winda Yulita, M.Cs.
NIP. 199307272022032022

Penguji

1. Ilham Firman Ashari, S.Kom., M.T
NIP. 199303142019031018
2. Eko Dwi Nugroho, S.Kom., M.Cs.
NIP. 199102092024061001

Disahkan oleh,
Koordinator Program Studi Teknik Informatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera

Andika Setiawan, S.Kom., M.Cs.
NIP. 199111272022031007

HALAMAN PERNYATAAN ORISINALITAS

Tugas Akhir dengan judul “RANCANG BANGUN SUBSISTEM OTOMATISASI PERANGKAT IOT BERBASIS WEBSITE (STUDI KASUS : *GREENHOUSE* KEBUN RAYA ITERA)” adalah karya saya sendiri, dan semua sumber baik yang dikutip maupun dirujuk telah saya nyatakan benar.

Nama : Muhammad Yusuf
NIM : 122140193

Tanda Tangan :
Tanggal :

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai civitas akademik Institut Teknologi Sumatera, saya yang bertanda tangan di bawah ini:

Nama : Muhammad Yusuf
NIM : 122140193
Program Studi : Teknik Informatika
Fakultas : Fakultas Teknologi Industri
Jenis Karya : Tugas Akhir

demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Institut Teknologi Sumatera Hak Bebas Royalti *Noneksklusif* (*Non-exclusive Royalty Free Right*) atas karya ilmiah saya yang berjudul:

RANCANG BANGUN SUBSISTEM OTOMATISASI PERANGKAT IOT
BERBASIS *WEBSITE*
(STUDI KASUS : *GREENHOUSE* KEBUN RAYA ITERA)

beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini Institut Teknologi Sumatera berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (database), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Lampung Selatan

Pada tanggal : 9-Januari-2025

Yang Menyatakan
Muhammad Yusuf
NIM. 122140193

KATA PENGANTAR

Puji syukur kehadiran Allah SWT/Tuhan Yang Maha Esa atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga penyusunan tugas akhir ini telah terselesaikan dengan baik. Dalam penyusunan tugas akhir ini penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. Prof. Dr. I Nyoman Pugeg Aryantha selaku Rektor Institut Teknologi Sumatera.
2. Hadi Teguh Yudistira, S.T., Ph.D. selaku Dekan Fakultas Teknologi Industri.
3. Andika Setiawan, S.Kom., M.Cs. selaku Ketua Program Studi Teknik Informatika.
4. Muhammad Habib Algifari, S.Kom., M.TI. selaku Dosen Pembimbing I atas ide, waktu, tenaga, perhatian, dan masukan yang telah disumbangsikan kepada penulis pada kelancaran penulisan dan pengerjaan Tugas Akhir.
5. Winda Yulita, M.Cs. selaku Dosen Pembimbing II, atas bimbingan, arahan, motivasi, serta saran-saran yang sangat membantu dalam penyempurnaan penelitian dan penulisan skripsi ini.
6. Terimakasih kepada seluruh Dosen Penelitian *Smart Farming* ITERAHERO karena sudah mengajarkan banyak hal tentang dunia pertanian dan penelitian.
7. Rasa syukur yang mendalam penulis haturkan kepada kedua orang tua dan segenap keluarga, yang telah menjadi fondasi utama serta pemberi pelajaran paling berharga tentang kesabaran dan perjuangan, hingga penulis mampu sampai pada titik ini.
8. Terima kasih tulus untuk teman-teman terbaik yang selalu ada untuk saling mendukung, mengingatkan, dan menguatkan. Perjalanan hingga tahap akhir ini terasa lebih ringan berkat kehadiran, setiap kritik, dan diskusi yang telah dilalui bersama.
9. Terimakasih untuk penulis karena sudah mampu sampai di fase yang tidak mudah untuk dilalui, banyak pelajaran, tantangan, dan penghargaan yang sudah dilalui dengan penuh kekuatan, kegigihan, serta semangat dan aksi yang berjalan beriringan.

Akhir kata penulis berharap semoga tugas akhir ini dapat memberikan manfaat bagi kita semua dan para pembaca.

MOTTO PENULIS

“Keep Doing, Keep Learning, Keep Growing”

“Do the BEST, let GOD the REST”

~ Muhammad Yusuf ~

RINGKASAN

RANCANG BANGUN SUBSISTEM OTOMATISASI PERANGKAT IOT BERBASIS *WEBSITE* (STUDI KASUS : *GREENHOUSE* KEBUN RAYA ITERA)

Muhammad Yusuf

Halaman Ringkasan berisi uraian singkat tentang latar belakang masalah, rumusan masalah, tujuan, metodologi penelitian, hasil dan analisis data, serta kesimpulan dan saran. Isi ringkasan tidak lebih dari 700 - 1000 kata.

ABSTRAK

RANCANG BANGUN SUBSISTEM OTOMATISASI PERANGKAT IOT BERBASIS *WEBSITE* (STUDI KASUS : *GREENHOUSE* KEBUN RAYA ITERA)

Muhammad Yusuf

Abstrak adalah ringkasan isi skripsi yang memuat secara padat, jelas, dan sistematis unsur-unsur penting dari suatu penelitian. Tujuan utama penulisan abstrak adalah untuk memberikan gambaran umum mengenai isi penelitian kepada pembaca tanpa harus membaca keseluruhan skripsi. Abstrak ditulis dalam satu paragraf tanpa sub judul atau poin-poin, menggunakan bahasa yang formal dan akademik. Panjang abstrak 200 - 300 kata dan harus disusun secara ringkas namun mencerminkan keseluruhan isi dari skripsi, mulai dari latar belakang, rumusan masalah, tujuan, metode yang digunakan, hasil utama, hingga kesimpulan dari penelitian.

Penulisan abstrak dilakukan dalam bahasa Indonesia yang baik dan benar. Untuk keperluan publikasi dan dokumentasi akademik, abstrak juga harus disediakan dalam bahasa Inggris (Abstract).

Di bagian akhir abstrak dicantumkan Kata Kunci yang ditulis dengan huruf tebal dan diikuti tanda titik dua, kemudian disusul oleh tiga hingga lima kata kunci yang relevan dengan topik, objek, dan metode penelitian. Kata kunci tersebut harus ditulis dalam satu baris dan dipisahkan dengan tanda koma. Kata kunci berfungsi sebagai penanda indeksasi dalam basis data ilmiah dan memudahkan pencarian dokumen oleh peneliti lain.

Kata Kunci: Otomatisasi Perangkat IoT; Agile Kanban; Blackbox Testing, MQTT Explorer; *System Usability Scale* (SUS);

ABSTRACT

WEBSITE-BASED AUTOMATION SUBSYSTEM FOR IOT DEVICES (CASE STUDY: ITERA BOTANICAL GARDEN GREENHOUSE)

Muhammad Yusuf

An abstract is a summary of the thesis content that presents, in a concise, clear, and systematic manner, the essential elements of a research study. The main purpose of writing an abstract is to provide readers with an overview of the research without requiring them to read the entire thesis. The abstract is written in a single paragraph without subheadings or bullet points, using formal and academic language. It should be between 200 and 300 words in length and must be structured succinctly while reflecting the overall content of the thesis, including the background, problem formulation, research objectives, methods used, main findings, and conclusions.

The abstract must be written in proper and correct Indonesian. For academic documentation and publication purposes, the abstract should also be provided in English.

At the end of the abstract, Keywords must be included, written in bold and followed by a colon, then listed with three to five terms that are relevant to the topic, object, and methods of the research. These keywords should appear in a single line and be separated by commas. Keywords serve as indexing markers in academic databases and help facilitate document searches by other researchers.

Keywords: IoT Device Automation; Agile Kanban; Black-Box Testing; MQTT Explorer; System Usability Scale (SUS).

DAFTAR ISI

LEMBAR PENGESAHAN.....	ii
HALAMAN PERNYATAAN ORISINALITAS	iii
HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS	iv
KATA PENGANTAR	v
MOTTO PENULIS.....	vi
RINGKASAN.....	vii
ABSTRAK	viii
ABSTRACT	ix
DAFTAR ISI	x
DAFTAR GAMBAR	xiii
DAFTAR TABEL.....	xiv
DAFTAR RUMUS	xv
DAFTAR KODE	xvi
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Rumusan Masalah	3
1.3 Tujuan Penelitian	3
1.4 Batasan Masalah	4
1.5 Sistematika Penulisan.....	4
1.6 Manfaat Penelitian	4
1.6.1 BAB I.....	5
1.6.2 BAB II.....	5
1.6.3 BAB III.....	5
1.6.4 BAB IV	5
1.6.5 BAB V.....	5
BAB II TINJAUAN PUSTAKA	6
2.1 Tinjauan Pustaka.....	6
2.2 Dasar Teori	7
2.2.1 <i>Subsistem</i>	8
2.2.2 <i>Website</i>	8
2.2.3 <i>Greenhouse Kebun Raya ITERA</i>	9

2.2.4	IoT.....	9
2.2.5	Protokol MQTT	9
2.2.6	MQTT Explorer.....	10
2.2.7	Perangkat IoT	11
2.2.8	<i>Otomatisasi Perangkat IoT</i>	12
2.2.9	<i>Agile Kanban</i>	12
2.2.10	<i>Blackbox Testing</i>	13
2.2.11	<i>System Usability Scale (SUS)</i>	14
2.2.12	<i>Diagram Use Case</i>	16
2.2.13	<i>Diagram Aktivitas</i>	17
2.2.14	<i>Entity Relationship Diagram</i>	18
2.2.15	<i>Purposive Sampling</i>	18
2.2.16	<i>Websocket</i>	19
2.2.17	Laravel	19
2.2.18	Redis	20
2.2.19	<i>XHR Polling</i>	20
2.2.20	<i>Event Driven</i>	20
2.2.21	<i>Cron Scheduler</i>	21
2.2.22	<i>Queue Worker</i>	21
BAB III METODE PENELITIAN		23
3.1	Alur Penelitian	23
3.2	Alat dan Bahan	24
3.2.1	Alat Penelitian	25
3.2.2	Bahan Penelitian	25
3.3	Tahapan Seluruh Alur Penelitian.....	26
3.3.1	Inisialisasi.....	26
3.3.2	Proses Metode Agile Kanban	31
3.3.3	Pengujian <i>Usability</i>	48
3.3.4	Analisis dan Kesimpulan.....	50
BAB IV HASIL DAN PEMBAHASAN		51
4.1	Implementasi Hasil	51
4.1.1	Implementasi Arsitektur.....	51

4.1.2	Implementasi <i>Backend</i>	63
4.1.3	Implementasi <i>Frontend</i>	66
4.2	Pengujian	75
4.2.1	Pengujian <i>Blackbox</i>	75
4.2.2	Pengujian Komunikasi Data dan Otomatisasi	76
4.2.3	Pengujian Responsif Halaman	76
4.3	<i>Deploy</i>	77
4.4	Review	77
4.5	Pengujian <i>Usability</i>	78
4.6	Analisis	79
BAB V KESIMPULAN		81
5.1	Kesimpulan	81
5.2	Saran	81
DAFTAR PUSTAKA		82
LAMPIRAN		86
A.	Surat Tugas Ibu Zunanik Mufidah, S.TP., M.Si. Sebagai Ketua Tim Penelitian <i>Greenhouse</i> Kebun Raya ITERA	86
B.	Format Instrumen dan Hasil Wawancara	87
C.	Daftar Bukti Dokumentasi Penelitian	90
D.	Rincian Proses Agile Kanban	92
E.	<i>Stakeholder</i> yang memiliki akses	94
F.	Rincian Pengujian <i>Blackbox Testing</i>	95

DAFTAR GAMBAR

Gambar 2.1 Visualisasi Subsistem.....	8
Gambar 2.2 Tampak Dalam dan Depan Greenhouse Kebun Raya ITERA	9
Gambar 2.3 Tampilan Aplikasi MQTT <i>Explorer</i>	11
Gambar 2.4 Papan Agile Kanban	13
Gambar 2.5 Visualisasi <i>Blackbox Testing</i>	13
Gambar 2.6 Instrumen Penilaian SUS	15
Gambar 2.7 <i>Entity Relationship Diagram</i> ERD	18
Gambar 3.1 Alur Penelitian.....	23
Gambar 3.2 Temuan Masalah Pooling Loop.....	26
Gambar 3.3 Arsitektur Otomatisasi Perangkat IoT	34
Gambar 3.4 Use Case Diagram Otomatisasi Perangkat IOT	35
Gambar 3.5 Activity Diagram Staff.....	36
Gambar 3.6 Activity Diagram Pengelola	37
Gambar 3.7 Activity Diagram Super Admin.....	38
Gambar 3.8 Desain ERD Subsistem Otomatisasi Perangkat IOT	39

DAFTAR TABEL

Tabel 2.1 Kajian Literatur Penelitian Terdahulu	6
Tabel 2.2 Daftar Pertanyaan SUS.....	14
Tabel 2.3 <i>Use Case Diagram Symbol</i>	16
Tabel 2.4 <i>Activity Diagram Symbol</i>	17
Tabel 3.1 Penjelasan Alur Penelitian	23
Tabel 3.2 Alat Peneltian	25
Tabel 3.3 Kebutuhan Bahan Penelitian	25
Tabel 3.4 Hasil Wawancara.....	27
Tabel 3.5 Daftar Perangkat Sensor	28
Tabel 3.6 Daftar Perangkat Aktuator	28
Tabel 3.7 Data Kebutuhan Otomatisasi	30
Tabel 3.8 Kebutuhan Fungsional	32
Tabel 3.9 Daftar Kebutuhan Non-Fungsional	33
Tabel 3.10 Daftar Desain Wireframe.....	39
Tabel 3.11 Rencana Pengujian Blackbox Testing	44
Tabel 3.12 Skenario Pengujian Komunikasi Data	45
Tabel 3.13 Pengujian Skenario Otomatisasi Perangkat IoT	46
Tabel 3.14 Gambaran Responden.....	49
Tabel 3.15 Skala Likert 1-5.....	49
Tabel 3.16 Simulasi Perhitungan 1 Responden SUS	49
Tabel 4.1 Implementasi Backend Subsistem Otomatisasi Perangkat IoT.....	63
Tabel 4.2 Hasil Implementasi	67
Tabel 4.3 Rekapitulasi Hasil Pengujian Blackbox Testing	75
Tabel 4.4 Rekapitulasi Hasil Pengujian Komunikasi Data dan Otomatisasi ..	76
Tabel 4.5 Pengujian Responsif Setiap Halaman.....	77
Tabel 4.6 Spesifikasi Lingkungan Produksi dan Status Akses.....	77
Tabel 4.7 Tahapan Review Hasil Implementasi	78
Tabel 4.8 Hasil Pengujian SUS	78

DAFTAR RUMUS

Rumus 2.1 Persentase Keberhasilan Blackbox Testing	14
Rumus 2.2 Skor Pertanyaan Ganjil.....	14
Rumus 2.3 Skor Pertanyaan Genap	15
Rumus 2.4 Perhitungan akhir SUS	15

DAFTAR KODE

Kode 4.1 Implementasi Event Broadcasting Sensor.....	57
Kode 4.2 Implementasi Client-side <i>WebSocket</i> Listener	58
Kode 4.3 Implementasi MQTT Subscriber	58
Kode 4.4 Pemrosesan Data Sensor	59
Kode 4.5 Evaluasi dan Eksekusi Otomatisasi Sensor	60
Kode 4.6 Implementasi Task Scheduler.....	60
Kode 4.7 Implementasi Job Queue untuk Aktuator.....	60
Kode 4.8 Pemrosesan Jadwal Otomatisasi	61
Kode 4.9 Implementasi Job untuk Menyalakan Aktuator.....	62
Kode 4.10 Evaluasi dan Eksekusi Otomatisasi Robot.....	63

BAB I

PENDAHULUAN

1.1 Latar Belakang

Peraturan Presiden No. 83 Tahun 2023 mengarahkan Kebun Raya di Indonesia untuk fokus pada tiga tugas utama: konservasi, penelitian, dan pendidikan [1]. Kebijakan ini sejalan dengan Undang Undang No. 11 Tahun 2019 yang mendorong pemanfaatan teknologi dalam kegiatan riset penelitian dan pengembangan inovasi [2]. Institut Teknologi Sumatera (ITERA) melalui Rencana Strategis 2025-2029 menargetkan peningkatan Kebun Raya ITERA untuk meraih akreditasi Kelas A dengan pemanfaatan teknologi digital sebagai pendukung proses bisnis dalam aspek operasional [3]. Kebun Raya ITERA sebagai pusat konservasi tumbuhan dan edukasi telah mengadopsi teknologi *Internet of Things* (IoT) pada *greenhouse* untuk mendukung operasionalnya [4]. Dengan perkembangan tersebut, kebutuhan akan sistem teknologi digital menjadi semakin penting agar pengelolaan *greenhouse* dapat berjalan lebih responsif dan terintegrasi terhadap kebutuhan konservasi maupun penelitian.

Riset Setyawan *et al.* menunjukkan tren peningkatan signifikan publikasi ilmiah tentang *smart greenhouse*, dari 4.8% pada 2017 menjadi 28.6% pada Agustus 2022, yang mengindikasikan pertumbuhan minat akademis terhadap teknologi di bidang pertanian [5]. Studi *systematic literature review* oleh Abou-mehdi-Hassani *et al.* memperkuat temuan ini dengan mengidentifikasi empat klaster riset *smart greenhouse* yang mencakup *Internet of Things* (IoT), *smart greenhouses*, *precision agriculture*, dan kombinasi *sustainability* dengan *Industry 4.0*, di mana IoT menjadi teknologi tulang punggung yang meningkatkan operasi *greenhouse* dan praktik pertanian [6]. Teknologi *smart greenhouse* menggunakan sensor dan aktuator untuk memantau dan mengontrol kondisi lingkungannya [5]. Platform *web* dipilih sebagai media monitoring karena sifatnya yang *cross-platform* dan tidak memerlukan instalasi tambahan, sehingga memberikan akses yang lebih luas dibandingkan aplikasi *native* [7].

Penelitian Nugraha (2023) telah berhasil meletakkan dasar digitalisasi yang solid pada *Greenhouse* Kebun Raya ITERA melalui pengembangan *website monitoring* dengan tingkat penerimaan pengguna yang baik, ditunjukkan oleh skor *System Usability Scale* (SUS) sebesar 75,83 [8]. Sebagai langkah pengembangan keberlanjutan, dilakukan penelusuran lebih lanjut terhadap struktur kode dan arsitektur sistem yang ada. Hasil investigasi menunjukkan bahwa mekanisme komunikasi data saat ini menerapkan metode *XHR Polling*. Mengacu pada studi komparasi performa oleh Appelqvist dan

Örnmyr (2017), pendekatan ini dinilai kurang efisien karena setiap permintaan HTTP membawa beban *header* sekitar 200 hingga 800 *bytes*, sangat kontras dengan protokol *WebSockets* yang hanya membutuhkan *overhead* 2 hingga 10 *bytes* per pesan setelah koneksi terbentuk [9]. Inefisiensi ini terkonfirmasi melalui pengujian empiris pada interval pesan 2000 ms, di mana *XHR Polling* mengonsumsi lalu lintas data pengiriman sebesar 49,5 KB/s, jauh lebih boros dibandingkan *WebSockets* yang hanya menggunakan 4,3 KB/s [9]. Selain itu, Kleppmann (2017) menyatakan bahwa pendekatan *polling* pada basis data dapat menciptakan beban komputasi yang tidak perlu (*overhead*) karena sistem terus memproses permintaan kosong saat tidak ada jadwal aktif [10]. Berdasarkan tinjauan tersebut, perancangan subsistem ini akan menerapkan arsitektur hibrida yang memadukan protokol *WebSockets* untuk efisiensi komunikasi data *real-time* serta mekanisme pemrosesan latar belakang asinkron (*asynchronous background processing*) guna menjamin eksekusi otomatisasi yang andal tanpa membebani kinerja utama sistem.

Pembaruan ini diperlukan sejalan dengan hasil wawancara bersama Ibu Zunanik Mufidah, S.TP., M.Si. selaku Penanggung Jawab *Greenhouse* Kebun Raya ITERA, yang mengharapkan adanya sistem kendali untuk menangani penjadwalan, pengaturan batas sensor, serta respons otomatis terhadap data robot. Pengembangan ini difokuskan untuk mengurangi keterlibatan manual pada tugas-tugas berulang dalam operasional *greenhouse*. Hal ini selaras dengan Suhandi *et al.*, yang menyatakan bahwa fungsi utama otomasi adalah mendukung efisiensi kegiatan operasional dengan mengurangi beban kerja manusia [11]. Secara teknis, pendekatan ini diperkuat oleh pandangan Abolhassani Khajeh *et al.*, yang menjelaskan bahwa penerapan IoT sebaiknya tidak hanya memantau, tetapi juga mampu mengaktifkan perangkat secara otomatis untuk merespons kondisi lingkungan [12]. Oleh karena itu, pengelolaan data IoT dan robot pada subsistem ini dirancang agar logika otomatisasi dapat dijalankan secara terstruktur dan konsisten sesuai parameter yang ditentukan.

Pengembangan subsistem ini menggunakan metode Agile Kanban setelah membandingkan berbagai pendekatan *Software Development Life Cycle* (SDLC). Model lama seperti *Waterfall* dianggap kurang cocok karena terlalu kaku terhadap perubahan kebutuhan [13]. Sementara itu, model *Spiral* yang fokus pada risiko dinilai terlalu rumit dan butuh banyak sumber daya untuk tim kecil [13]. Dalam kelompok metode *Agile*, *Scrum* memang terstruktur namun mewajibkan peran khusus dan waktu kerja yang kaku [14], [15], yang dimana kontras dengan *Kanban* yang lebih luwes dan mengalir terus-menerus tanpa perlu perencanaan yang berlebihan [14], [15], sehingga lebih pas untuk tim kecil *capstone* penelitian ini yang hanya berisi dua orang

yang masing masing memiliki fokus pengembangan yang berbeda dan membutuhkan hasil yang terukur jelas [16]. Karena alasan itu, *Kanban* dipilih sebab mampu menggambarkan alur kerja secara visual dan membatasi pekerjaan yang sedang berjalan atau *Work-In-Progress* (WIP) [15]. Cara ini mencegah tugas menumpuk dan memastikan hasil kerja siap pakai atau *production-ready* untuk sebuah produk [15].

Pengujian pada penelitian subsistem otomatisasi perangkat IoT yang mencakup pengendalian berdasarkan jadwal, ambang batas sensor, serta rekomendasi data robot dilakukan menggunakan aplikasi MQTT (*Message Queuing Telemetry Transport*) Explorer untuk pengujian komunikasi dan metode *Black-Box Testing* untuk pengujian fungsional, karena pendekatan ini dinilai efisien dari sisi waktu dan biaya serta memadai untuk memastikan perangkat *Internet of Things* (IoT) dapat berkomunikasi dan berfungsi sesuai kebutuhan sistem tanpa perlu menelaah kode program secara mendalam [8]. Aspek kepuasan pengguna terhadap subsistem diukur menggunakan kuesioner *System Usability Scale* (SUS), yang digunakan untuk memperoleh gambaran tingkat penerimaan dan kemudahan penggunaan subsistem secara menyeluruh dari sudut pandang pengguna [17]. Selanjutnya, sesuai dengan prinsip standar internasional mengenai *pengguna tertentu* serta kebutuhan penyelesaian masalah nyata dari mitra, penelitian ini menerapkan teknik *purposive sampling* dalam pemilihan responden, yaitu dengan melibatkan pihak-pihak yang memahami proses bisnis dan penggunaan sistem agar hasil pengujian yang diperoleh lebih relevan dan sesuai dengan konteks penerapan subsistem [18], [19], [20].

1.2 Rumusan Masalah

Latar belakang yang ada menunjukkan bahwa rumusan masalah dalam penelitian ini dapat dijabarkan sebagai berikut:

1. Bagaimana cara rancang bangun subsistem otomatisasi perangkat IoT berbasis *website* menggunakan metode Agile Kanban?
2. Bagaimana hasil pengujian subsistem otomatisasi perangkat IoT berdasarkan *Black-Box Testing*, MQTT Explorer, serta *usability* berdasarkan SUS?

1.3 Tujuan Penelitian

Latar belakang dan rumusan masalah memberikan dasar bahwa tujuan dari penelitian ini adalah sebagai berikut:

1. Menghasilkan rancangan dan implementasi subsistem otomatisasi perangkat IoT berbasis *website* dengan menerapkan Agile Kanban.

2. Mengukur tingkat keberhasilan fungsi, uji komunikasi, dan *usability* subsistem otomatisasi menggunakan metode *Black-Box Testing*, MQTT Explorer, dan *usability* menggunakan SUS.

1.4 Batasan Masalah

Penelitian ini dibatasi dengan batasan-batasan pengembangan sebagai berikut:

1. Penelitian difokuskan pada pengembangan subsistem otomatisasi perangkat *Internet of Things* (IoT) berbasis website yang mencakup penjadwalan, ambang batas sensor, dan rekomendasi data robot.
2. Penelitian tidak membahas perancangan dan detail teknis perangkat keras IoT serta mekanisme algoritma rekomendasi berbasis *Artificial Intelligence* (AI) pada robot.
3. Penelitian tidak mencakup analisis maupun perbandingan hasil tanaman yang dibudidayakan di dalam *greenhouse*.
4. Penerapan dan pengujian subsistem dibatasi pada *greenhouse* Kebun Raya ITERA dengan responden yang memiliki akses terhadap sistem.
5. Metode pengembangan dibatasi pada pendekatan *Agile Kanban*, sedangkan pengujian dilakukan menggunakan *Black-Box Testing*, MQTT Explorer, dan *System Usability Scale* (SUS).
6. Pengujian komunikasi dibatasi pada verifikasi jalur komunikasi MQTT menggunakan MQTT Explorer tanpa pengujian performa jaringan lanjutan.
7. Penelitian ini tidak melakukan perbandingan performa terhadap sistem yang dikembangkan pada penelitian sebelumnya.

1.5 Sistematika Penulisan

Sistematika penulisan penelitian ini disusun untuk memberikan gambaran umum mengenai struktur laporan tugas akhir sebagai berikut:

1.6 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat bilamana rumusan masalah terjawab dan tujuan tercapai, antara lain:

1. Tersedianya subsistem otomatisasi yang dapat membantu proses pengelolaan *greenhouse* menjadi lebih efisien melalui penerapan otomatisasi perangkat IoT berbasis *website*.
2. Memberikan data pengujian serta evaluasi mengenai kinerja fungsi otomatisasi dan tingkat kepuasan pengguna sebagai tolak ukur keberhasilan subsistem.

1.6.1 BAB I

Bab ini menjelaskan latar belakang, rumusan masalah, tujuan, batasan, manfaat penelitian, serta sistematika penulisan penelitian.

1.6.2 BAB II

Bab ini memaparkan tinjauan pustaka yang meliputi kajian teori dan penelitian terdahulu yang relevan dengan penelitian ini.

1.6.3 BAB III

Bab ini menjelaskan alur sistematis dan metodologi penelitian yang digunakan untuk menyelesaikan penelitian ini.

1.6.4 BAB IV

Bab ini memaparkan hasil pengembangan dan hasil pengujian sesuai dengan metodologi penelitian. Selanjutnya hasil akan dievaluasi dan dianalisis.

1.6.5 BAB V

Bab ini menjelaskan kesimpulan dari penelitian yang dilakukan, serta saran untuk pengembangan lebih lanjut.

BAB II

TINJAUAN PUSTAKA

2.1 Tinjauan Pustaka

Tabel 2.1 menyajikan rangkuman penelitian-penelitian sebelumnya yang relevan. Rangkuman ini mencakup metodologi, temuan, dan kontribusi dari setiap penelitian rujukan. Kajian ini digunakan sebagai landasan logis untuk penelitian ini. Penelitian ini memposisikan sebagai rancang bangun subsistem otomatisasi menggunakan metode agile kanban, pengujian menggunakan *black-box testing*, MQTT Explorer untuk uji komunikasi, dan evaluasi *usability* dengan SUS untuk pengujian secara menyeluruh terkait subsistem otomatisasi pada *website greenhouse* Kebun Raya ITERA.

Tabel 2.1 Kajian Literatur Penelitian Terdahulu

No.	Penulis [tahun][judul]	Permasalahan dan Tujuan	Metode	Hasil
1.	Nugraha [2023] Sistem <i>Monitoring Smart Greenhouse</i> berbasis IoT Dengan Menggunakan Metode Agile Kanban (Studi Kasus Greenhouse Melon ITERA) [8]	<i>Monitoring hardware IoT pada greenhouse spesifik tanaman melon melalui website;</i> bertujuan membangun <i>website</i> pemantauan berbasis IoT.	Agile Kanban, <i>Blackbox Testing</i> , dan SUS	Sistem <i>monitoring</i> berhasil dibangun dan diuji dengan hasil skor SUS 75.875 (kategori "Good").
2.	Morchid <i>et al.</i> [2024] <i>Agri-tech innovations for sustainability: A fire detection system based on MQTT broker and IoT to improve environmental risk management</i> [21]	Sistem deteksi kebakaran pertanian belum memiliki visualisasi data IoT real-time yang terpusat. Penelitian ini bertujuan memanfaatkan MQTT Explorer untuk memantau data sensor dan status sistem secara langsung.	<i>MQTT Explorer</i>	<i>MQTT Explorer</i> efektif untuk monitoring IoT <i>real-time</i> dengan akurasi 98,77% dan respons ± 2 detik.

No.	Penulis [tahun][judul]	Permasalahan dan Tujuan	Metode	Hasil
3.	Bagus Hardika <i>et al.</i> [2024] Merdekawan, A. P., & Sari, P. [2022] [Studi Awal Rancang Bangun <i>Indoor Farming Monitoring System</i> Berbasis IoT dengan Protokol Websocket]	Penelitian ini bertujuan mengembangkan sistem monitoring <i>indoor farming</i> berbasis IoT untuk mengatasi kendala lahan dan mobilitas tinggi masyarakat kota	Protokol <i>WebSocket</i> yang diuji fungsionalitasnya (<i>blackbox</i>) dan latensi jaringannya (<i>delay</i>)	Prototipe sukses menampilkan data real-time dengan delay pengiriman kategori bagus sebesar 0,028 detik.
4.	Sekar Sari dan Oktavia Citra Resmi Rachmawati [2025] <i>The Implementation of Agile Kanban in the Development of an IoT-Based Sugarcane Growth Monitoring System</i> [23]	Permasalahan utama adalah belum adanya sistem pemantauan lingkungan real-time untuk mendukung keputusan budidaya tebu; Tujuannya adalah mengembangkan sistem IoT pemantauan pertumbuhan tebu menggunakan Agile Kanban.	Pendekatan rekayasa eksperimental menggunakan Agile Kanban, melibatkan studi literatur, perencanaan, implementasi, dan analisis iteratif dengan <i>Kanban Board</i> .	Sistem pemantauan tebu IoT berhasil diselesaikan tepat waktu. Efisiensi kerja Kanban terbukti andal, di mana rata-rata satu tugas selesai dalam 2,5 hari, dan total 10 tugas tuntas setiap dua minggu.
5.	H. Y. Riskiawan <i>et al.</i> [2023] <i>Artificial Intelligence Enabled Smart Monitoring and Controlling of IoT-Green House</i> [24]	Masalah utama adalah kontrol <i>manual</i> rumah kaca; tujuannya mengembangkan sistem IoT-AI untuk otomasi.	Arsitektur <i>Laravel</i> sebagai pusat perintah terintegrasi perangkat <i>IoT</i> via protokol <i>MQTT</i> .	Memfasilitasi arsitektur <i>Smart Agent Based on Cloud</i> untuk mekanisme <i>Publish</i> data dan <i>Subscribe</i> otomasi.

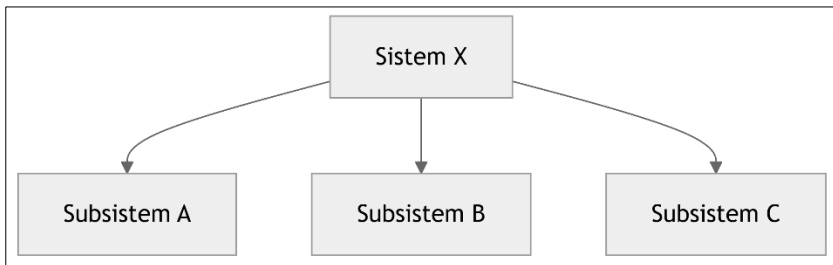
2.2 Dasar Teori

Sub-bab ini menguraikan landasan teori dan konsep-konsep fundamental yang menjadi acuan utama dalam pelaksanaan penelitian.

Pemaparan teori ini bertujuan untuk memberikan kerangka pemikiran yang sistematis serta memperkuat validitas metodologi yang diterapkan. Pembahasan mencakup tinjauan mendalam terhadap teknologi pengembangan sistem, arsitektur perangkat lunak, serta standar pengujian yang relevan guna mendukung analisis dan perancangan solusi yang diusulkan.

2.2.1 *Subsistem*

Subsistem didefinisikan sebagai bagian atau komponen yang terpadu dan saling keterkaitan di dalam sebuah sistem yang lebih besar (Lihat Gambar 2.1). Setiap subsistem harus mendapatkan perhatian penuh karena memiliki fungsi spesifik dan kinerjanya berkontribusi langsung pada pencapaian tujuan organisasi secara keseluruhan; misalnya, Sistem Akuntansi merupakan subsistem dari sistem organisasi. Konsep ini penting karena jika satu subsistem tidak berfungsi dengan baik, hal itu akan berdampak pada proses hingga hasil yang dikeluarkan oleh sistem induk, menekankan bahwa keberhasilan sistem bergantung pada kerjasama tim dan integrasi subsistemnya [25].



Gambar 2.1 Visualisasi Subsistem

2.2.2 *Website*

Website merupakan sekumpulan konten digital yang tersimpan dalam domain tertentu di internet, dirancang dengan tujuan spesifik dan saling berkaitan satu sama lain. Konten tersebut mencakup berbagai elemen multimedia seperti teks, gambar, video, dan berbagai bentuk media digital lainnya yang dapat diakses melalui peramban *web* menggunakan nama domain atau alamat IP yang unik [26]. Sebagai salah satu bentuk media informasi dan publikasi, situs *web* menawarkan kemudahan akses yang fleksibel tanpa batasan waktu dan lokasi geografis, sehingga dapat dimanfaatkan oleh berbagai pihak termasuk institusi pendidikan tinggi [27]. Dalam perspektif yang lebih luas, situs *web* dapat dipahami sebagai kumpulan halaman yang berada dalam jaringan *World Wide Web* (WWW) yang dapat diakses melalui koneksi internet [26].

2.2.3 *Greenhouse Kebun Raya ITERA*

Greenhouse atau rumah tanaman adalah bangunan tertutup yang berfungsi sebagai tempat budidaya tanaman dengan pengaturan kondisi lingkungan yang mendukung pertumbuhan optimal. Saat ini, *greenhouse* banyak dimanfaatkan untuk kegiatan penelitian karena memungkinkan penyesuaian suhu, kelembapan, dan pencahayaan sesuai kebutuhan eksperimen. Di Balai Penelitian dan Pengembangan Tanaman Pangan Kabupaten Pemalang, *greenhouse* digunakan sebagai laboratorium lapangan atau kebun percobaan yang menjadi fasilitas penelitian bagi berbagai jenis tanaman hortikultura, terutama sayur dan buah [28]. Gambar 2.2 menunjukkan *greenhouse* yang ada di Kebun Raya ITERA.



Gambar 2.2 Tampak Dalam dan Depan *Greenhouse* Kebun Raya ITERA

2.2.4 **IoT**

Internet of Things (IoT) adalah teknologi yang memungkinkan koneksi dan komunikasi antarperangkat melalui jaringan internet untuk mengotomatisasi aktivitas seperti penelusuran, pemrosesan, serta pengiriman data dalam sistem terintegrasi [29]. Arsitektur *IoT* menghubungkan perangkat keras dan lunak dalam satu ekosistem yang terdiri atas tiga komponen utama: sensor, *gateway*, dan *cloud* [30]. Sensor berfungsi mengumpulkan data lingkungan seperti suhu, kelembapan, nutrisi dari objek yang terhubung, sedangkan *gateway* bertugas meneruskan data tersebut ke basis data *cloud* melalui protokol komunikasi internet serta mengatur konektivitas perangkat secara otomatis. Data yang tersimpan di server *cloud* kemudian dapat diakses untuk pengendalian sistem *IoT* secara terpusat, sehingga mendukung interoperabilitas dan *remote monitoring* terhadap berbagai perangkat dalam jaringan [31].

2.2.5 **Protokol MQTT**

MQTT, singkatan dari *Message Queuing Telemetry Transport*, merupakan sebuah protokol komunikasi *machine-to-machine (M2M)* yang dirancang secara khusus agar sangat ringan untuk perangkat dalam ekosistem *Internet of Things (IoT)* [32]. Protokol ini sangat ideal untuk

diimplementasikan pada sistem *embedded*, seperti Raspberry Pi yang terpasang pada robot bergerak, karena dibangun untuk perangkat dengan keterbatasan daya dan *bandwidth*. MQTT mengadopsi model arsitektur *publish-subscribe* (*Pub/Sub*), yang memisahkan tanggung jawab antara pihak yang mengirim informasi (*publisher*) dengan pihak yang menerima pesan (*subscriber*) melalui perantara terpusat yang disebut broker [33].

Elemen-elemen inti yang wajib hadir untuk menjalankan protokol MQTT dalam suatu implementasi sistem [33]:

1. *Broker*

Broker bertindak sebagai pusat kendali dan perantara pesan, yang merupakan *server* utama yang bertanggung jawab menerima semua data yang dipublikasikan (*published*) dan kemudian mendistribusikan data tersebut kepada semua klien (*client*) yang telah mendaftar (*subscribe*) pada label (*topic*) yang relevan.

2. *Publisher*

Publisher merupakan klien atau perangkat yang memiliki peran mengirimkan data atau informasi spesifik ke label (*topic*) tertentu yang telah didefinisikan pada *broker*.

3. *Subscriber*

Merupakan klien atau perangkat yang berfungsi menerima atau menarik informasi dengan cara mendaftarkan diri (*subscribing*) ke label (*topic*) tertentu yang berada di *broker*.

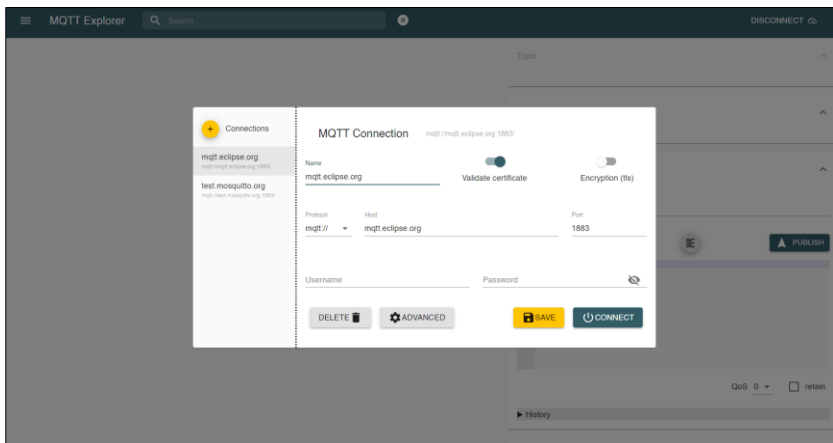
4. *Topic*

Dapat diibaratkan sebagai alamat digital atau label berformat *string* yang terorganisasi di dalam *broker*. *Topic* berfungsi sebagai wadah penyimpanan sementara untuk data yang dikirimkan oleh *publisher*.

2.2.6 MQTT Explorer

MQTT Explorer didefinisikan sebagai perangkat lunak klien MQTT (*Message Queuing Telemetry Transport*) yang berfungsi untuk memvisualisasikan, memantau, dan mengelola struktur topik serta muatan pesan (*payload*) dalam ekosistem *Internet of Things* (IoT) (Lihat Gambar 2.3) [34]. Aplikasi ini menyediakan antarmuka grafis yang menampilkan hierarki topik secara terstruktur sehingga memungkinkan pengembang untuk mengamati pertukaran data secara *real-time*, melakukan *debugging* sistem, serta memverifikasi integritas pengiriman pesan dari perangkat *publisher* ke *broker*. Selain itu, MQTT Explorer mendukung pengamatan mekanisme *Quality of Service* (QoS) yang digunakan dalam komunikasi MQTT, mencakup QoS level 0 (*at most once*), QoS level 1 (*at least once*), dan QoS level 2 (*exactly once*), sehingga memungkinkan evaluasi perilaku pengiriman

pesan terkait keandalan transmisi, potensi duplikasi data, dan latensi komunikasi [35]. Dalam penelitian implementasi sistem kendali IoT, MQTT Explorer dimanfaatkan sebagai instrumen validasi untuk memastikan bahwa parameter yang dikirimkan oleh mikrokontroler diterima pada sisi *broker* dan *subscriber* dengan format pesan serta tingkat QoS yang sesuai dengan perancangan sistem, sehingga membantu mengidentifikasi kesalahan konfigurasi protokol komunikasi selama tahap pengembangan [35].



Gambar 2.3 Tampilan Aplikasi MQTT Explorer

2.2.7 Perangkat IoT

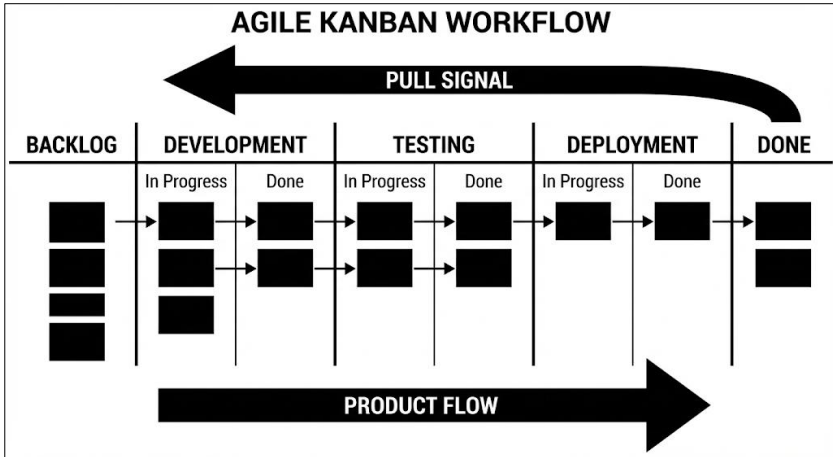
Perangkat *Internet of Things* (IoT) merupakan komponen perangkat keras fisik yang berfungsi sebagai jembatan interaksi antara dunia nyata dan sistem komputasi digital. Dalam arsitektur IoT, perangkat ini beroperasi pada lapisan persepsi (*perception layer*) yang terdiri dari dua elemen fundamental, yaitu sensor dan aktuator. Sensor berperan sebagai unit akuisisi data yang bertugas mendeteksi perubahan parameter fisik di lingkungan, seperti suhu, kelembaban, gerak, atau intensitas cahaya, untuk kemudian mengonversinya menjadi sinyal listrik atau data digital untuk diproses lebih lanjut. Sebaliknya, aktuator berfungsi sebagai unit eksekusi yang menerima sinyal kendali dari sistem untuk melakukan aksi mekanis atau fisik, seperti mengaktifkan motor, membuka katup, atau menyalakan alarm. Sinergi antara kemampuan penginderaan sensor dan kemampuan aksi aktuator inilah yang memungkinkan sistem IoT untuk melakukan pemantauan dan pengendalian lingkungan secara otonom dan *real-time* [36], [37].

2.2.8 Otomatisasi Perangkat IoT

Otomatisasi didefinisikan sebagai penerapan sistem dan proses teknologi untuk melaksanakan tugas dengan mengurangi intervensi manusia guna mencapai presisi, konsistensi, dan kecepatan operasional [11]. Dalam ekosistem *Internet of Things* (IoT), perangkat fisik saling terhubung untuk memungkinkan pertukaran data secara *real-time*, di mana sensor-sensor cerdas mampu melakukan tugas pemantauan dan eksekusi secara otonom [11]. Integrasi mendalam antara otomatisasi dan IoT ini berpotensi menciptakan sistem yang dapat mengatur dirinya sendiri (*self-regulating systems*) untuk mengoptimalkan operasi secara dinamis berdasarkan kondisi lingkungan terkini [11]. Mekanisme penjadwalan waktu nyata (*real-time scheduling*) menjadi komponen vital dalam arsitektur ini untuk memastikan perangkat merespons kebutuhan komputasi segera tanpa penundaan yang berarti [12]. Sinergi teknologi ini tidak hanya meningkatkan efisiensi, tetapi juga memberikan manfaat eksponensial yang melampaui kontribusi masing-masing teknologi jika diterapkan secara terpisah [11].

2.2.9 Agile Kanban

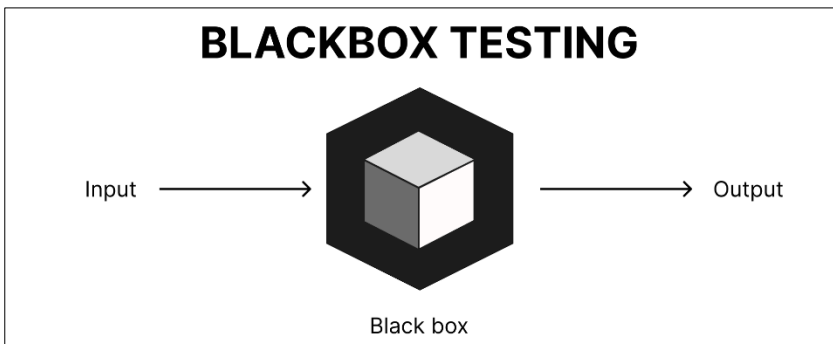
Agile Kanban mengadaptasi prinsip fleksibilitas Agile dengan fokus utama pada visualisasi alur kerja dan pengiriman berkelanjutan (*continuous flow*) tanpa batasan *sprint* yang kaku [15]. Seperti yang ditunjukkan pada Gambar 2.4, metode ini membagi proses pengembangan ke dalam tahapan spesifik, mulai dari *Backlog*, *Development*, *Testing*, hingga *Deployment*, yang di mana setiap kartu tugas bergerak secara linear mengikuti arah *Product Flow*. Mekanisme *Pull Signal* digunakan untuk menarik tugas dari kolom sebelumnya hanya ketika kapasitas tersedia, yang dikombinasikan dengan pembatasan *Work-In-Progress* (WIP) untuk mencegah *bottlenecks* dan menjamin transparansi penyelesaian tugas di setiap sub-kolom (*In progress* dan *Done*) [38]. Selain itu, penerapan standar *definition of done* memastikan bahwa setiap luaran fitur di kolom terakhir memiliki kualitas *production-ready* secara konsisten [23]. Dengan demikian, pendekatan ini mampu menawarkan efisiensi sumber daya sekaligus responsivitas yang tinggi terhadap perubahan prioritas [14].



Gambar 2.4 Papan dan Metode Agile Kanban [39]

2.2.10 Blackbox Testing

Blackbox Testing adalah teknik evaluasi perangkat lunak yang berfokus pada verifikasi respons sistem terhadap data masukan tanpa menganalisis arsitektur internal atau alur logika program (Lihat Gambar 2.5). Pendekatan ini dapat diimplementasikan untuk menguji aspek fungsional maupun *non-fungsional* sistem [8]. Penguji tidak diharuskan memiliki pemahaman mendalam mengenai struktur kode sumber yang digunakan dalam aplikasi. Melalui metode ini, dapat dilakukan deteksi anomali pada sistem serta validasi bahwa setiap fungsi operasional sesuai dengan kriteria dan spesifikasi yang telah ditetapkan sebelumnya [40]. Setelah dilakukan pengujian seluruh skenario, hasil akan di kalkulasi dengan Rumus 2.1.

Gambar 2.5 Visualisasi *Blackbox Testing*

$$\text{Persentase Keberhasilan} = \frac{\text{Jumlah Skenario yang Berhasil}}{\text{Jumlah Total Skenario}} \times 100\%$$

Rumus 2.1 Persentase Keberhasilan *Blackbox Testing* [41]

2.2.11 *System Usability Scale* (SUS)

System Usability Scale (SUS) adalah instrumen kuesioner ciptaan John Brooke yang dirancang untuk mengevaluasi kebergunaan sistem komputer berdasarkan persepsi subjektif pengguna. Instrumen ini menawarkan kemudahan interpretasi melalui sistem *skoring* numerik 0 hingga 100 dan bersifat praktis tanpa memerlukan komputasi statistik yang kompleks [42]. Selain dapat diakses secara gratis, SUS terbukti memiliki validitas dan reliabilitas yang kuat meskipun diterapkan pada sampel berukuran kecil [8]. Implementasinya dilakukan menggunakan 10 butir pernyataan standar untuk mengukur pengalaman pengguna terhadap perangkat lunak yang dikembangkan (Lihat Tabel 2.2). Berbagai keunggulan tersebut menjadikan metode ini sangat diandalkan dalam bidang evaluasi *usability* hingga saat ini.

Tabel 2.2 Daftar Pertanyaan SUS [42]

No.	Pertanyaan	Rentang Nilai				
		1	2	3	4	5
1.	Saya merasa akan sering menggunakan sistem ini.	○	○	○	○	○
2.	Saya merasa sistem ini memiliki kerumitan yang tidak perlu.	○	○	○	○	○
3.	Saya merasa sistem ini mudah untuk digunakan.	○	○	○	○	○
4.	Saya merasa perlu bantuan teknisi atau ahli untuk bisa mengoperasikan sistem ini.	○	○	○	○	○
5.	Saya melihat fungsi-fungsi dalam sistem ini saling terhubung dengan baik.	○	○	○	○	○
6.	Saya merasa ada terlalu banyak inkonsistensi dalam sistem ini.	○	○	○	○	○
7.	Saya membayangkan kebanyakan orang dapat mempelajari penggunaan sistem ini dengan cepat.	○	○	○	○	○
8.	Saya merasa sistem ini terasa canggung atau tidak nyaman digunakan.	○	○	○	○	○
9.	Saya merasa percaya diri ketika menggunakan sistem ini.	○	○	○	○	○
10.	Saya merasa perlu mempelajari banyak hal terlebih dahulu sebelum benar-benar dapat menggunakan sistem ini.	○	○	○	○	○

Catatan: 1 (Sangat tidak setuju) – 5 (Sangat setuju)

Pertanyaan yang nantinya akan di isi responden akan di kalkulasi dengan Rumus 2.2, 2.3, 2.4, serta berikut langkahnya [8]:

- 1) Pertanyaan nomor ganjil

$$\text{Skor Pertanyaan} = \text{Penilaian pengguna nomor ganjil} - 1$$

Rumus 2.2 Skor Pertanyaan Ganjil

- 2) Pertanyaan nomor genap

$$\text{Skor Pertanyaan} = 5 - \text{Penilaian pengguna nomor genap}$$

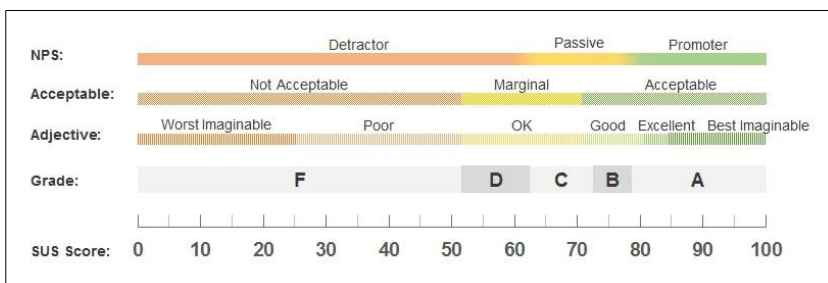
Rumus 2.3 Skor Pertanyaan Genap

3) Perhitungan Akhir

$$\text{Nilai SUS} = \frac{(\sum \text{Skor pertanyaan } 1 - 10) \times 2.5}{\text{Total Responden}}$$

Rumus 2.4 Perhitungan akhir SUS

Hasil perhitungan akhir akan di kalkulasi dan di nilai dengan standar instrumen penilaian subjektif pada Gambar 2.6 dan Gambar 2.7.



Gambar 2.6 Instrumen Penilaian SUS [42]

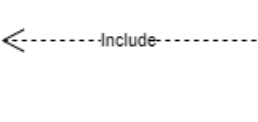
Grade	SUS	Percentile range	Adjective	Acceptable	NPS
A+	84.1-100	96-100	Best Imaginable	Acceptable	Promoter
A	80.8-84.0	90-95	Excellent	Acceptable	Promoter
A-	78.9-80.7	85-89		Acceptable	Promoter
B+	77.2-78.8	80-84		Acceptable	Passive
B	74.1 – 77.1	70 – 79		Acceptable	Passive
B-	72.6 – 74.0	65 – 69		Acceptable	Passive
C+	71.1 – 72.5	60 – 64	Good	Acceptable	Passive
C	65.0 – 71.0	41 – 59		Marginal	Passive
C-	62.7 – 64.9	35 – 40		Marginal	Passive
D	51.7 – 62.6	15 – 34	OK	Marginal	Detractor
F	25.1 – 51.6	2– 14	Poor	Not Acceptable	Detractor
F	0-25	0-1.9	Worst Imaginable	Not Acceptable	Detractor

Gambar 2.7 Detail Angka Penilaian SUS [43]

2.2.12 Diagram Use Case

Use case diagram merupakan diagram perilaku dalam *Unified Modeling Language* (UML) yang digunakan untuk memodelkan interaksi antara aktor dan sistem berdasarkan kebutuhan pengguna akhir [44]. Diagram ini menampilkan fungsi-fungsi utama yang dijalankan oleh aktor terhadap sistem, seperti pengguna atau administrator. Dengan menampilkan relasi antar unsur dan batas sistem, *use case diagram* memudahkan pemahaman mengenai peran serta interaksi yang terjadi dalam sistem secara menyeluruh. *Use case diagram* juga berfungsi sebagai alat komunikasi antara *developer* dan *stakeholder* untuk memastikan pemahaman yang sama terhadap fungsionalitas sistem [44]. Dalam konteks perancangan *interface*, diagram ini menjadi dasar untuk mengidentifikasi kebutuhan halaman dan alur navigasi yang harus dirancang. Penjelasan rinci mengenai elemen-elemen tersebut disajikan pada Tabel 2.3.

Tabel 2.3 *Use Case Diagram Symbol*

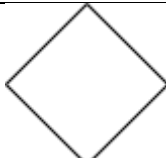
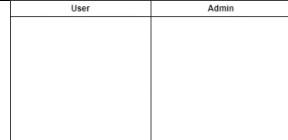
No	Simbol	Nama simbol	Fungsi
1.		<i>Use Case</i>	Mewakili bagian dari fungsionalitas sistem dan ditempatkan dalam batasan sistem.
2.		Aktor	Menggambarkan tokoh atau sistem yang berhubungan dengan sistem.
3.		Asosiasi	Menghubungkan aktor dengan <i>use case</i> atau dengan <i>use case</i> lain nya.
4.		<i>Generalisasi</i>	Menunjukkan generalisasi dari <i>use case</i> khusus ke umum.
5.		<i>Include</i>	Hubungan antar <i>use case</i> tambahan dan <i>use case</i> lain nya.

No	Simbol	Nama simbol	Fungsi
6.		<i>Extend</i>	Hubungan <i>use case</i> dengan <i>use case</i> tambahan yang berdiri sendiri.

2.2.13 Diagram Aktivitas

Activity diagram merupakan diagram perilaku dalam *Unified Modeling Language* (UML) yang digunakan untuk memodelkan alur aktivitas atau proses kerja dalam sistem. Diagram ini mengurutkan kegiatan pada proses bisnis atau sistem informasi, baik yang linier maupun bercabang, sehingga memudahkan perancangan prosedur dan identifikasi hambatan dalam alur kerja [44]. Melalui representasi visual yang jelas, *activity diagram* membantu memahami logika proses secara sistematis. Dalam perancangan *user interface*, diagram ini berperan penting untuk memvisualisasikan alur interaksi pengguna mulai dari titik awal hingga pencapaian tujuan akhir [44]. Penjabaran lebih lanjut mengenai elemen diagram ini tercantum pada Tabel 2.4.

Tabel 2.4 *Activity Diagram Symbol*

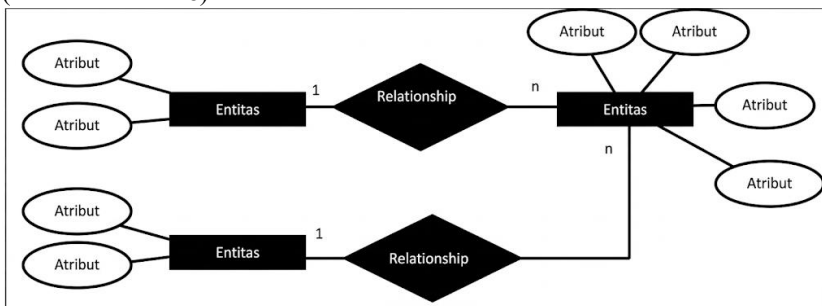
No	Simbol	Nama simbol	Fungsi
1		Mulai	Menunjukkan simbol awal dari aktivitas sistem.
2		Aktivitas	Menggambarkan kegiatan yang dijalankan oleh sistem.
3		Transisi	Memperlihatkan alur proses yang berlangsung.
4		Kondisi	Menjelaskan kondisi bercabang apabila terdapat lebih dari satu kemungkinan keadaan.
5		<i>Swimlane</i>	Mengklasifikasikan jenis tugas atau aktivitas yang dilakukan.
6		Akhir	Menandakan pula simbol akhir yang menunjukkan

No	Simbol	Nama simbol	Fungsi
			berakhimya suatu aktivitas dalam sistem

2.2.14 Entity Relationship Diagram

Entity Relationship Diagram (ERD) merupakan diagram visual yang dibuat untuk mendeskripsikan data atau objek-objek berdasarkan kondisi dunia nyata yang disebut sebagai entitas. Diagram ini secara spesifik berfungsi menggambarkan hubungan (*relationship*) yang terjalin antar entitas tersebut dengan menggunakan serangkaian untuk memvisualisasikan struktur data secara jelas [45]. Manfaat utama dari penerapan diagram ini adalah untuk mempermudah proses perancangan desain basis data dengan memetakan objek-objek nyata ke dalam skema logis sistem. Dalam proses pengembangan perangkat lunak, ERD berperan sebagai cetak biru yang memastikan bahwa seluruh kebutuhan penyimpanan informasi dapat terakomodasi dengan baik sebelum tahap implementasi dilakukan [45].

ERD memvisualisasikan data melalui tiga komponen utama: Entitas (objek penting, dilambangkan kotak), Atribut (detail dari entitas, dilambangkan oval), dan Relasi (hubungan antar entitas, dilambangkan belah ketupat). Komponen-komponen ini dihubungkan oleh Garis Penghubung, yang juga dilengkapi dengan indikator Kardinalitas (seperti 1 atau n) untuk menjelaskan seberapa banyak *instance* dari satu entitas dapat berinteraksi dengan entitas lainnya, sehingga menghasilkan skema basis data yang logis (Lihat Gambar 2.8).



Gambar 2.8 Entity Relationship Diagram (ERD) [45]

2.2.15 Purposive Sampling

Sampling bertujuan (*purposive sampling*) adalah sebuah metode pengambilan sampel *non-probabilitas* di mana peneliti secara sengaja memilih partisipan berdasarkan penilaian (*judgment*) dan tujuan spesifik dari penelitian [19]. Berbeda dari sampling acak (*probability sampling*) yang mengejar generalisasi statistik, *purposive sampling* digunakan untuk mengidentifikasi individu atau kasus yang "kaya informasi" (*information-rich*) dan paling sesuai

untuk berkontribusi pada pemahaman mendalam tentang fenomena yang diteliti. Teknik ini sering digunakan, baik dalam penelitian kualitatif maupun kuantitatif, ketika peneliti menargetkan individu yang memiliki pengetahuan khusus, pengalaman mendalam, atau memegang peran/posisi tertentu yang relevan dengan fokus studi [20]. Salah satu variasi dari teknik ini adalah *Total Population Sampling* (Sampling Populasi Total), yang diterapkan ketika populasi yang dituju berukuran kecil, memiliki karakteristik unik, dan terdefinisi dengan jelas [20]. Dalam pendekatan ini, seluruh anggota populasi tersebut dilibatkan sebagai responden (Tertera pada Lampiran F), yang bertujuan untuk mengeliminasi bias sampling dan memberikan pemahaman yang komprehensif terhadap kelompok spesifik tersebut [19].

2.2.16 Websocket

Websocket dapat dipahami sebagai sebuah mekanisme komunikasi yang memungkinkan browser dan server saling bertukar data secara langsung dan terus-menerus melalui satu jalur koneksi yang tetap terbuka. Berbeda dengan komunikasi berbasis HTTP biasa, yang selalu mengharuskan klien mengirim permintaan terlebih dahulu sebelum server memberikan jawaban, *Websocket* menghilangkan kebutuhan akan proses tersebut. Setelah koneksi awal dibuka dan disetujui oleh server, keduanya dapat mengirim data kapan saja tanpa harus memulai ulang siklus komunikasi. Cara kerja yang langsung ini membuat pertukaran informasi berlangsung lebih cepat dan stabil. Selain itu, data yang dikirim melalui *Websocket* dikemas dengan cara yang sederhana dan ringan sehingga tidak membebani jaringan. Karena itulah, secara teoritis *Websocket* mampu memberikan kinerja yang lebih efisien dibanding metode komunikasi lain seperti *polling* atau *long polling*, terutama pada aplikasi yang memerlukan pembaruan data secara *real-time*, misalnya sistem monitoring, aplikasi chat, atau notifikasi langsung [46].

2.2.17 Laravel

Laravel adalah kerangka kerja pengembangan aplikasi web modern yang menyediakan berbagai alat canggih untuk membangun sistem, termasuk fitur-fitur khusus untuk menangani komunikasi data yang kompleks [47]. Yang digunakan pada penelitian ini adalah Laravel Reverb, Laravel Horizon, Laravel Notifications. Laravel Reverb berfungsi sebagai jalur komunikasi super cepat yang memungkinkan aplikasi memperbarui data di layar pengguna secara langsung (*real-time*) [48]. Sementara itu, Laravel Horizon bertindak sebagai pusat kendali cerdas yang memantau dan mengatur pekerjaan berat di belakang layar [49]. Untuk menjaga pengguna tetap terinformasi, Laravel Notifications bertugas mengirimkan pesan penting atau peringatan melalui berbagai saluran seperti email atau aplikasi pesan secara otomatis [50]. Ketika digabungkan, Reverb dan Horizon memastikan notifikasi dapat muncul seketika di layar

pengguna sementara proses pengirimannya dikelola dengan baik agar tidak memberatkan kinerja aplikasi [48], [49].

2.2.18 Redis

Redis (*Remote Dictionary Server*) didefinisikan sebagai penyimpanan struktur data *in-memory* jarak jauh yang bersifat *open-source* dan berkinerja tinggi, yang beroperasi dengan mekanisme pemetaan kunci ke berbagai jenis struktur data seperti *strings*, *lists*, *sets*, *hashes*, dan *sorted sets*. Berbeda dengan basis data tradisional yang menyimpan data pada *hard disk* dengan latensi akses tinggi, Redis menyimpan data di dalam memori utama (RAM) yang secara teoritis 150.000 kali lebih cepat, menjadikannya solusi efektif untuk mengatasi masalah *bottleneck* pada kinerja pembacaan (*read performance*) sistem. Selain kecepatan, Redis menawarkan fitur persistensi data ke disk, replikasi untuk redundansi, serta mendukung *horizontal scaling* yang lebih ekonomis dibandingkan *vertical scaling* untuk menangani beban *throughput* yang besar [51].

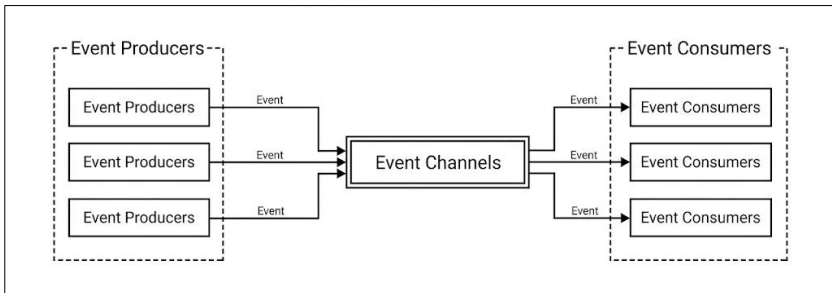
2.2.19 XHR Polling

XHR Polling merupakan mekanisme komunikasi konvensional di mana *client* secara berkala mengirimkan permintaan HTTP ke *server* pada interval waktu yang telah ditentukan untuk memvalidasi ketersediaan data baru. Metode ini memiliki kelemahan mendasar berupa inefisiensi penggunaan *bandwidth* dan beban kerja *server* yang berlebihan, karena koneksi harus terus dibangun berulang kali meskipun tidak terdapat pembaruan informasi yang signifikan untuk dikirimkan kembali. Latensi penerimaan data menjadi konsekuensi yang tidak terhindarkan karena klien harus menunggu siklus permintaan berikutnya untuk mendapatkan status terkini. Kondisi ini menjadikan metode *polling* kurang reliabel untuk diterapkan pada arsitektur sistem yang menuntut standar responsivitas waktu nyata [9].

2.2.20 Event Driven

Arsitektur *Event Driven* adalah paradigma sistem yang mengandalkan produksi dan deteksi peristiwa (*events*) sebagai pemicu utama alur eksekusi program secara asinkron [52]. Dalam pendekatan ini, ketika suatu peristiwa terjadi di dalam atau di luar sistem, peristiwa tersebut segera didistribusikan kepada semua pihak yang tertarik dan memicu tindakan yang sesuai. Model komunikasi berbasis *event* memfasilitasi pemisahan ketergantungan (*decoupling*) antar komponen, di mana produsen pesan tidak perlu mengetahui konsumen spesifik dari pesan tersebut, sehingga memungkinkan komponen beroperasi secara independen [52]. Karakteristik *loose coupling* ini

memberikan fleksibilitas dalam menangani perubahan sistem tanpa memerlukan modifikasi pada komponen produsen pesan.



Gambar 2.9 Ilustrasi Arsitektur *Event-Driven*

Gambar 2.9 mengilustrasikan mekanisme kerja mendasar dalam arsitektur *event-driven*. Pada paradigma ini, produksi dan pendeteksian peristiwa (*events*) berfungsi sebagai pemicu utama alur eksekusi program yang berjalan secara asinkron. Model komunikasi tersebut memfasilitasi pemisahan ketergantungan (*decoupling*) antar komponen, yang memungkinkan bagian produsen dan konsumen pesan untuk beroperasi secara independen.

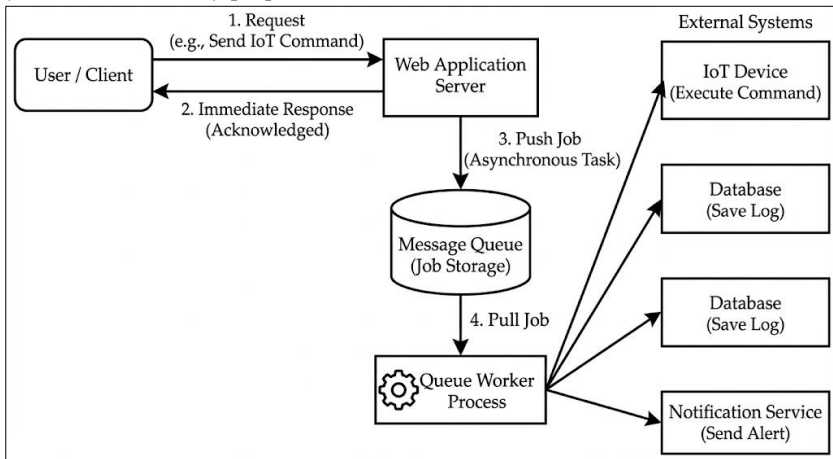
2.2.21 *Cron Scheduler*

Cron scheduler adalah mekanisme penjadwalan otomatis yang berasal dari sistem Unix/Linux. Fungsinya adalah menjalankan perintah atau *script* pada waktu-waktu tertentu yang telah ditentukan sebelumnya, misalnya setiap jam, setiap hari, atau pada waktu spesifik seperti jam 6 pagi. Dalam sistem otomatisasi berbasis web, *cron scheduler* berfungsi sebagai pemicu (*trigger*) untuk mengecek aturan otomatis yang disimpan di database. Alurnya sederhana: (1) *scheduler* menjalankan perintah pada waktu yang dijadwalkan, (2) perintah tersebut membaca aturan otomatis dari database untuk melihat mana yang harus dijalankan, (3) perintah menciptakan tugas (*job*) yang akan dijalankan kemudian. Keuntungan pemisahan ini adalah *scheduler* hanya bertugas memicu, sementara eksekusi sesungguhnya ditangani oleh sistem lain (*queue worker*). Dengan cara ini, konfigurasi jadwal dapat diubah tanpa perlu menghentikan *scheduler* [53].

2.2.22 *Queue Worker*

Queue worker adalah sistem yang menangani pekerjaan-pekerjaan berat secara terpisah dan tidak sinkron (*asynchronous*). Sistem ini menyimpan pekerjaan dalam antrian (*queue*), kemudian *worker* mengambil satu pekerjaan, mengerjakannya sampai selesai, dan menandainya sebagai selesai atau gagal. Dalam sistem otomatisasi IoT, *queue worker* digunakan untuk menangani operasi yang memakan waktu atau tergantung pada sumber eksternal, seperti

mengirim perintah ke perangkat IoT, menyimpan *log* ke *database*, atau mengirim notifikasi. Keuntungan utama adalah aplikasi *web* tidak perlu menunggu operasi-operasi ini selesai, namun aplikasi dapat langsung merespons pengguna, sementara pekerjaan berat dikerjakan di belakang layar oleh *worker*. Jika pekerjaan gagal, sistem dapat secara otomatis mencoba lagi (Lihat Gambar 2.10) [54].



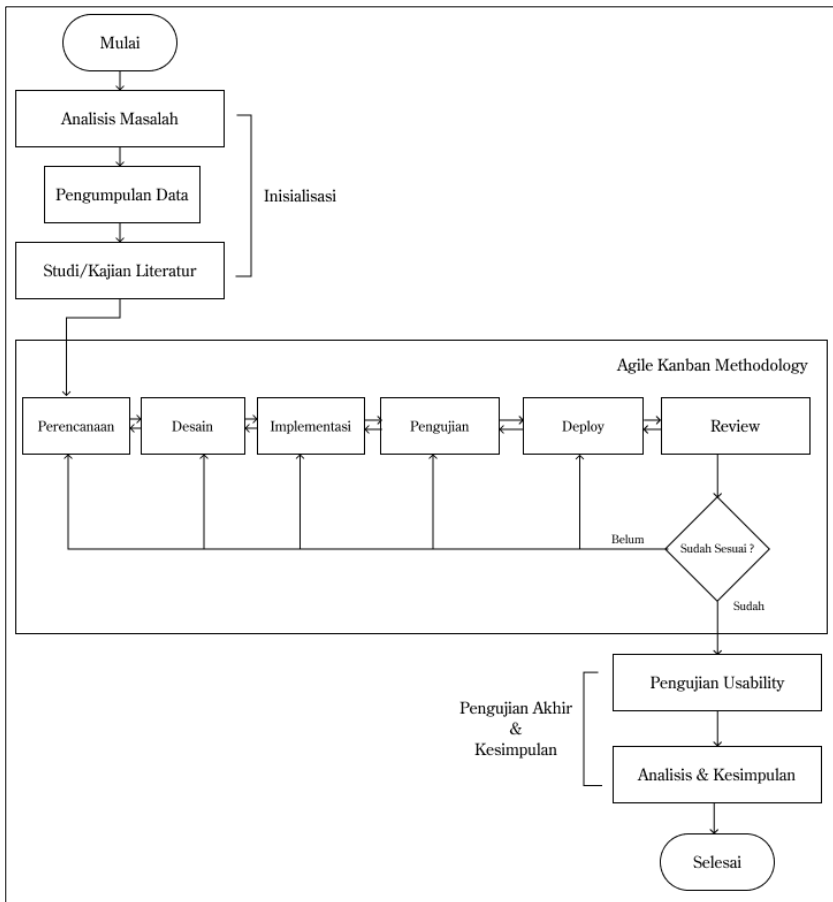
Gambar 2.10 Ilustrasi *Queue Worker*

BAB III

METODE PENELITIAN

3.1 Alur Penelitian

Alur penelitian merupakan rangkaian tahapan sistematis yang dilakukan mulai dari perencanaan hingga penyelesaian penelitian. Gambar 3.1 menunjukkan visualisasi alur dalam bentuk *flowchart*, sedangkan Tabel 3.1 berisi penjelasan singkat dari setiap tahapan (Tahapan detail seluruh alur penelitian terdapat pada sub-bab 3.3).



Gambar 3.1 Alur Penelitian

Tabel 3.1 Penjelasan Alur Penelitian

Tahapan	Proses	Deskripsi
Inisialisasi	Analisis Masalah	Mengidentifikasi inefisiensi komunikasi data metode <i>XHR Polling</i> pada sistem lama serta memetakan kebutuhan

Tahapan	Proses	Deskripsi
		subsistem otomatisasi (penjadwalan, ambang batas sensor, dan rekomendasi data robot) berdasarkan masukan Penanggung Jawab Greenhouse.
	Pengumpulan Data	Mengumpulkan data teknis dan operasional melalui wawancara mendalam dengan mitra dan observasi langsung di Greenhouse Kebun Raya ITERA.
	Studi/Kajian Literatur	Mengkaji landasan teori terkait IoT, protokol komunikasi <i>real-time</i> (MQTT & <i>Websocket</i>) serta manajemen proses asinkron (<i>Queue & Scheduler</i>), serta metode pengembangan Agile Kanban dan standar pengujian yang relevan.
Agile Kanban Metodologi (Detail implementasi terdapat pada Lampiran D)	Perencanaan	Mengidentifikasi kebutuhan fungsional dan non-fungsional dari subsistem <i>website</i> otomatisasi perangkat IOT berdasarkan analisis masalah.
	Desain	Merancang arsitektur komunikasi, <i>use case diagram</i> , <i>activity diagram</i> , ERD, dan <i>wireframe</i> sebagai <i>blueprint</i> arsitektur subsistem.
	Implementasi	Tahap mengembangkan <i>frontend</i> dan <i>backend</i> sesuai desain yang dibuat.
	Pengujian	Melakukan pengujian <i>blackbox testing</i> dan MQTT Explorer dari setiap fitur yang dikembangkan pada subsistem.
	<i>Deploy</i>	Menjalankan sistem ke lingkungan operasional agar dapat digunakan pengguna.
	<i>Review oleh penanggung jawab penelitian di greenhouse Kebun Raya ITERA</i>	Melakukan evaluasi hasil pengembangan dengan <i>penanggung jawab greenhouse</i> Kebun Raya ITERA untuk memastikan kesesuaian dengan kebutuhan.
Pengujian Akhir & Kesimpulan	Pengujian <i>Usability</i>	Mengukur tingkat kebergunaan sistem menggunakan metode <i>System Usability Scale</i> (SUS) kepada <i>stakeholder</i> serta pengguna terkait.
	Analisis & Kesimpulan	Menganalisis hasil pengujian berdasarkan hasil <i>Blackbox Testing</i> , Uji Komunikasi, serta <i>usability</i> dengan SUS.

3.2 Alat dan Bahan

Penelitian ini menggunakan sejumlah alat dan bahan pendukung yang spesifikasinya diuraikan secara rinci pada sub-bab 3.2.1 Alat Penelitian dan sub-bab 3.2.2 Bahan Penelitian.

3.2.1 Alat Penelitian

Penelitian ini menggunakan perangkat keras dan perangkat lunak dengan spesifikasi yang disesuaikan dengan kebutuhan analisis, perancangan, dan implementasi. Rincian alat penelitian disajikan pada Tabel 3.2.

Tabel 3.2 Alat Penelitian

Nama Alat	Spesifikasi	Keterangan
<i>Laptop</i>	Laptop AMD Ryzen 4600H, RAM 16 GB, SSD 512 GB, OS Windows 11	Perangkat utama peneliti untuk pengembangan dan pengujian subsistem <i>website</i>
<i>Visual Studio Code</i>	Version 1.105.1	Editor kode untuk implementasi pengembangan subsistem
<i>Figma & Draw.io</i>	Figma : Versi 125.10.5 Draw.io : <i>Website</i>	Perancangan desain <i>Use Case</i> , ERD, <i>activity diagram</i> , dan <i>wireframe</i>
<i>Browser</i>	Chrome, Brave, dll	Kebutuhan pengembangan dan tahap pengujian subsistem <i>website</i> otomatisasi perangkat IOT
MQTT Explorer	0.4.0-beta.6	Sebagai alat pengujian komunikasi subsistem ke perangkat IoT atau sebaliknya.

3.2.2 Bahan Penelitian

Bahan penelitian yang digunakan mencakup data dari wawancara penanggung jawab *greenhouse* Kebun Raya ITERA, dokumentasi *website*, serta data yang diperoleh selama proses desain, implementasi, pengujian dan analisis kesimpulan. Rincian bahan penelitian disajikan pada Tabel 3.3.

Tabel 3.3 Kebutuhan Bahan Penelitian

Nama Bahan	Sumber	Kegunaan
Dokumentasi Sistem Terdahulu	Laporan dan kode program penelitian sebelumnya [8].	Dasar analisis untuk mengidentifikasi inefisiensi komunikasi data (XHR <i>Polling</i>) yang menjadi urgensi perancangan ulang subsistem.
Data Kebutuhan Pengguna	Data wawancara dari penanggung jawab <i>greenhouse</i> Kebun Raya ITERA. (Lihat Tabel 3.4)	Memetakan kebutuhan fungsional pengguna terkait permasalahan, alur kerja operasional, dan aturan logika otomatisasi di <i>greenhouse</i> .
Referensi Literatur & Teknis	Jurnal ilmiah, buku, dan dokumentasi resmi (<i>official documentation</i>).	Landasan teoritis penerapan teknologi komunikasi <i>real-time</i> (Websocket/MQTT), manajemen proses asinkron (<i>Queue</i>), metode <i>Agile Kanban</i> , dan standar pengujian.
Data Perangkat IoT dan Aturan Otomatisasi	Kebutuhan otomatisasi dari wawancara <i>penanggung jawab</i> , dan salah satu pengelola pada <i>greenhouse</i> Kebun Raya ITERA.	Menentukan angka ambang batas (<i>threshold</i>) sensor, aturan waktu penjadwalan, dan logika respon terhadap data robot sebagai inti algoritma kendali otomatis.
Data pengujian	Proses pengujian dengan <i>Black Box Testing</i> , MQTT Explorer, dan kuesioner SUS.	Bukti empiris untuk memvalidasi keberhasilan fungsi fitur, stabilitas komunikasi data, dan tingkat

Nama Bahan	Sumber	Kegunaan
		penerimaan pengguna terhadap sistem baru.

3.3 Tahapan Seluruh Alur Penelitian

Pembahasan lengkap alur penelitian dalam penelitian ini terdiri dari tiga tahap utama, yaitu Inisialisasi, Tahapan Agile Kanban, dan Analisis Kesimpulan. Setiap tahap memiliki peran penting dalam memastikan proses penelitian berjalan secara sistematis dan terarah. Uraian dari masing-masing tahap dijelaskan dengan rinci pada 3.3.1 – 3.3.4.

3.3.1 Inisialisasi

1. Analisis Masalah

Analisis kode sistem terdahulu (Lihat Gambar 3.2 dan 3.3) mengidentifikasi inefisiensi metode *XHR Polling* yang menghasilkan trafik data 49,5 KB/s dan beban CPU 4,11%, jauh melampaui *Websockets* yang hanya membutuhkan 4,3 KB/s dan CPU 0,26% [46]. Kesenjangan performa ini disebabkan oleh *overhead* HTTP sebesar 800 bytes per permintaan dibandingkan *Websockets* yang hanya berkisar 2 hingga 10 bytes. Oleh sebab itu, transisi menuju integrasi protokol komunikasi *real-time* dan manajemen proses asinkron diterapkan guna mengeliminasi beban server dan menjamin responsivitas perangkat IoT [46].

```
const getValueRefreshFirst = async () => {
  setTimeout(() => {
    axios.get(`${brokerSensor}/${idSensor}`) // Request XHR
    .then(response => {
      setValueSensor(response.data.data[0].value)
      setStatus(response.data.data[0].status)
      setTime(response.data.data[0].createdAt)
      setOnRefresh(true) // Memicu useEffect untuk loop selanjutnya
    })
  }, 1000) // Interval 1 detik
}

const getValueRefreshSecond = async () => {
  setTimeout(() => {
    axios.get(`${brokerSensor}/${idSensor}`) // Request XHR
    .then(response => {
      setValueSensor(response.data.data[0].value)
      setStatus(response.data.data[0].status)
      setOnRefresh(false) // Memicu useEffect untuk loop selanjutnya
    })
  }, 1000) // Interval 1 detik
}
```

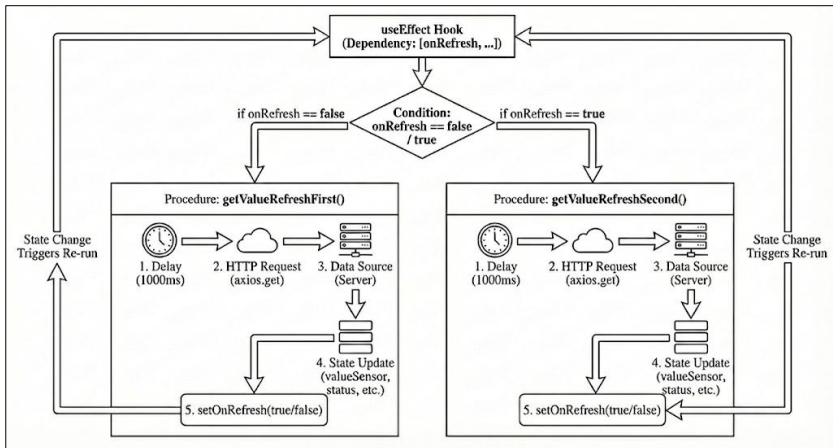
(a) Fungsi Eksekusi Polling (Request & Delay)

```
const onRefreshUpdate = () => {
  // logic ping-pong: jika false panggil First, jika true panggil Second
  if(onRefresh === false){
    getValueRefreshFirst()
  }
  if (onRefresh === true){
    getValueRefreshSecond()
  }
}

useEffect(() => {
  if(firstCheck === true){
    getValue()
  }
  else{
    onRefreshUpdate() // Dipanggil setiap kali state 'onRefresh' berubah
  }
  return () => setFirstCheck(false);
  // Dependency array ini menyebabkan loop tak terbatas (Polling)
  // karena onRefresh selalu berubah di dalam fungsi getValueRefresh...
}, [onRefresh, firstCheck, valueSensor, status]);
```

(b) Logika Pengulangan (Looping Logic)

Gambar 3.2 (a) dan (b) Masalah *XHR Polling* Code Sensor [8]



Gambar 3.3 Alur Logika Mekanisme XHR Polling

Gambar 3.3 memvisualisasikan alur kerja sistem terdahulu dengan mekanisme *XHR Polling*, di mana *hooks useEffect* memicu permintaan data berulang setiap 1000 milidetik. Logika ini memaksa aplikasi terus mengirimkan *HTTP Request* ke *server* untuk memeriksa pembaruan, tanpa memedulikan kondisi data aktual di lapangan. Siklus tersebut mengakibatkan inefisiensi kinerja karena membebani proses komputasi serta meningkatkan konsumsi *bandwidth* akibat pengiriman *header* berulang yang seharusnya dapat dihindari.

2. Pengumpulan Data

Pengumpulan data dilakukan untuk mengidentifikasi spesifikasi teknis dan logika operasional yang diperlukan dalam pengembangan subsistem otomatisasi perangkat IoT. Proses ini dilaksanakan melalui observasi infrastruktur sensor dan aktuator di lapangan serta wawancara dengan pengelola (Lihat Tabel 3.4, detail instrumentasi pada Lampiran B) guna merumuskan skenario kendali otomatis yang presisi sesuai kebutuhan mitra. Selain itu, dokumentasi sistem terdahulu dikaji sebagai acuan standar integrasi arsitektur komunikasi. Seluruh rincian parameter data yang menjadi landasan analisis dan perancangan sistem dijabarkan pada Tabel 3.5, Tabel 3.6, dan Tabel 3.7.

Tabel 3.4 Hasil Wawancara

No.	Ringkasan Poin Jawaban
1.	Sistem lama membantu, tetapi respons perangkat sering lambat.
2.	Pengaturan suhu, penyiraman, dan pemupukan berjalan terpisah.
3.	Notifikasi sering terlambat dan kurang informatif.
4.	Tampilan antarmuka kurang modern dan tidak konsisten di HP.
5.	Diperlukan penjadwalan otomatis untuk penyiraman dan pengadukan tandon.
6.	Sistem lama tidak stabil saat server berat sehingga jadwal kadang telat.
7.	<i>Greenhouse</i> berencana mengintegrasikan robot pemantau tanaman.

No.	Ringkasan Poin Jawaban
8.	Pengguna membutuhkan notifikasi keberhasilan/gagal otomatisasi dan informasi stok nutrisi.
9.	Pengguna ingin mengetahui perangkat mana yang menyala, kapan menyala, dan siapa yang mengubah aturan.

Tabel 3.5 Daftar Perangkat Sensor

No.	[Kode] Nama + Gambar Sensor	Deskripsi Singkat dan Fungsi
1.	[S-01] Suhu dan Kelembapan 	Tipe: Temperature and Humidity Transmitter Modbus SHT20 Sensor XY-MD02 - XY-MD02 Mengukur suhu udara dan kelembapan relatif di dalam <i>greenhouse</i> secara <i>real-time</i> untuk pemantauan iklim mikro.
2.	[S-02] Total Dissolved Solids (TDS) 	Tipe: MODBUS RTU RS485 0 - 2000 PPM Membaca dan menganalisis konsentrasi nutrisi terlarut (TDS).
3.	[S-03] pH 	Tipe: DFRobot : Analog pH Sensor Meter Kit V2 Membaca tingkat keasaman (pH) larutan untuk mengontrol komposisi hara.
4.	[S-04] Ultrasonik Tandon Air 	Tipe: Waterproof Ultrasonic Sensor - A02YYUW - UART Mengukur level ketinggian permukaan air baku dalam tandon menggunakan gelombang ultrasonik untuk estimasi volume tersedia.
5.	[S-05] Ultrasonik Nutrisi Pupuk 	Tipe: Waterproof Ultrasonic Sensor - A02YYUW - UART Memantau ketinggian stok larutan konsentrat pupuk (Makro A dan B) dalam tandon untuk memastikan ketersediaan bahan baku.

Tabel 3.6 Daftar Perangkat Aktuator

No.	[Kode Nama + Gambar Aktuator	Deskripsi Singkat dan Fungsi
1.	[A-01] Cooling Pad 	Perangkat ini bekerja dengan memanfaatkan proses penguapan air saat udara melewati media basah. Proses tersebut menyerap panas dari udara lingkungan, sehingga menurunkan suhu dan meningkatkan kelembapan udara sebelum masuk ke area tanam.
2.	[A-02] Exhaust Fan 	Berfungsi menarik udara panas dari dalam <i>greenhouse</i> ke luar ruangan. Mekanisme ini meningkatkan pergantian udara dan membantu menurunkan suhu di dalam ruangan.
3.	[A-03] (2) → Agrigator Pencampuran  Keterangan: 1 TANDON PENCAMPURAN: Berfungsi sebagai tempat penyimpanan larutan nutrisi A, nutrisi B, dan Air. 2 AGITATOR PENCAMPURAN: Alat yang berfungsi untuk mengaduk larutan A-B-Mix. 3 MOTOR LISTRIK: Memutar fungsi untuk menggerakkan agitan. 4 AGITATOR BAHAN: Alat yang berfungsi untuk mengaduk larutan nutrisi A, dan nutrisi B. 5 TANDON BAHAN: Berfungsi sebagai wadah penyimpanan larutan nutrisi A, dan nutrisi B. 6 POMPA LISTRIK: Berfungsi untuk mengalirkan larutan nutrisi A, dan B ke sistem semprotan/pengirisan. 7 SENSOR EC DAN SUHU: Berfungsi untuk mengukur konduktivitas larutan A-B-Mix. 8 PIPA SIRKULASI: Memili fungsi untuk mengalirkan larutan A-B-Mix ke dalam sistem penanaman. 9 SOLENOID DAN SENSOR DEBIT: Berfungsi untuk membuka dan menutup aliran ke-ke yang dibutuhkan sehingga jumlah nutrisi terukur.	Bertugas mengaduk larutan di tandon pencampuran akhir menggunakan motor penggerak. Tujuannya adalah memastikan nutrisi tercampur rata sebelum dialirkan ke sistem penyiraman, serta mencegah terjadinya endapan.
	[A-04] (4) → Agrigator Bahan  Keterangan: 1 TANDON PENCAMPURAN: Berfungsi sebagai tempat penyimpanan larutan nutrisi A, nutrisi B, dan Air. 2 AGITATOR PENCAMPURAN: Alat yang berfungsi untuk mengaduk larutan A-B-Mix. 3 MOTOR LISTRIK: Memutar fungsi untuk menggerakkan agitan. 4 AGITATOR BAHAN: Alat yang berfungsi untuk mengaduk larutan nutrisi A, dan nutrisi B. 5 TANDON BAHAN: Berfungsi sebagai wadah penyimpanan larutan nutrisi A, dan nutrisi B. 6 POMPA LISTRIK: Berfungsi untuk mengalirkan larutan nutrisi A, dan B ke sistem semprotan/pengirisan. 7 SENSOR EC DAN SUHU: Berfungsi untuk mengukur konduktivitas larutan A-B-Mix. 8 PIPA SIRKULASI: Memili fungsi untuk mengalirkan larutan A-B-Mix ke dalam sistem penanaman. 9 SOLENOID DAN SENSOR DEBIT: Berfungsi untuk membuka dan menutup aliran ke-ke yang dibutuhkan sehingga jumlah nutrisi terukur.	Berfungsi menjaga larutan bahan nutrisi di tangki penyimpanan agar tetap tercampur rata dan mencegah pengendapan yang dapat mempengaruhi ketepatan takaran nutrisi.

No.	[Kode Nama + Gambar Aktuator [A-05] (6) → Pompa Listrik Nutrisi	Deskripsi Singkat dan Fungsi
	 Keterangan: 1 TANDON PENCAMPURAN: Tangki untuk mencampurkan nutrisi dari A, B, dan C. 2 AGITATOR PENCAMPURAN: Alat yang berfungsi untuk mengaduk nutrisi dalam tangki. 3 MOTOR LISTRIK: Motor untuk menggerakkan agitator. 4 AGITATOR BAHAN: Alat yang berfungsi untuk mengaduk nutrisi dalam tangki. 5 TANDON BAHAN: Tangki untuk menyimpan nutrisi. 6 POMPA LISTRIK: Pompa untuk mengalirkan nutrisi ke sistem. 7 SENSOR EC DAN SUHU: Sensor untuk mengukur konsentrasi larutan A, B, dan C. 8 PIPA SIRKULASI: Pipa untuk mengalirkan nutrisi ke sistem. 9 SOLENOID DAN SENSOR DEBIT: Solenoid untuk membuka dan menutup aliran nutrisi yang dikontrol dengan sistem.	Berfungsi memindahkan cairan nutrisi dari tangki penyimpanan menuju tandon pencampuran untuk proses pengenceran sesuai takaran yang dibutuhkan.
4.	[A-06] Solenoid Valve Tandon Pencampuran	Merupakan katup elektrik yang mengatur buka-tutup aliran cairan pada jalur masuk atau keluar tandon pencampuran. Kendali katup ini diatur secara elektronik berdasarkan perintah dari sistem pusat.
5.	[A-07] 2 Pompa Pengaliran Nutrisi Tercampur Tandon Utama	Bertugas memberikan tekanan yang cukup untuk mengalirkan larutan nutrisi yang sudah tercampur dari tandon utama menuju seluruh jaringan pipa penyiraman di area tanam.

Tabel 3.7 Data Kebutuhan Otomatisasi

No.	Nama Instrumen	Kode Aktuator & Sensor Terlibat	Deskripsi Mekanisme Otomatisasi
Tipe Otomatisasi: Ambang Batas Sensor			
1.	Pengendalian Suhu [Pendinginan]	[S-01] Suhu [A-01] Exhaust Fan [A-02] Cooling Pad	ON saat suhu $\geq 31^{\circ}\text{C}$ OFF saat suhu $\leq 29^{\circ}\text{C}$
2.	Pengendalian Kelembapan	[S-01] Kelembapan [A-02] Cooling Pad	ON saat kelembapan $\leq 65\%$ OFF saat kelembapan $\geq 75\%$

No.	Nama Instrumen	Kode Aktuator & Sensor Terlibat	Deskripsi Mekanisme Otomatisasi
3.	Pengisian Air Tandon [Auto-Refill]	[S-04] Level Air [A-05] Solenoid Tandon	Isi saat level $\leq 10\%$ Stop saat level 95%
4.	Peringatan Stok Nutrisi	[S-05] Level Nutrisi [Sistem Notifikasi]	Kirim notifikasi saat stok $\leq 10\text{ L}$
5.	Penyesuaian Konsentrasi [TDS]	[S-02] TDS [A-04] Pompa / [A-06] Solenoid	ON saat TDS < minimum OFF saat TDS $\geq$ target
6.	Penyesuaian Keasaman [pH]	[S-03] pH [A-06] Solenoid Makro	Koreksi aktif saat pH di luar rentang optimal
7.	Anti-Sumbatan [Debit]	[S-02] Debit [A-04] Agitator / [A-07] Pompa	<i>Agitator</i> ON saat debit < minimum [0,8 L/menit]
Tipe Otomatisasi: Penjadwalan			
8.	Jadwal Pendingin <i>Greenhouse</i>	[A-01] Exhaust Fan [A-02] Cooling Pad	Aktif pukul 07:00 [Durasi: 10 jam]
9.	Pengadukan Nutrisi Bahan Baku	[A-07] Motor Agitator	Aktif pukul 06:30 & 16:30 [Durasi: 300 detik]
10.	Irigasi Tetes [Nutrisi]	[A-04] Pompa Nutrisi [A-03] Solenoid Valve	Mulai pukul 07:00 [5 siklus, interval 2 jam]
Tipe Otomatisasi: Data rekomendasi robot			
11.	ml/Tanaman	Indikator ml/Tanaman (120-380) [A-07] Pompa Distribusi Nutrisi	ON saat ml/tanaman = 120-380 OFF setelah beberapa menit

3. Studi dan Kajian Literatur

Tahap studi literatur telah dilakukan dan diuraikan secara lengkap pada BAB II, meliputi sub-bab 2.1 Tinjauan Pustaka dan 2.2 Dasar Teori sebagai landasan *teoritis, konseptual*, serta *metodologis* penelitian ini.

3.3.2 Proses Metode Agile Kanban

Metodologi pengembangan sistem dalam penelitian ini menerapkan kerangka kerja *Agile Kanban* untuk mengakomodasi kebutuhan alur kerja yang adaptif dan transparan. Pendekatan ini dipilih karena keunggulannya dalam memvisualisasikan progres tugas secara jelas serta meminimalkan hambatan (*bottleneck*) melalui mekanisme pembatasan pekerjaan yang sedang berlangsung (*Work In Progress*) [23]. Sesuai dengan alur penelitian, siklus pengembangan dilaksanakan secara iteratif dan terstruktur melalui lima tahapan utama, yakni Perencanaan, Desain, Implementasi, Pengujian, dan Tinjauan penanggung jawab *greenhouse* Kebun Raya ITERA.

1. Perencanaan

Tahap perencanaan difokuskan pada pendefinisian ruang lingkup teknis untuk subsistem otomatisasi berdasarkan hasil analisis masalah dan kebutuhan mitra. Pada fase ini, seluruh persyaratan operasional dipetakan menjadi spesifikasi kebutuhan sistem yang terukur guna memastikan fungsi otomatisasi yang dikembangkan tepat sasaran dan dapat diimplementasikan. Spesifikasi yang disusun sebagai landasan pengembangan ini dikelompokkan menjadi dua kategori utama, yaitu:

a. Kebutuhan Fungsional

Kebutuhan fungsional pada Tabel 3.8 menjelaskan fungsi-fungsi yang harus disediakan oleh subsistem yang dirancang untuk mendukung aktivitas pengguna. Setiap kebutuhan diberi kode [F-XX] yang merujuk pada analisis masalah dan pengumpulan data. Daftar lengkap kebutuhan fungsional tersebut dirincikan pada Tabel 3.8.

Tabel 3.8 Kebutuhan Fungsional

Kode	Nama Fitur	Hak Akses	Keterangan Singkat
F-01	Login dan Logout Subsistem	Staff, Pengelola, Super Admin	Mengelola autentikasi pengguna untuk akses fitur berbasis hak akses serta terminasi sesi sistem.
F-02	Manajemen Perangkat Sensor	Staff (lihat), Pengelola & Super Admin (kelola)	Memantau nilai dan status konektivitas sensor secara <i>real-time</i> serta mengelola konfigurasi perangkat.
F-03	Manajemen Perangkat Aktuator	Staff (lihat), Pengelola & Super Admin (kelola)	Memantau status operasional aktuator dan mengelola kontrol manual serta konfigurasi perangkat.
F-04	Otomatisasi Berdasarkan Penjadwalan	Staff (lihat), Pengelola & Super Admin (kelola)	Memicu eksekusi perangkat IoT (ON/OFF) secara otomatis berdasarkan parameter waktu yang ditentukan.
F-05	Otomatisasi Berdasarkan Ambang Batas Sensor	Staff (lihat), Pengelola & Super Admin (kelola)	Memicu aksi perangkat IoT secara otomatis saat nilai sensor melewati batas parameter (<i>threshold</i>) yang ditetapkan.
F-06	Otomatisasi Berdasarkan Data Rekomendasi Robot	Staff (lihat), Pengelola & Super Admin (kelola)	Mengeksekusi instruksi kontrol aktuator secara otomatis berdasarkan data rekomendasi dari robot AI.
F-07	Notifikasi Otomatisasi dan Rangkuman Otomatisasi Harian	Staff, Pengelola, Super Admin	Mengirimkan peringatan (<i>alert</i>) sistem terkait kondisi kritis, stok habis, atau anomali perangkat. Menyajikan rekapitulasi data aktivitas otomatisasi yang telah berjalan dalam periode 24 jam.
F-08	Log Seluruh Aktivitas dan Otomatisasi	Super Admin	Merekam seluruh riwayat aktivitas sistem, pemicu otomatisasi, dan interaksi pengguna untuk keperluan tinjauan aktivitas.

Masalah: W = Wawancara, O = Observasi Lapangan, A = Analisis Kebutuhan Otomatisasi, D = Dokumentasi Sistem Lama

b. Kebutuhan Non-Fungsional

Kebutuhan non-fungsional pada Tabel 3.9 menjelaskan atribut kualitas sistem yang harus dipenuhi. Setiap kebutuhan diberi kode [NF-XX].

Tabel 3.9 Daftar Kebutuhan *Non-Fungsional*

Kode	Aspek	Deskripsi	Metrik Pengukuran	Target
NF-01	<i>Performance</i> (Responsivitas Real-time)	Kecepatan sinkronisasi data visual (sensor & status) secara <i>real-time</i> .	Jeda visual (<i>latency</i>) pembaruan data dari MQTT broker ke layar website.	< 3 detik.
NF-02	<i>Reliability</i> (Keandalan Komunikasi)	Jaminan pengiriman perintah kendali tanpa kehilangan data.	Verifikasi penerapan MQTT QoS Level 1 (<i>at least once</i>) pada log pengiriman.	100% <i>passed</i>
NF-03	<i>Compatibility</i> (Kompatibilitas)	Dukungan operasional penuh pada berbagai peramban modern.	Website berfungsi penuh pada berbagai browser modern standar.	Chrome, Firefox, Safari, Opera, dan Brave.
NF-04	<i>Responsive Design</i> (Responsivitas Tampilan)	Fleksibilitas tata letak antarmuka pada berbagai dimensi layar.	Verifikasi visual pada breakpoint: Mobile ($\leq 425\text{px}$), Tablet ($768\text{px s.d. } < 1024\text{px}$), dan Desktop ($\geq 1024\text{px}$). Tools: <i>Toogle Device Toolbar</i>	100% <i>passed</i>
NF-05	<i>Usability</i> (Kemudahan Penggunaan)	Tingkat kemudahan pengoperasian sistem oleh pengguna.	Skor <i>System Usability Scale</i> (SUS)	≥ 68 (Kategori " <i>Good</i> ").

Keterangan : W4 = Wawancara Nomor 4

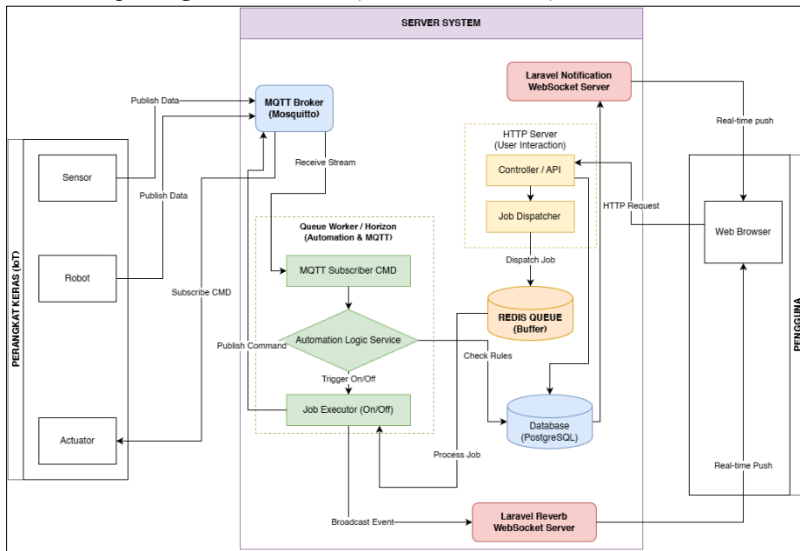
2. Desain

Tahap desain dilakukan setelah kebutuhan sistem terdefinisi dengan jelas. Desain disusun dalam bentuk *use case diagram*, *activity diagram*, ERD dan *wireframe* untuk memvisualisasikan sistem sebelum tahap implementasi.

a. Desain Rancangan Alur Komunikasi Subsistem

Rancangan komunikasi sistem ini difokuskan pada integrasi protokol komunikasi *real-time* dan manajemen proses asinkron guna mengatasi inefisiensi metode terdahulu. Alur dimulai saat data dari sensor dan robot *publish* data, diterima oleh MQTT Broker, dan diteruskan ke layanan *subscriber* untuk dievaluasi terhadap aturan otomatisasi di basis data. Apabila kondisi aturan terpenuhi, sistem tidak langsung mengeksekusi perintah, melainkan mengirimkan tugas (*dispatch job*) ke dalam antrian (*queue*) berbasis Redis. Selanjutnya, *Queue Worker* memproses tugas tersebut secara latar belakang (*background processing*) untuk mengirimkan perintah kendali ke

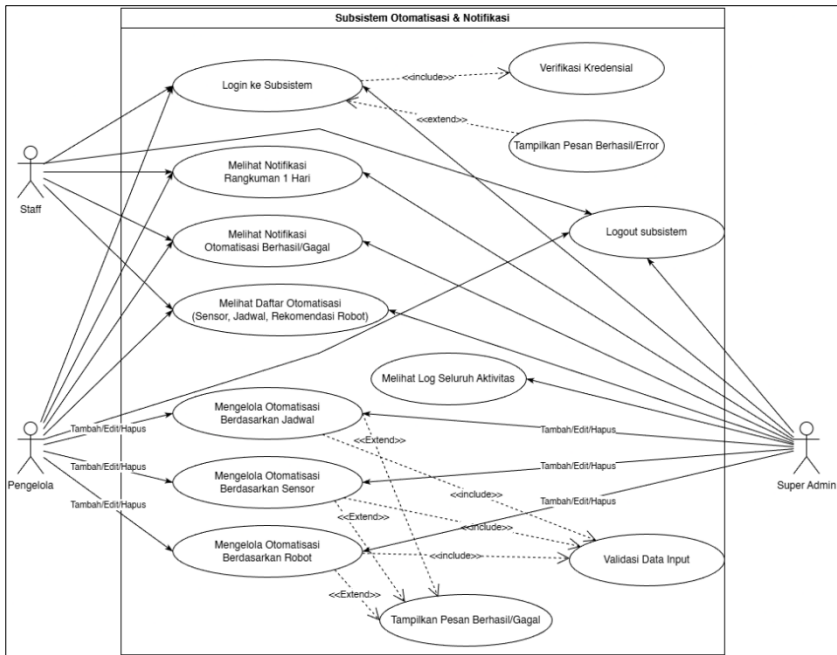
perangkat aktuator melalui MQTT Broker. Secara paralel, setiap pembaruan data sensor maupun perubahan status aktuator langsung disiarkan (*broadcast*) menggunakan teknologi *WebSocket* (Laravel Reverb) ke antarmuka pengguna, sehingga *monitoring* berjalan secara waktu nyata tanpa memerlukan mekanisme *polling* dari sisi klien (Lihat Gambar 3.3).



Gambar 3.4 Arsitektur Otomatisasi Perangkat IoT

b. Desain Diagram Use Case

Diagram *Use Case* pada Gambar 3.4 mengilustrasikan interaksi tiga aktor, yakni *Staff*, *Pengelola*, dan *Super Admin*, yang semuanya memiliki akses dasar untuk *login*, menerima notifikasi operasional, dan *logout*. Aktor *Staff* memiliki kewenangan terbatas yang hanya mencakup fungsi melihat data pemantauan dan status perangkat di dalam subsistem. Berbeda dengan *Pengelola* yang memiliki hak tambahan untuk mengelola konfigurasi aturan otomatisasi berdasarkan jadwal, ambang batas *sensor*, dan rekomendasi *robot*. Sementara itu, *Super Admin* memiliki hak akses penuh yang mencakup seluruh fungsi *Pengelola* serta kemampuan mengakses riwayat lengkap aktivitas sistem untuk keperluan peninjauan internal.



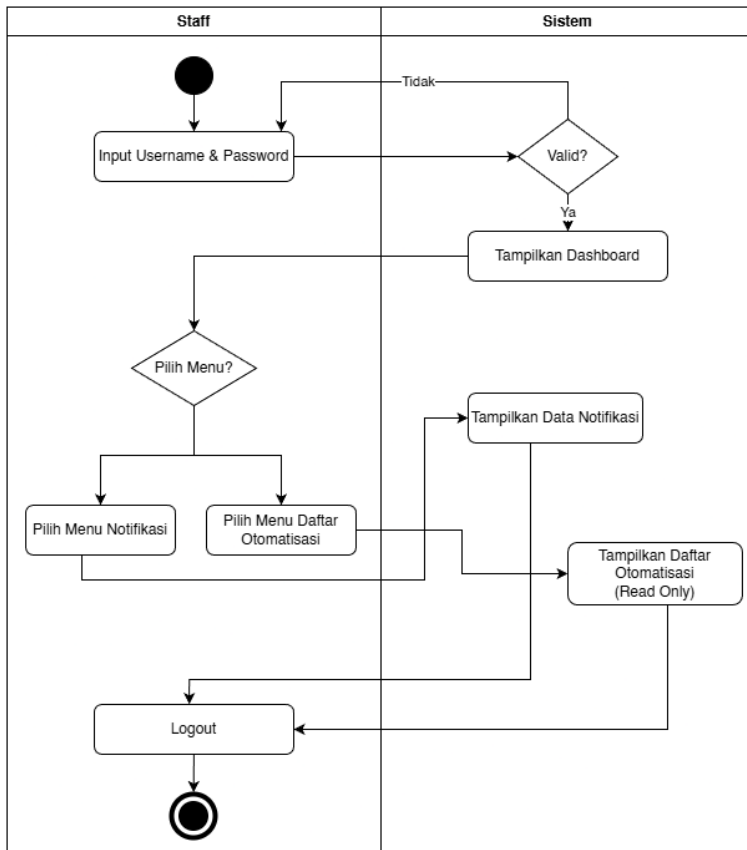
Gambar 3.5 Use Case Diagram Otomatisasi Perangkat IOT

c. Desain Diagram Activity

Diagram aktivitas subsistem Otomatisasi Perangkat IOT menggambarkan alur interaksi dinamis antara aktor (Staff, Pengelola, dan Super Admin) dengan sistem berdasarkan hak akses masing-masing, mencakup proses login, pengelolaan data, hingga logout.

c.1. Diagram Activity Staff

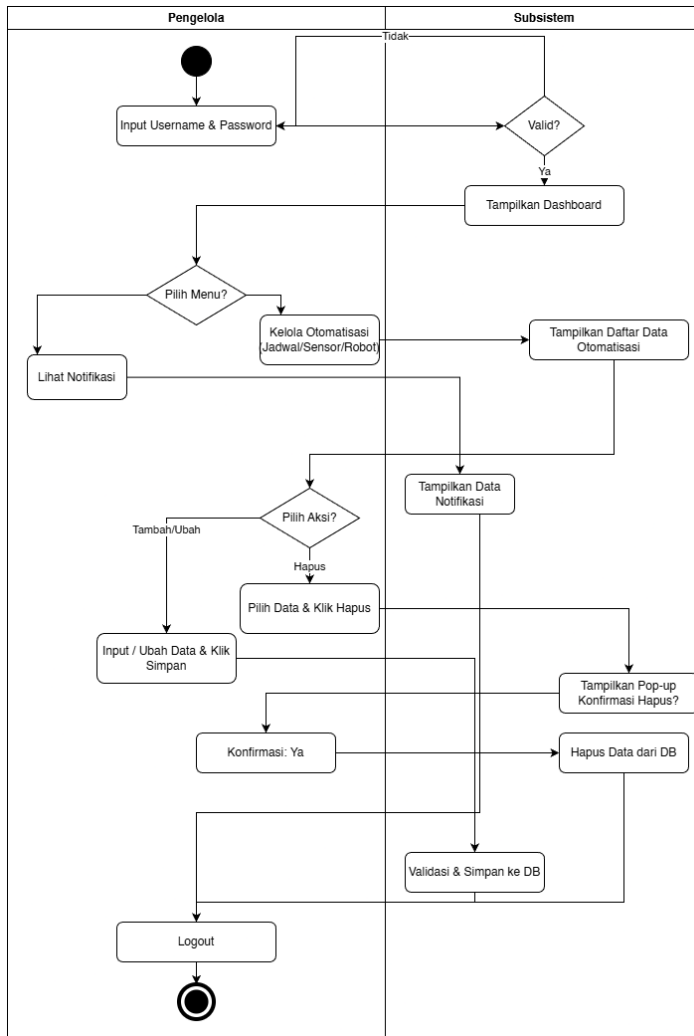
Aktivitas aktor Staff berfokus pada fungsi pemantauan data tanpa hak akses manipulasi konfigurasi. Seperti yang diilustrasikan pada Gambar 3.6, proses dimulai dengan validasi kredensial login. Setelah berhasil masuk ke *dashboard* utama, Staff dihadapkan pada dua opsi menu navigasi: melihat notifikasi sistem (rangkuman, keberhasilan, atau kegagalan otomatisasi) atau melihat daftar jadwal otomatisasi yang bersifat *read-only*. Alur kerja ini memastikan Staff mendapatkan informasi operasional terkini sebelum melakukan *logout* dari sistem.



Gambar 3.6 Activity Diagram Staff

c.2. Diagram Activity Pengelola

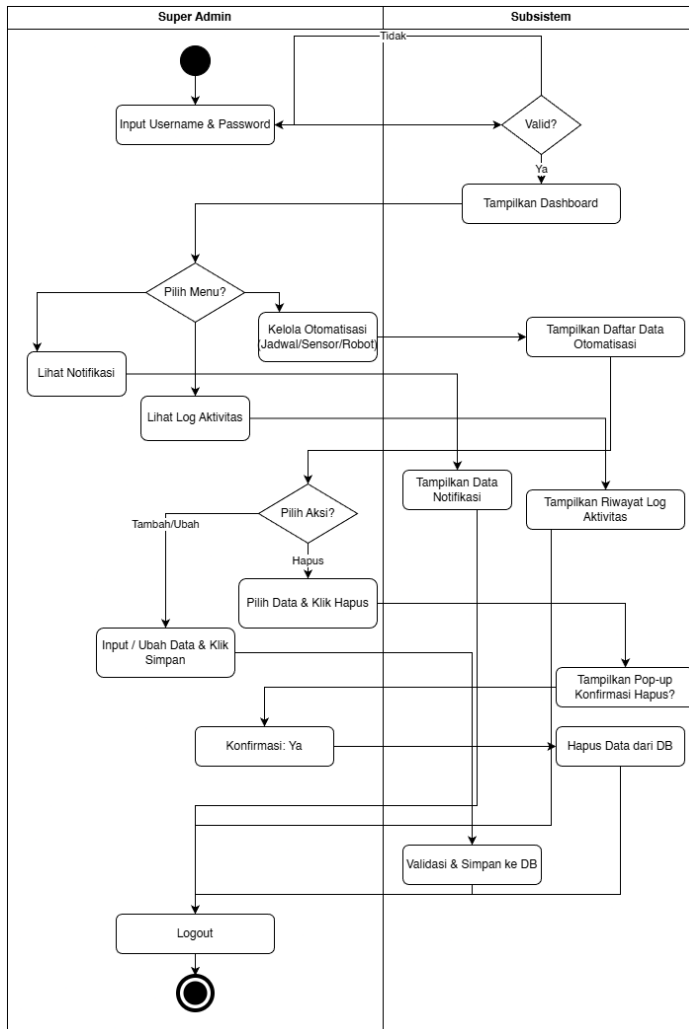
Aktivitas aktor Pengelola memiliki wewenang administratif tingkat menengah yang meliputi fungsi *monitoring* dan fungsi eksekusi data. Mengacu pada Gambar 3.7, setelah melewati proses autentikasi, Pengelola dapat mengakses menu pengelolaan untuk melakukan operasi (Tambah/Edit/Hapus) pada konfigurasi jadwal, ambang batas sensor, dan parameter robot. Setiap perubahan data yang diinputkan akan divalidasi oleh sistem dan disimpan ke dalam basis data, diakhiri dengan konfirmasi keberhasilan sebelum pengguna keluar dari sistem.



Gambar 3.7 Activity Diagram Pengelola

c.3. Diagram Activity Super Admin

Aktor Super Admin memegang hak akses tertinggi dalam subsistem dengan fokus utama pada pemeliharaan integritas data (*data maintenance*). Sebagaimana ditunjukkan pada Gambar 3.8, alur aktivitas ini memungkinkan Super Admin untuk mengakses seluruh fitur pemantauan, operasi mirip pengelola (Tambah/Ubah/Hapus). Yang membedakan, super admin memiliki hak melihat seluruh aktivitas yang terjadi di subsistem. Proses penghapusan ini dilengkapi dengan mekanisme konfirmasi hapus untuk mencegah ketidaksengajaan penghapusan data kritis dari basis data.

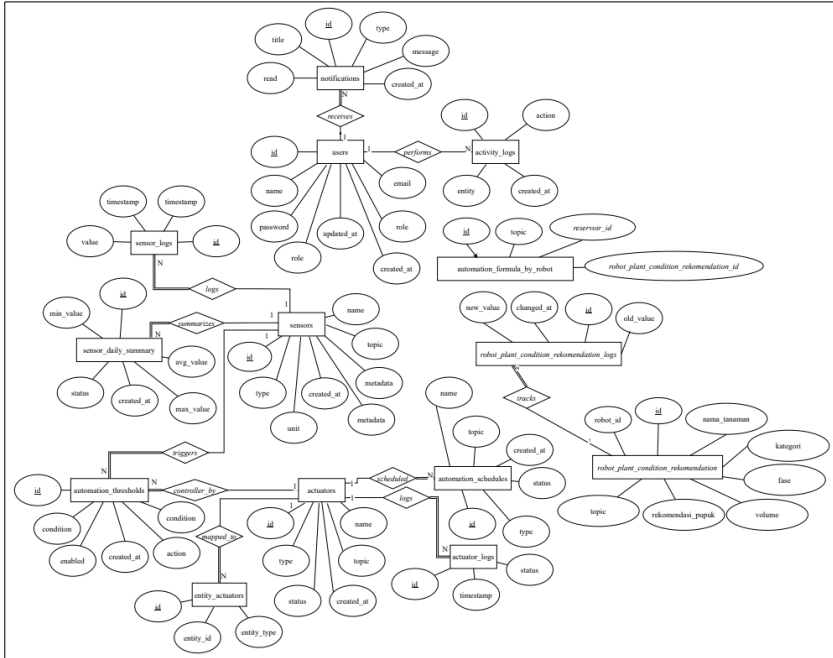


Gambar 3.8 Activity Diagram Super Admin

d. Desain ERD

Desain *Entity Relationship Diagram* (ERD) ini menggambarkan struktur data yang menghubungkan pengguna dengan seluruh ekosistem IoT, mulai dari pemantauan sensor hingga pengendalian aktuator secara otomatis. Sistem ini dirancang agar sensor dan aktuator dapat bekerja responsif melalui tiga mekanisme utama: jadwal waktu (*schedule*), ambang batas kondisi (*threshold*), dan formula rekomendasi cerdas untuk perangkat robot. Untuk mendukung keamanan dan transparansi, setiap tindakan pengguna dicatat dalam log aktivitas, sementara kinerja perangkat direkam dalam log sensor dan

aktuator, yang kemudian dilengkapi dengan fitur notifikasi *real-time* agar pengguna selalu mengetahui status terkini sistem (Lihat Gambar 3.9).



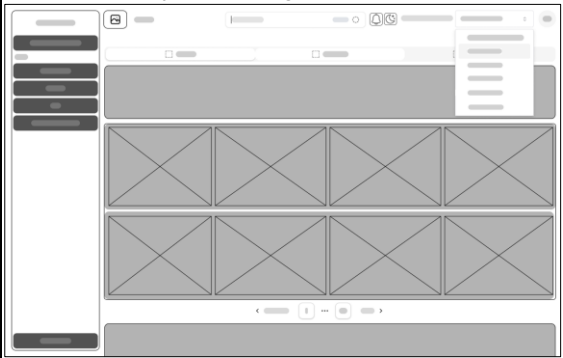
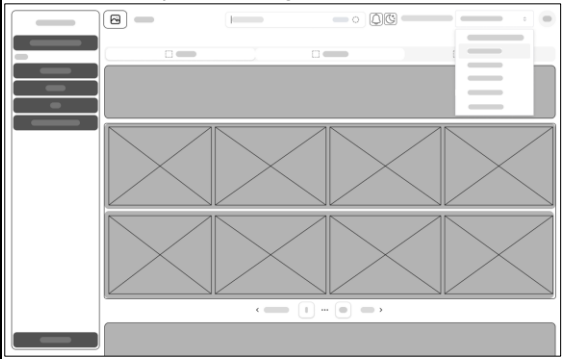
Gambar 3.9 Desain ERD Subsistem Otomatisasi Perangkat IOT

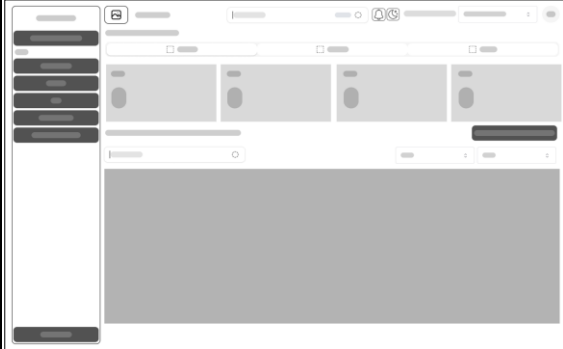
e. Desain Wireframe

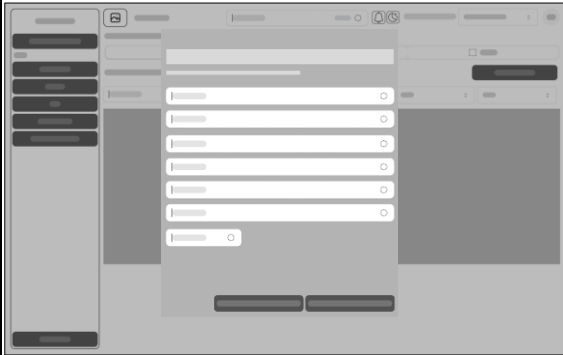
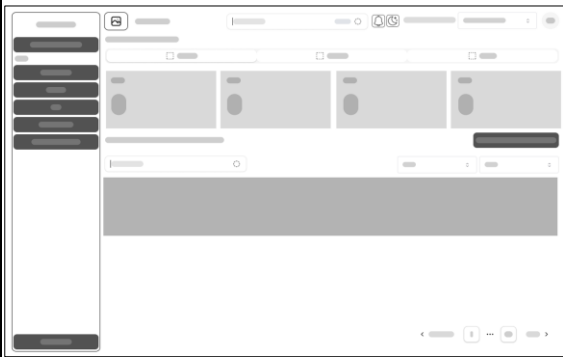
Wireframe berfungsi sebagai rancangan visual awal yang memetakan struktur antarmuka sistem secara mendasar. Tahapan ini berfokus pada penyusunan tata letak elemen, alur navigasi, serta penempatan komponen fungsional sebelum memasuki proses pengembangan antarmuka yang lebih mendetail. Tujuannya adalah memastikan kerangka interaksi telah tersusun secara logis dan sesuai dengan kebutuhan fungsional yang di definisikan. Rincian rancangan antarmuka untuk setiap subsistem dapat dilihat pada Tabel 3.10 berikut.

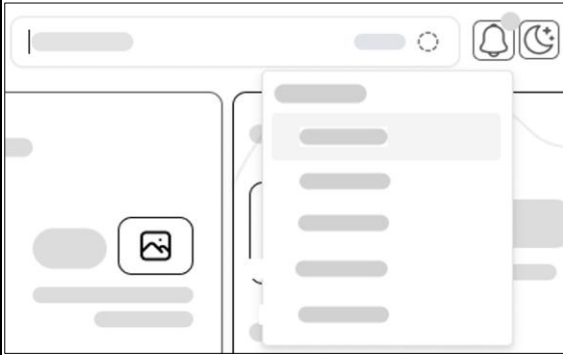
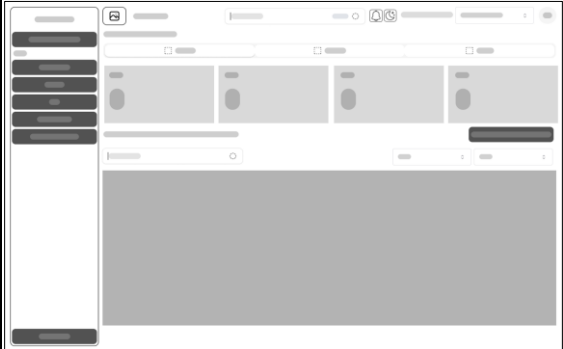
Tabel 3.10 Daftar Desain *Wireframe*

[Kode Kebutuhan Fungsional] Nama Desain + Gambar <i>Wireframe</i>	Deskripsi <i>Wireframe</i>
[F-01] Halaman Login	Halaman ini berfungsi sebagai tempat pengguna masuk ke sistem dengan memasukkan email dan kata sandi. Tampilan dirancang sederhana agar mudah digunakan, serta dilengkapi pesan bantuan

	jika pengguna salah mengetik atau belum mengisi kolom yang wajib diisi.
[F-02] Halaman Manajemen Perangkat Sensor 	Halaman ini menampilkan daftar seluruh perangkat sensor yang digunakan di lapangan. Pengguna dapat mencari sensor tertentu, menyaring tampilan berdasarkan jenis atau lokasi, dan melihat nilai terkini yang diukur sensor tersebut. Tersedia juga grafik untuk melihat perubahan data sensor berdasarkan tanggal yang dipilih.
[F-03] Halaman Manajemen Perangkat Aktuator 	Halaman ini menampilkan semua perangkat yang berfungsi menggerakkan atau mengalirkan sesuatu, misalnya pompa atau katup. Pengguna dapat melihat status perangkat (menyala atau mati), mengubah mode pengoperasian, serta menyalakan atau mematikan perangkat dengan satu tombol. Setiap tindakan penting akan diminta konfirmasi terlebih dahulu agar lebih aman.
[F-05] Halaman Manajemen Otomatisasi Berdasarkan Ambang Batas Sensor	Halaman ini menampilkan daftar aturan otomatisasi yang bekerja berdasarkan batas nilai tertentu dari sensor, misalnya ketika suhu terlalu tinggi atau

	kadar nutrisi terlalu rendah. Pengguna dapat melihat ringkasan jumlah aturan yang aktif, mencari aturan tertentu, dan mengelola seluruh data yang ada.
[F-05] Tambah dan Ubah Data Otomatisasi Berdasarkan Ambang Batas Sensor 	Bagian ini berupa jendela <i>pop-up</i> yang muncul ketika pengguna ingin membuat aturan baru atau mengubah aturan yang sudah ada. Formulir disusun agar mudah dipahami sehingga pengguna dapat mengatur otomatisasi dengan cepat.
[F-04] Halaman Manajemen Otomatisasi Berdasarkan Penjadwalan 	Halaman ini menampilkan daftar otomatisasi yang dijalankan berdasarkan waktu tertentu, misalnya pompa menyala setiap pagi atau setiap dua jam sekali. Pengguna dapat melihat rangkuman statistik, melakukan pencarian, serta menambah, mengubah, atau menghapus jadwal otomatisasi.
[F-04] Tambah dan Ubah Data Otomatisasi Berdasarkan Ambang Batas Sensor	Jendela <i>pop-up</i> ini digunakan untuk membuat jadwal otomatisasi baru atau memperbarui jadwal yang sudah ada. Seluruh pengaturannya dibuat sederhana agar pengguna

	mudah menentukan waktu dan aksi yang akan dijalankan.
[F-06] Halaman Manajemen Otomatisasi Berdasarkan Rekomendasi Robot 	Halaman ini menampilkan seluruh aturan otomatisasi yang mengikuti rekomendasi dari sistem robot atau kecerdasan buatan. Pengguna dapat mencari data tertentu, melihat ringkasan kondisi, dan mengelola keseluruhan aturan secara penuh.
[F-06] Tambah dan Ubah Data Otomatisasi Berdasarkan Rekomendasi Robot 	Bagian <i>pop-up</i> ini berfungsi untuk membuat atau memperbarui aturan otomatisasi yang mengikuti rekomendasi robot. Pengguna hanya perlu memilih reservoir dan rekomendasi yang ingin diterapkan.
[F-07] & [F-08] Tooltip dan Halaman Notifikasi	Fitur ini menampilkan informasi singkat mengenai kejadian penting di sistem, misalnya hasil proses otomatisasi atau laporan harian. Notifikasi muncul secara langsung sehingga

	pengguna dapat memantau kondisi tanpa harus membuka halaman tertentu.
	
[F-09] Halaman Log Aktivitas Seluruh Aktivitas 	Halaman ini berisi daftar lengkap seluruh tindakan atau kejadian yang pernah terjadi di dalam sistem. Catatan ini membantu super admin mengetahui riwayat peristiwa, memeriksa kesalahan, serta melihat siapa yang melakukan suatu tindakan.

3. Implementasi

Tahap implementasi merupakan proses penerjemahan seluruh rancangan desain menjadi kode program yang dapat dijalankan. Pada fase ini, antarmuka pengguna dikembangkan mengacu pada *wireframe* dan *activity diagram* yang telah dibuat, sementara logika sistem dan struktur basis data dibangun berdasarkan *Use Case Diagram* dan ERD untuk menangani proses otomatisasi serta manajemen data. Fokus utama tahap ini adalah menghasilkan

subsistem yang berfungsi penuh sesuai spesifikasi kebutuhan, di mana hasil tampilan antarmuka dan potongan kode program secara rinci akan dipaparkan pada BAB IV, subbab 4.1.

4. Pengujian

a. *Blackbox Testing*

Blackbox Testing dilakukan untuk memverifikasi fungsionalitas subsistem tanpa meninjau struktur internal kode program. Fokus utama pengujian ini adalah validasi fungsional berdasarkan fitur-fitur yang telah dikembangkan. Setiap fitur diuji melalui serangkaian skenario yang ditetapkan untuk memastikan subsistem otomatisasi menghasilkan keluaran (*output*) yang sesuai dengan ekspektasi. Rincian rencana dan skenario pengujian terdapat pada Lampiran F, untuk rangkumannya disajikan pada Tabel 3.11.

Tabel 3.11 Rencana Pengujian *Blackbox Testing*

No	Kode	Kategori Fitur	Jumlah Skenario	Skenario Berhasil	Skenario Gagal	Persentase Keberhasilan
1	BT-01	Login dan Logout	10	Jumlah Berhasil/Gagal		0-100%
2	BT-02	Perangkat Sensor	18			
3	BT-03	Perangkat Aktuator	15			
4	BT-04	Halaman Otomatisasi	8			
5	BT-05	Otomatisasi Jadwal	10			
6	BT-06	Otomatisasi Sensor	9			
7	BT-07	Otomatisasi Robot	10			
8	BT-08	Notifikasi	8			
9	BT-09	Log Aktivitas	11			
10	BT-10	Hak Akses	52			
Total	10 Kategori		151			

Adapun hasil akhir pengujian akan dikalkulasi menggunakan persamaan yang merujuk pada Rumus 2.1, di mana tingkat keberhasilan diperoleh dari perbandingan rasio antara jumlah skenario yang berhasil terhadap total skenario yang diujikan. Berikut contoh perhitungannya jika ada 150 dari 151 pengujian dan persentase keberhasilan mendapatkan hasil 99,33%.

$$\text{Persentase Keberhasilan} = \frac{150}{151} \times 100\% = 99,33\%$$

b. Pengujian Komunikasi Data dan Otomatisasi

Pengujian ini bertujuan untuk memverifikasi keandalan komunikasi data dua arah antara subsistem dan perangkat *IoT* menggunakan *MQTT Explorer* sebagai alat pemantauan, serta memvalidasi kinerja logika otomatisasi yang diterapkan di *Greenhouse* Kebun Raya ITERA. Proses validasi mencakup pemeriksaan akurasi transmisi pesan pada perangkat sensor

dan aktuator (Tabel 3.12), serta memastikan bahwa seluruh skenario otomatisasi, baik yang dipicu oleh ambang batas sensor, penjadwalan waktu, maupun rekomendasi data robot, berjalan sesuai dengan spesifikasi yang telah ditetapkan (Tabel 3.13). Seluruh parameter pengujian tersebut mengacu pada kebutuhan operasional otomatisasi pada Tabel 3.7.

Tabel 3.12 Skenario Pengujian Komunikasi Data

No	Kode	Nama Pengujian + Topic	Payload	Hasil yang diharapkan
Pengujian Manual Perangkat Sensor				
1.	MQTT-01	[S-01] Suhu dan Kelembapan Topic: "iterahero/kebun-roya-itera/sensor/sensor-szma/data" "iterahero/kebun-roya-itera/sensor/sensor-6evp/data"	25-40	100% Sesuai
2.	MQTT-02	[S-02] Total Dissolved Solids (TDS) Topic: "iterahero/rumah-pemupukan /sensor/sensor-3vs5/data"	100-2000	100% Sesuai
3.	MQTT-03	[S-03] pH Topic: "iterahero/rumah-pemupukan /sensor/sensor-oiux/data"	1-12	100% Sesuai
4.	MQTT-04	[S-04] Ultrasonik Tandon Air (Volume Tinggi Air) Topic: "iterahero/rumah-pemupukan /sensor/sensor-lecb/data"	0-100%	100% Sesuai
5.	MQTT-05	[S-05] Ultrasonik Nutrisi Pupuk (Volume Tinggi Air Nutrisi) Topic: "iterahero/rumah-pemupukan /sensor/sensor-huxq/data"	0-100%	100% Sesuai
Pengujian Manual Perangkat Aktuator				
6.	MQTT-06	[A-01] Cooling Pad Topic:	ON dan OFF	100% Sesuai

No	Kode	Nama Pengujian + Topic	Payload	Hasil yang diharapkan
		"iterahero/kebun- raya- itera/actuator/actuator- xuix/cmd"		
7.	MQTT-07	[A-02] Exhaust Fan Topic: "iterahero/kebun- raya- itera/actuator/actuator- tk4m/cmd"	ON dan OFF	100% Sesuai
8.	MQTT-08	[A-03] (2) → Agrigator Pencampuran Topic:	ON dan OFF	100% Sesuai
9.	MQTT-09	[A-04] (4) → Agrigator Bahan Topic:	ON dan OFF	100% Sesuai
10.	MQTT-10	[A-05] (6) → Pompa Listrik Nutrisi Topic:	ON dan OFF	100% Sesuai
11.	MQTT-11	[A-06] Solenoid Valve Tandon Pencampuran Topic:	ON dan OFF	100% Sesuai
12.	MQTT-12	[A-07] 2 Pompa Pengaliran Nutrisi Tercampur Tandon Utama Topic:	ON dan OFF	100% Sesuai
Pengujian Manual Data Robot				
13.	MQTT-13	Volume Tanaman (ml/tanaman) Topic: "iterahero/kebun- raya/robot/robot-tes- l/data/mltanaman"	Volume (120-380)	100% Sesuai

Tabel 3.13 Pengujian Skenario Otomatisasi Perangkat IoT

No	Kode	Nama Pengujian	Trigger	Payload	Hasil yang diharapkan
Pengujian Otomatisasi Perangkat Berdasarkan Ambang Batas Sensor					
1.	AUTO-01	Pengendalian Suhu [Pendinginan]	ON saat suhu $\geq 31^{\circ}\text{C}$	ON	A-01 dan A-02 Hidup
			OFF saat suhu $\leq 29^{\circ}\text{C}$	OFF	A-01 dan A-02 Mati

No	Kode	Nama Pengujian	Trigger	Payload	Hasil yang diharapkan
		[S-01] Suhu [A-01] <i>Exhaust Fan</i> [A-02] <i>Cooling Pad</i>			
2.	AUTO-02	Pengendalian Kelembapan	ON saat kelembapan $\leq 65\%$	ON	A-02 Hidup
		[S-01] Kelembapan [A-02] <i>Cooling Pad</i>	OFF saat kelembapan $\geq 75\%$	OFF	A-02 Mati
3.	AUTO-03	Pengisian Air Tandon	ON saat level $\leq 10\%$	ON	A-05 Hidup
		[S-04] Level Air [A-05] Solenoid Tandon	OFF saat level 95%	OFF	A-05 Mati
4.	AUTO-04	Peringatan Stok Nutrisi [S-05] Level Nutrisi [Sistem Notifikasi]	Kirim notifikasi saat stok ≤ 10 L	-	Notifikasi "Stok Rendah" diterima user
5.	AUTO-05	Penyesuaian Konsentrasi [TDS]	ON saat TDS < minimum	ON	[A-04] Hidup
			OFF saat TDS $\geq$ target	OFF	[A-04] Mati
Pengujian Otomatisasi Perangkat Berdasarkan Penjadwalan					
6.	AUTO-06	Jadwal Pendingin Greenhouse [A-01] Exhaust Fan [A-02] <i>Cooling Pad</i>	Aktif pukul 07:00 [Durasi: 10 jam]	ON	[A-01] Exhaust Fan dan [A-02] <i>Cooling Pad</i> Hidup pada pukul 07:00 dengan durasi 10 jam
7.	AUTO-07	Pengadukan Nutrisi Bahan Baku [A-04] Motor Agitator	Aktif pukul 06:30 & 16:30 [Durasi: 300 detik]	ON	[A-04] Motor Agitator Hidup pada pukul 06:30 dan 16:30
8.	AUTO-08	Irigasi Tetes [Nutrisi] [A-07] Pompa Nutrisi	Mulai pukul 07:00 [5 siklus, interval 2 jam]	ON	[A-07] Pompa Nutrisi Hidup mulai pukul
Pengujian Otomatisasi Perangkat Berdasarkan Data Rekomendasi Robot					
9.	AUTO-11	MI/tanaman [A-07] Pompa Nutrisi	ON saat data ml/tanaman = 120	ON	[A-07] Pompa Nutrisi menyala selama 5 menit

No	Kode	Nama Pengujian	Trigger	Payload	Hasil yang diharapkan
			[5 siklus, 5 menit interval 2 jam]		
			Setiap 5 menit setelah hidup	OFF	[A-07] Pompa Nutrisi mati
			Pengulangan 5 kali dengan interval 2 jam	ON	[A-07] Pompa Nutrisi menyala kembali selama 5 menit sampai iterasi selesai

5. *Deploy*

Tahapan *deployment* merupakan fase pemindahan kode program dari lingkungan pengembangan lokal menuju lingkungan server produksi untuk memastikan ketersediaan sistem dalam skala yang lebih luas. Proses ini melibatkan konfigurasi teknis pada infrastruktur server dan jaringan agar aplikasi yang dikembangkan dapat diakses secara daring dan stabil melalui protokol internet. Ketersediaan akses publik ini dirancang secara khusus untuk memfasilitasi keterlibatan pengguna atau klien dalam melakukan interaksi langsung dengan antarmuka sistem yang telah dibangun. Selanjutnya, aktivitas pengujian yang dilakukan pada lingkungan *live* ini bertujuan untuk memverifikasi validitas fungsionalitas perangkat lunak serta memastikan kesesuaiannya dengan spesifikasi kebutuhan yang telah ditetapkan pada tahap perancangan sebelumnya.

6. **Review Penanggung jawab *greenhouse* Kebun Raya ITERA**

Tahap tinjauan (*review*) oleh *penanggung jawab greenhouse* Kebun Raya ITERA dilaksanakan untuk memvalidasi kesesuaian hasil pengujian *Blackbox Testing* dan *Responsive Testing* terhadap spesifikasi kebutuhan yang telah disepakati. Proses ini bertujuan untuk memastikan bahwa seluruh skenario uji dan luaran fungsional sistem benar-benar selaras dengan ekspektasi operasional di *Greenhouse*. Persetujuan formal dari *Penanggung jawab greenhouse* Kebun Raya ITERA pada tahap ini menjadi prasyarat mutlak sebelum sistem dinyatakan layak untuk dilanjutkan ke fase pengujian *usability* dengan pengguna akhir.

3.3.3 **Pengujian *Usability***

Pengujian *usability* dilakukan untuk mengevaluasi tingkat keberterimaan dan kualitas interaksi pengguna terhadap subsistem otomatisasi yang telah dikembangkan. Tahapan ini bertujuan memvalidasi bahwa seluruh otomatisasi perangkat IoT bisa diterima bagi seluruh level pengguna di

lingkungan operasional Greenhouse Kebun Raya ITERA, dengan menerapkan standar dan instrumen *System Usability Scale* (SUS).

Jumlah responden yang dilibatkan dalam pengujian *usability* ini adalah 32 pengguna. Angka ini didasarkan pada *purposive sampling* yakni keseluruhan *stakeholder* terkait yang memiliki kewenangan dan keterlibatan langsung dalam penelitian subsistem *greenhouse* Kebun Raya ITERA dan memiliki akses ke website ini, sebagaimana telah didefinisikan dalam Lampiran F. Komposisi responden terdiri dari 12 Staff, 18 Pengelola, dan 2 Super Admin, yang mencerminkan pembagian peran dari para *stakeholder* tersebut, sebagaimana ditunjukkan pada Tabel 3.14.

Tabel 3.14 Gambaran Responden

Pengguna	Jumlah
Staff	12
Pengelola	18
Super Admin	2
Total Responden	32

Pengukuran kepuasan subjektif pengguna secara menyeluruh dilakukan menggunakan kuesioner *System Usability Scale* (SUS) yang diadopsi dari John Brooke. Responden akan diminta memberikan penilaian terhadap 10 butir pernyataan standar (lihat Tabel 2.7) dengan skala likert 1 sampai 5 (Lihat Tabel 3.18). Data mentah dari kuesioner tersebut selanjutnya akan dikalkulasi menggunakan aturan perhitungan skor SUS, di mana skor pernyataan ganjil dikurangi 1, dan 5 dikurangi skor pernyataan genap, kemudian totalnya dikalikan 2.5 untuk mendapatkan nilai akhir (0-100). Simulasi perhitungan untuk satu responden dapat dilihat pada Tabel 3.19.

Tabel 3.15 Skala Likert 1-5 [53]

No.	Skor	Keterangan
1.	5	Sangat Setuju (SS)
2.	4	Setuju (S)
3.	3	Netral (N)
4.	2	Tidak Setuju (TS)
5.	1	Sangat Tidak Setuju (STS)

Tabel 3.16 Simulasi Perhitungan 1 Responden SUS

No.	Tipe Pertanyaan	Skor Responden (1-5)	Rumus Perhitungan	Skor Akhir Poin
1.	Ganjil	4	4-1	3
2.	Genap	2	5-2	3
3.	Ganjil	5	5-1	4
4.	Genap	1	5-1	4
5.	Ganjil	5	5-1	4
6.	Genap	2	5-2	3
7.	Ganjil	4	4-1	3
8.	Genap	2	5-2	3

No.	Tipe Pertanyaan	Skor Responden (1-5)	Rumus Perhitungan	Skor Akhir Poin
9.	Ganjil	4	4-1	3
10.	Genap	2	5-2	3
Total Skor				33
Nilai Akhir SUS			33×2.5	82.5

Berdasarkan hasil simulasi perhitungan pada Tabel 3.13, diperoleh skor akhir *System Usability Scale* (SUS) sebesar 82,5. Untuk menentukan kelayakan sistem, nilai tersebut diinterpretasikan menggunakan pedoman visual penilaian SUS (Brooke, 2013) sebagaimana ditunjukkan pada Gambar 2.5.

Hasil pemetaan skor 82,5 menunjukkan bahwa sistem berada pada kategori Grade A, yang mengindikasikan performa kegunaan sangat tinggi. Secara kualitatif melalui *Adjective Rating*, skor ini diklasifikasikan sebagai "*Excellent*" dan masuk dalam rentang "*Acceptable*" dari sisi keberterimaan (*acceptability*) pengguna. Lebih jauh, jika ditinjau dari perspektif *Net Promoter Score* (NPS), nilai ini menempatkan pengguna pada kategori *Promoter*, yang menyiratkan bahwa pengguna merasa puas dan memiliki kecenderungan kuat untuk merekomendasikan sistem ini kepada pihak lain.

3.3.4 Analisis dan Kesimpulan

Tahap akhir penelitian ini berfokus pada analisis terhadap seluruh data yang diperoleh dari pengujian *Blackbox* dan *Usability*. Hasil analisis tersebut disintesis untuk mengevaluasi apakah fitur otomatisasi yang dibangun telah menjawab rumusan masalah, serta menarik kesimpulan mengenai kelayakan teknis dan operasional sistem sebelum diserahkan sepenuhnya kepada pihak penanggung jawab *greenhouse* Kebun Raya ITERA.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Implementasi Hasil

Tahap implementasi merupakan realisasi teknis dari rancangan desain yang telah didefinisikan pada BAB III, mengubah *wireframe* dan skema logika menjadi kode program yang fungsional. Seluruh antarmuka dikembangkan untuk memastikan interaksi pengguna yang responsif, sementara logika *backend* dibangun untuk menangani komunikasi data *real-time* melalui protokol MQTT dan *WebSocket*. Rincian lengkap mengenai komponen fitur yang berhasil diimplementasikan beserta deskripsi fungsi utamanya disajikan sebagai berikut.

4.1.1 Implementasi Arsitektur

Arsitektur *backend* pada subsistem otomatisasi perangkat IoT dibangun dengan menggabungkan beberapa pendekatan. Pendekatan *event-driven* digunakan untuk merespons data perangkat secara reaktif. Pendekatan *message queue* digunakan untuk memproses perintah aktuator secara *asynchronous*. Pendekatan *scheduled tasks* digunakan untuk menjalankan otomatisasi berbasis waktu secara periodik. Tabel 4.1 menyajikan komponen arsitektur yang digunakan dalam implementasi subsistem.

Tabel 4.1 Komponen Arsitektur Subsistem Otomatisasi Perangkat IoT

No	Komponen	Penjelasan
1	<i>Event-Driven</i>	Data sensor dan robot memicu <i>event</i> yang diproses untuk menjalankan aturan otomatisasi
2	<i>Message Queue</i>	Perintah aktuator diproses melalui <i>Laravel Queue</i> secara <i>asynchronous</i>
3	<i>Scheduled Tasks</i>	Cron job memeriksa jadwal otomatisasi setiap menit
4	<i>Real-time Broadcasting</i>	Pembaruan status disiarkan melalui <i>WebSocket</i> menggunakan <i>Laravel Reverb</i>
5	<i>Publish-Subscribe</i>	Protokol MQTT digunakan untuk komunikasi dengan perangkat IoT

Bagian berikut menjelaskan implementasi komponen arsitektur yang mendukung tiga jenis otomatisasi: berbasis penjadwalan, berbasis ambang batas sensor, dan berbasis data robot.

A. Penerimaan Data Perangkat melalui MQTT

Sistem menerima data dari perangkat IoT melalui protokol MQTT. Protokol ini memiliki *overhead* rendah sehingga cocok untuk perangkat dengan sumber daya terbatas. Penerimaan data dijalankan melalui *Artisan command* yang berjalan terus-menerus sebagai *long-running process* pada sisi server. *Command* tersebut berlangganan ke beberapa *topic* sekaligus dan mendelegasikan pemrosesan ke *handler* yang sesuai. Kode 4.1 menunjukkan pendaftaran *handler* untuk setiap *topic* MQTT.

Kode 4.1 Pendaftaran *handler topic* MQTT

```
// app/Console/Commands/MqttSubscribe.php

protected function registerTopicHandlers(): void
{
    $this->topicHandlers = [
        // Topic untuk perintah aktuator
        'iterahero/+/aktuator/+/cmd' => fn($topic, $message)
            => $this->handleAktuatorCommandTopic($topic, $message),

        // Topic untuk data sensor
        'iterahero/+/sensor/+/data' => fn($topic, $message)
            => $this->handleSensorDataTopic($topic, $message),

        // Topic untuk status robot
        'iterahero/robot/+/status' => fn($topic, $message)
            => $this->handleRobotStatusTopic($topic, $message),

        // Topic untuk data dinamis robot
        'iterahero/+/robot/+/data/+' => fn($topic, $message)
            => $this->handleRobotDynamicDataTopic($topic, $message),
    ];
}
```

B. Pemrosesan Data Sensor

Data sensor yang diterima melalui MQTT, sistem menjalankan beberapa proses secara berurutan. Pertama, sistem mengekstrak kode perangkat dari *topic* dan mencari sensor yang sesuai di basis data. Kedua, nilai mentah dikalibrasi menggunakan rumus yang dikonfigurasi pada sensor. Ketiga, data disimpan ke basis data dan disiarkan ke antarmuka pengguna melalui *WebSocket*. Terakhir, sistem mengevaluasi aturan otomatisasi berbasis ambang batas sensor. Kode 4.2 menunjukkan alur pemrosesan data sensor.

Kode 4.2 Alur pemrosesan data sensor

```
// app/Console/Commands/MqttSubscribe.php
public function handleSensorDataTopic(string $topic, string $message): void
{
    // Ekstrak kode perangkat dari topic
    preg_match('/^iterahero\/([^\|]+)\/sensor\/([^\|]+)\/data\/', $topic,
    $matches);
    [$location, $deviceCode] = [$matches[1], $matches[2]];

    // Cari sensor berdasarkan kode perangkat
    $sensor = Sensor::byDeviceCode($deviceCode)->with('threshold')->first();

    // Kalibrasi nilai mentah
    $rawValue = (float) $message;
    $calibrated = $sensor->calibrate($rawValue);

    // Siarkan ke frontend melalui WebSocket
    event(new SensorValueUpdated($sensor, $rawValue, $calibrated, true));

    // Simpan ke basis data
    SensorLog::create([
        'sensor uuid' => $sensor->uuid,
        'raw value' => $rawValue,
        'value' => $calibrated,
        'recorded at' => now()->utc(),
    ]);

    // Perbarui status sensor
    $sensor->update([
        'last seen at' => now()->utc(),
        'is online' => true,
        'last_value' => $calibrated,
    ]);

    // Proses aturan otomatisasi berbasis sensor
    $this->processSensorAutomations($sensor, $calibrated);
}
```

C. Evaluasi Kondisi Otomatisasi Berbasis Ambang Batas Sensor

Otomatisasi berbasis sensor bekerja dengan membandingkan nilai sensor terhadap konstanta ambang batas. Sistem mengambil semua aturan otomatisasi yang aktif untuk sensor tersebut dari basis data. Setiap aturan dievaluasi menggunakan *operator* perbandingan yang dikonfigurasi, seperti lebih besar (>), lebih kecil (<), sama dengan (==), dan tidak sama dengan (!=). Jika kondisi terpenuhi dan aktuator tidak sedang dikontrol otomatisasi lain,

sistem mengirim perintah ke aktuator. Kode 4.3 menunjukkan proses evaluasi kondisi otomatisasi sensor.

Kode 4.3 Evaluasi kondisi otomatisasi berbasis sensor

```
// app/Console/Commands/MqttSubscribe.php
protected function registerTopicHandlers(): void
{
    $this->topicHandlers = [
        // Topic untuk perintah aktuator
        'iterahero/+/actuator/+/cmd' => fn($topic, $message)
            => $this->handleActuatorCommandTopic($topic, $message),

        // Topic untuk data sensor
        'iterahero/+/sensor/+/data' => fn($topic, $message)
            => $this->handleSensorDataTopic($topic, $message),

        // Topic untuk status robot
        'iterahero/robot/+/status' => fn($topic, $message)
            => $this->handleRobotStatusTopic($topic, $message),

        // Topic untuk data dinamis robot
        'iterahero/+/robot/+/data/+' => fn($topic, $message)
            => $this->handleRobotDynamicDataTopic($topic, $message),
    ];
}
```

D. Pemrosesan Data Robot

Sistem juga menerima data dari robot melalui MQTT dengan format *topic* yang berbeda dari sensor. Data robot memiliki tipe yang beragam, meliputi angka, teks, *boolean*, dan enumerasi. Ketika data diterima, sistem memvalidasi nilai sesuai tipe data yang didefinisikan, kemudian menyimpan ke basis data. Setelah penyimpanan, sistem mengevaluasi aturan otomatisasi berbasis data robot yang aktif. Kode 4.4 menunjukkan alur pemrosesan data robot.

Kode 4.4 Alur pemrosesan data robot

```
// app/Console/Commands/MqttSubscribe.php

public function handleRobotDynamicDataTopic(string $topic, string $message): void
{
    // Ekstrak informasi dari topic
    preg_match('/^iterahero\/([^\/]+)\\/robot\/([^\/]+)\\/data\/([^\/]+)$/ ', $topic,
    $matches);
    [$location, $robotCode, $dataKey] = [$matches[1], $matches[2], $matches[3]];

    // Cari definisi data robot
    $robotDataDefinition = RobotDataDefinition::where('topic', $topic)
        ->with('robot')
        ->first();
    $robot = $robotDataDefinition?->robot;

    // Validasi nilai enum
    $dataDefinition = RobotDataDefinition::where('robot_uuid', $robot->uuid)
        ->where('key', $dataKey)
        ->first();

    if ($dataDefinition && $dataDefinition->type === 'enum') {
        $enumOptions = $dataDefinition->enum_options;
        if (!in_array($message, $enumOptions)) {
            return;
        }
    }

    // Simpan log data robot
    RobotDataLog::create([
        'robot_uuid' => $robot->uuid,
        'key' => $dataKey,
        'value' => $message,
        'recorded_at' => now(),
    ]);

    // Siarkan pembaruan status robot
    event(new RobotStatusUpdated($robot));

    // Proses aturan otomatisasi berbasis data robot
    $this->processRobotAutomations($robot, $dataKey, $message);
}
```

E. Evaluasi Kondisi Otomatisasi Berbasis Data Robot

Evaluasi kondisi untuk data robot mendukung berbagai tipe data. Untuk tipe angka, sistem menggunakan *operator* perbandingan numerik standar. Untuk tipe teks dan enumerasi, sistem mendukung *operator* tambahan seperti *contains*, *startsWith*, dan *endsWith*. Jika aturan memiliki konfigurasi iterasi, eksekusi didelegasikan ke *job* terpisah. Jika tidak, aktuatur dieksekusi langsung dengan opsi penjadwalan *auto-off* sesuai durasi. Kode 4.5 menunjukkan logika evaluasi kondisi dan eksekusi otomatisasi robot.

F. Penjadwalan Otomatisasi Berbasis Waktu

Otomatisasi berbasis waktu menggunakan *task scheduling* yang berjalan setiap menit melalui *cron*. Sistem memeriksa jadwal yang aktif dan mencocokkan dengan hari serta waktu saat ini. Jadwal yang sudah dijalankan pada hari yang sama tidak akan dieksekusi ulang. Untuk jadwal dengan konfigurasi iterasi, eksekusi didelegasikan ke *job* terpisah. Kode 4.6 menunjukkan konfigurasi *task scheduling*.

G. Eksekusi Iterasi Penjadwalan

Jadwal dengan konfigurasi iterasi diproses oleh *job* terpisah yang menjadwalkan beberapa siklus nyala-mati. Setiap siklus terdiri dari durasi

aktuator menyala ditambah interval jeda sebelum siklus berikutnya. Sistem memeriksa *lock* aktuator sebelum menjadwalkan siklus untuk mencegah konflik. Waktu yang sudah terlewat lebih dari 10 detik akan dilewati. Kode 4.8 menunjukkan implementasi *job* iterasi penjadwalan.

H. Eksekusi Siklus Aktuator

Setiap siklus aktuator dieksekusi oleh *job* terpisah yang menyalakan aktuator dan menjadwalkan waktu mati. Sistem mengakuisisi *lock* dengan *TTL (Time to Live)* sesuai durasi ditambah *buffer* 30 detik. Jika *lock* gagal diperoleh, siklus dilewati untuk mencegah konflik. Setelah *lock* berhasil, perintah nyala dikirim dan perintah mati dijadwalkan sesuai durasi. Kode 4.9 menunjukkan implementasi *job* siklus aktuator.

I. Eksekusi Perintah Aktuator melalui Antrian

Perintah untuk menyalakan atau mematikan aktuator diproses melalui sistem antrian (*queue*). Pendekatan ini memastikan proses utama tidak terhambat oleh komunikasi MQTT. *Job* mengimplementasikan *interface ShouldBeUnique* untuk mencegah duplikasi perintah. Setelah perintah dikirim melalui MQTT, status aktuator diperbarui di basis data dan disiarkan ke *frontend*. Kode 4.10 menunjukkan implementasi *job* untuk menyalakan aktuator.

J. Pencegahan Konflik Antar Otomatisasi

Ketika beberapa aturan otomatisasi menargetkan aktuator yang sama, sistem menggunakan mekanisme *lock* berbasis *cache*. Setiap jenis otomatisasi memiliki tipe *lock* tersendiri: *schedule*, *sensor_threshold*, *robot_automation*, dan *cycle*. Sistem hanya mengizinkan otomatisasi dengan sumber yang sama untuk memperbarui *lock* yang sudah ada. *Lock* memiliki *TTL* yang akan kedaluwarsa secara otomatis jika tidak diperbarui. Kode 4.11 menunjukkan implementasi manajemen *lock* aktuator.

K. Penyiaran Status Perangkat melalui WebSocket

Setiap perubahan status perangkat disiarkan ke antarmuka pengguna melalui *WebSocket*. Sistem menggunakan *Laravel Reverb* sebagai *WebSocket server* dan *Laravel Echo* di sisi *frontend*. *Event* mengimplementasikan *interface ShouldBroadcastNow* untuk penyiaran langsung tanpa melalui antrian. *Channel* dibuat berdasarkan UUID perangkat dan juga lokasi perangkat untuk mendukung langganan terpadu. Kode 4.12 menunjukkan implementasi *event* penyiaran status sensor.

L. Penerimaan Pembaruan Real-time di Frontend

Antarmuka pengguna menerima pembaruan status melalui *WebSocket* menggunakan *Laravel Echo*. Sistem menggunakan pola *subscription manager* untuk mengelola koneksi secara efisien. Kelas ini menyimpan referensi *channel* yang sudah dibuat untuk mencegah duplikasi langganan. Ketika data diterima, *state* diperbarui melalui *store* yang dikelola

oleh *Zustand*. Kode 4.13 menunjukkan implementasi penerimaan pembaruan status aktuator di sisi *frontend*.

M. Otorisasi Channel *WebSocket*

Setiap *channel WebSocket* memerlukan otorisasi untuk memastikan hanya pengguna yang berhak dapat menerima pembaruan. Sistem menggunakan mekanisme *callback* yang memvalidasi identitas pengguna sebelum mengizinkan akses ke *channel*. Konfigurasi otorisasi didefinisikan pada berkas *routes* dan diproses oleh *Laravel Broadcasting*. Untuk *channel* umum seperti sensor dan aktuator, semua pengguna terautentikasi diizinkan berlangganan. Kode 4.14 menunjukkan konfigurasi otorisasi *channel*.

N. Notifikasi *Push* Otomatisasi

Setiap eksekusi otomatisasi menghasilkan notifikasi *push* yang dikirim ke pengguna yang berwenang. Sistem menggunakan *Web Push Notification* yang memanfaatkan *Service Worker* di sisi *browser*. Notifikasi dikirim melalui dua *channel*: *WebPush* untuk notifikasi *browser* dan *BroadcastDatabaseChannel* untuk penyimpanan dan penyiaran *real-time*. Mekanisme *throttling* berbasis *cache* mencegah pengiriman notifikasi berulang dalam interval 60 detik. Kode 4.15 menunjukkan implementasi *service* pengiriman notifikasi otomatisasi.

O. Pencatatan Log Aktivitas Otomatisasi

Setiap eksekusi otomatisasi dicatat dalam dua jenis log: log eksekusi otomatisasi dan log aktivitas umum. Log eksekusi otomatisasi (*AutomationExecutionLog*) menyimpan informasi spesifik seperti tipe otomatisasi, status keberhasilan, dan metadata eksekusi. Log aktivitas umum (*ActivityLog*) menyimpan catatan yang lebih komprehensif termasuk informasi pengguna, alamat IP, dan kategori aksi. Pemisahan ini memungkinkan agregasi data untuk ringkasan harian sekaligus pelacakan aktivitas sistem secara menyeluruh. Kode 4.17 menunjukkan metode pencatatan log eksekusi otomatisasi.

1. Komunikasi *Real-Time* dengan *WebSockets*

Menangani pemantauan data sensor secara *real-time*, sistem menerapkan protokol *WebSocket* menggunakan *Laravel Reverb*. Berbeda dengan metode *XHR Polling* yang mengharuskan *client* meminta data secara berulang setiap 1000 milidetik, mekanisme ini memungkinkan *server* untuk melakukan *push* data secara langsung ketika terjadi perubahan nilai pada sensor. Implementasi ini direalisasikan melalui mekanisme *Event Broadcasting*.

Kode Program 4.1 menunjukkan kelas (SensorValueUpdated) yang mengimplementasikan

antarmuka (ShouldBroadcastNow). Penggunaan (ShouldBroadcastNow) menginstruksikan sistem untuk menyiarkan *event* tersebut secara langsung tanpa masuk ke antrian, sehingga data dikirimkan ke *dashboard* pengguna pada saat *event* terjadi. *Event* ini menyiarkan data ke PrivateChannel yang spesifik untuk setiap sensor, memastikan hanya pengguna yang ter-autentikasi dan memiliki akses ke sensor tersebut yang menerima pembaruan data.

```
class SensorValueUpdated implements ShouldBroadcastNow
{
    use SerializesModels;
    public function __construct(
        public Sensor $sensor,
        public float $raw,
        public float $value,
        public bool $is_online
    ) {
        // Inisialisasi data sensor yang akan disiarkan
    }
    public function broadcastOn(): array
    {
        // Menyiarkan ke channel privat spesifik per sensor
        return [
            new PrivateChannel('sensor.' . $this->sensor->uuid),
        ];
    }
    public function broadcastAs(): string
    {
        return 'sensor.value.updated';
    }
    public function broadcastWith(): array
    {
        return [
            'sensor_uuid' => $this->sensor->uuid,
            'device_code' => $this->sensor->device_code,
            'raw_value' => $this->raw,
            'value' => $this->value,
            'is_online' => $this->is_online,
            'timestamp' => now()->toIso8601String()
        ];
    }
}
```

Kode 4.5 Implementasi *Event Broadcasting* Sensor

2. Integrasi *WebSocket* pada Sisi Klien

Pada sisi antarmuka pengguna (*frontend*), aplikasi menggunakan pustaka Laravel Echo untuk berlangganan (*subscribe*) ke saluran siaran yang telah didefinisikan di sisi *server*. Implementasi ini dilakukan di dalam komponen React menggunakan *hook* *useEffect* untuk mengelola siklus hidup koneksi *WebSocket*. Kode Program 4.2 memperlihatkan mekanisme *listening* terhadap *event* *sensor.value.updated* pada saluran privat. Ketika data baru diterima dari *server* melalui *WebSocket*, *state* lokal komponen diperbarui secara langsung, memicu *re-render* pada komponen antarmuka.

```
useEffect(() => {
    if (typeof window.Echo === 'undefined' || !sensor.uuid) return;
    // Berlangganan ke private channel spesifik sensor
    const channelName = `sensor.${sensor.uuid}`;
    const channel = window.Echo.private(channelName);
    const handleUpdate = (event: {
        value: number;
        is_online: boolean
    }) => {
```

```

// Memperbarui state tampilan saat event diterima
setCurrentValue(event.value);
setCurrentIsOnline(event.is_online);
};
// Mendengarkan event spesifik dengan prefix dot
channel.listen('.sensor.value.updated', handleUpdate);
// Cleanup: menutup koneksi saat komponen unmount
return () => {
    window.Echo.leave(channelName);
};
}, [sensor.uuid]);

```

Kode 4.6 Implementasi *Client-side WebSocket Listener*

3. MQTT *Subscriber* untuk Penerimaan Data IoT

Komponen utama dalam arsitektur ini adalah layanan MQTT Subscriber yang berjalan sebagai proses background. Layanan ini bertugas menerima data dari perangkat IoT melalui broker MQTT dan memproses data tersebut sesuai dengan aturan otomatisasi yang telah dikonfigurasi. Kode Program 4.3 menunjukkan implementasi layanan subscriber yang menangani beberapa topic MQTT secara bersamaan. Layanan ini mendaftarkan beberapa *handler* untuk *topic* yang berbeda menggunakan pola *wildcard* (+). Setiap *handler* memproses pesan sesuai dengan jenis datanya: data sensor, perintah aktuator, atau data robot.

```

class MqttSubscribe extends Command
{
    protected $signature = 'mqtt:subscribe';
    protected array $topicHandlers = [];
    public function handle(): void
    {
        $this->registerTopicHandlers();

        $mqttClient = new MqttClient(
            env('MQTT_HOST', 'mosquitto'),
            (int) env('MQTT_PORT', 1883),
            env('MQTT_CLIENT_ID', 'laravel-mqtt-client')
        );
        $mqttClient->connect();
        foreach ($this->topicHandlers as $pattern => $handler) {
            $mqttClient->subscribe($pattern, $handler, 0);
        }
        $mqttClient->loop(true);
    }
    protected function registerTopicHandlers(): void
    {
        $this->topicHandlers = [
            // Handler untuk data sensor
            'iterahero/+sensor/+/data' => fn($topic, $message)
                => $this->handleSensorDataTopic($topic, $message),
            // Handler untuk perintah aktuator
            'iterahero/+actuator/+/cmd' => fn($topic, $message)
                => $this->handleActuatorCommandTopic($topic, $message),
            // Handler untuk status robot
            'iterahero/robot/+/status' => fn($topic, $message)
                => $this->handleRobotStatusTopic($topic, $message),
            // Handler untuk data dinamis robot
            'iterahero/+robot/+/data/+' => fn($topic, $message)
                => $this->handleRobotDynamicDataTopic($topic, $message),
        ];
    }
}

```

Kode 4.7 Implementasi *MQTT Subscriber*

4. Pemrosesan Data Sensor dan Evaluasi Otomatisasi

Ketika data sensor diterima melalui MQTT, sistem melakukan serangkaian proses: kalibrasi nilai, penyimpanan

log, *broadcasting* ke *frontend*, dan evaluasi aturan otomatisasi. Kode Program 4.4 menunjukkan alur pemrosesan data sensor.

```
public function handleSensorDataTopic(string $topic, string $message): void
{
    // Parsing topic untuk mendapatkan lokasi dan kode perangkat
    preg_match('/^iterahero\/([^\/]+)\sensor\/([^\/]+)\data$/',
        $topic, $matches);
    [$locationCode, $deviceCode] = [$matches[1], $matches[2]];

    $sensor = Sensor::byDeviceCode($deviceCode)->with('threshold')->first();

    // Kalibrasi nilai mentah
    $rawValue = (float) $message;
    $calibrated = $sensor->calibrate($rawValue);
    // Broadcast ke frontend via WebSocket
    event(new SensorValueUpdated($sensor, $rawValue, $calibrated, true));
    // Simpan log ke database
    $log = SensorLog::create([
        'sensor_uuid' => $sensor->uuid,
        'raw_value' => $rawValue,
        'value' => $calibrated,
        'recorded_at' => now(),
    ]);
    // Evaluasi threshold untuk notifikasi peringatan
    app(SensorThresholdService::class)->evaluate($sensor, $log);
    // Update status online sensor
    $sensor->update(['last_seen_at' => now(), 'is_online' => true]);
    // Proses aturan otomatisasi berbasis sensor
    $this->processAutomations($sensor, $calibrated);
}
```

Kode 4.8 Pemrosesan Data Sensor

5. Eksekusi Otomatisasi Berbasis Ambang Batas Sensor

Aturan otomatisasi berbasis sensor dievaluasi setiap kali data sensor baru diterima. Jika kondisi aturan terpenuhi, sistem mengirimkan perintah ke aktuator terkait. Kode Program 4.5 menunjukkan logika evaluasi dan eksekusi otomatisasi.

```
public function processAutomations(Sensor $sensor, float $calibrated): void
{
    $automations = AutomationSensor::where('sensor_uuid', $sensor->uuid)
        ->where('is_active', true)
        ->with('actuator')
        ->get();

    foreach ($automations as $automation) {
        // Evaluasi kondisi (>, >=, <, <=, ==, !=)
        $conditionMet = $this->evaluateCondition(
            $calibrated,
            $automation->condition,
            $automation->constant
        );

        if (!$conditionMet) continue;

        $actuator = $automation->actuator;

        // Skip jika aktuator sudah dalam status yang diinginkan
        if ($actuator->is_active === $automation->action) continue;
        // Cek dan akuisisi lock untuk mencegah konflik
        $lockStatus = ActuatorLockManager::isLocked($actuator->uuid);
        if ($lockStatus['is_locked'] &&
            $lockStatus['lock_info']['lock_type'] !== 'sensor') {
            continue; // Skip jika terkunci oleh tipe otomatisasi lain
        }
        ActuatorLockManager::acquireLock(
            $actuator->uuid,
            ActuatorLockManager::LOCK_TYPE_SENSOR,
            $automation->uuid,
            60 // TTL 1 menit
        );
        // Dispatch job ke queue untuk eksekusi
        $this->dispatchActuatorAction(
```

```

        $actuator->uuid,
        $automation->action,
        $sensor->name
    );
    // Kirim notifikasi ke pengguna
    AutomationNotificationService::notifySensorTriggered(
        $automation->uuid,
        $sensor->name,
        $actuator->name,
        $calibrated,
        $automation->condition,
        $automation->constant,
        $automation->action,
        $sensor->unit?->symbol
    );
}
}

```

Kode 4.9 Evaluasi dan Eksekusi Otomatisasi Sensor

6. Otomatisasi *Asynchronous* dengan *Queue Worker*

Manajemen otomatisasi perangkat, seperti penjadwalan penyiraman atau pengendalian aktuator, dipisahkan dari proses utama aplikasi web menggunakan mekanisme *Queue Worker*. Pendekatan ini memisahkan proses eksekusi dari interaksi pengguna pada antarmuka website. Sistem menggunakan *Task Scheduler* (Kode Program 4.6) yang berjalan setiap menit untuk memeriksa jadwal aktif. Ketika jadwal terpenuhi, sistem tidak langsung mengeksekusi perintah ke perangkat keras, melainkan mendispatch *Job* ke dalam antrian Redis yang dikelola oleh Laravel Horizon.

```

// routes/console.php - Konfigurasi Task Scheduler
// Menjalankan pengecekan jadwal otomatisasi setiap menit
Schedule::command('automation:run-schedules')->everyMinute();
// Memeriksa status koneksi perangkat (heartbeat) setiap menit
Schedule::command('devices:check-status')->everyMinute();
// Mengirim rangkuman harian pada pukul 18:00 WIB
Schedule::command('summary:send-daily')
->dailyAt('18:00')
->timezone('Asia/Jakarta');

```

Kode 4.10 Implementasi *Task Scheduler*

Eksekusi logika kendali pada fitur otomatisasi dijalankan dan dieksekusi oleh kelas *Job* seperti *RunActuatorCycle* (Kode Program 4.7). *Job* ini berjalan di latar belakang (*background process*) pada antrian khusus bernama *schedules*. Dengan arsitektur ini, sistem memproses perintah otomatisasi secara paralel di latar belakang.

```

class RunActuatorCycle implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public function __construct(string $actuatorUuid, int $duration, Carbon $startTime)
    {
        // Mengirim job ke antrian khusus 'schedules'
        $this->onQueue('schedules');
    }

    public function handle(): void
    {
        // Logika eksekusi: Menghitung waktu berhenti dan mengirim perintah ke MQTT
        $expectedOffTime = $this->startTime->copy()->addSeconds($this->duration);

        // ... logika pengendalian perangkat ...
    }
}

```

Kode 4.11 Implementasi *Job Queue* untuk Aktuator

Penerapan arsitektur hibrida ini menggabungkan *WebSockets* untuk jalur data sensor dan Queue Worker untuk jalur otomatisasi, membentuk struktur sistem yang memadukan komunikasi data sensor dan pemrosesan otomatisasi sesuai dengan perancangan sistem.

7. Pemrosesan Jadwal Otomatisasi

Kode Program 4.8 menunjukkan implementasi perintah yang memproses jadwal otomatisasi. Perintah ini berjalan setiap menit dan memeriksa jadwal yang memenuhi kriteria waktu dan hari.

```
class RunAutomationSchedules extends Command
{
    protected $signature = 'automation:run-schedules';
    public function handle()
    {
        $now = Carbon::now();
        $day = $now->dayOfWeek; // 0 = Minggu, 6 = Sabtu
        $time = $now->format('H:i'); // Waktu dalam format 24 jam
        // Query jadwal aktif yang sesuai dengan hari dan waktu
        $schedules = AutomationSchedule::with('actuator')
            ->where('is_active', true)
            ->whereJsonContains('days', $day)
            ->where('start_time', '<=', $time)
            ->where(function ($query) use ($now) {
                $query->whereNull('last_run_at')
                    ->orWhereDate('last_run_at', '<', $now->toDateString());
            })
            ->get();
        foreach ($schedules as $schedule) {
            $actuator = $schedule->actuator;

            if (is_null($schedule->interval) || is_null($schedule->iterations)) {
                // Mode single-run: langsung eksekusi ON + jadwalkan OFF
                dispatch(new TurnActuatorOn(
                    $actuator->uuid, false, $actuator->name
                ));
                $expectedOffTime = Carbon::createFromFormat('H:i', $schedule-
                    >start_time)
                    ->setDateFrom($now)
                    ->addSeconds($schedule->duration);
                dispatch(new TurnActuatorOff(
                    $actuator->uuid, false, $actuator->name
                ))->delay($expectedOffTime);
            } else {
                // Mode interval: delegasikan ke job iterasi
                dispatch(new RunScheduleIterationsJob($schedule->uuid));
            }
            // Tandai jadwal sudah dieksekusi hari ini
            $schedule->update(['last_run_at' => now()]);
        }
    }
}
```

Kode 4.12 Pemrosesan Jadwal Otomatisasi

8. Eksekusi Job Aktuator

Eksekusi logika kendali dijalankan oleh kelas Job seperti *TurnActuatorOn* (Kode Program 4.9). Job ini berjalan di background process pada antrian khusus bernama *actuators*, memproses perintah otomatisasi secara paralel tanpa mengganggu respons web server.

```
class TurnActuatorOn implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;
    public function __construct(
        public string $actuatorUuid,
        public bool $isManual = false,
        public string $actuatorName,
        public ?string $userId = null,
        public ?string $userName = null
    ) {
    }
```

```

        $this->onQueue('actuators');
    }
    public function handle(): void
    {
        $actuator = Actuator::where('uuid', $this->actuatorUuid)->firstOrFail();
        // Skip jika sudah dalam status ON
        if ($actuator->is_active) return;
        // Inisialisasi koneksi MQTT
        $mqttClient = new MqttClient(
            env('MQTT_HOST', 'mosquitto'),
            (int) env('MQTT_PORT', 1883),
            'publisher-on-' . $actuator->uuid . '-' . Str::random(8)
        );

        $mqttClient->connect();

        // Publish perintah ON ke topic aktuator dengan retained flag
        $mqttClient->publish(
            $actuator->topic, // Topic: iterahero/{lokasi}/aktuator/{kode}/cmd
            'ON', // Payload
            1, // QoS Level 1 (at least once)
            true // Retained message
        );

        $mqttClient->disconnect();
        // Update status aktuator di database
        $actuator->update(['is_active' => true]);
        $actuator->refresh();
        // Broadcast perubahan ke frontend via WebSocket
        broadcast(new ActuatorToggled($actuator));
        // Log ke database untuk audit trail
        ActuatorLog::create([
            'actuator_uuid' => $actuator->uuid,
            'value' => 'ON',
            'is_manual' => $this->isManual,
            'recorded_at' => now(),
        ]);
        // Kirim notifikasi ke pengguna
        $this->sendNotification($actuator);
    }
}

```

Kode 4.13 Implementasi Job untuk Menyalakan Aktuator

9. Otomatisasi Berbasis Data Robot

Selain sensor dan penjadwalan, sistem juga mendukung otomatisasi berbasis data rekomendasi dari robot. Kode Program 4.10 menunjukkan pemrosesan data robot dan evaluasi aturan otomatisasi terkait.

```

public function processRobotAutomations(Robot $robot, string $key, mixed $value):
void
{
    $automations = AutomationRobot::with(['dataDefinition', 'actuator'])
        ->where('robot_uuid', $robot->uuid)
        ->whereHas('dataDefinition', function ($q) use ($key) {
            $q->where('key', $key);
        })
        ->where('is_active', true)
        ->get();
    foreach ($automations as $automation) {
        $dataDefinition = $automation->dataDefinition;

        // Evaluasi kondisi berdasarkan tipe data (number, boolean, enum)
        $conditionMet = $this->evaluateRobotCondition(
            $value,
            $automation->condition,
            $automation->constant,
            $dataDefinition->type
        );
        if ($conditionMet) {
            $actuator = $automation->actuator;

            // Skip jika status sudah sesuai
            if ($actuator->is_active === $automation->action) continue;
            // Cek dan akuisisi lock
            $lockStatus = ActuatorLockManager::isLocked($actuator->uuid);

```

```

        if ($lockStatus['is_locked'] &&
            $lockStatus['lock_info']['lock_type'] !== 'robot') {
            continue;
        }
        $lockTtl = ($automation->duration ?? 60) + 30;
        ActuatorLockManager::acquireLock(
            $actuator->uuid,
            ActuatorLockManager::LOCK_TYPE_ROBOT,
            $automation->uuid,
            $lockTtl
        );
        // Dispatch job untuk eksekusi
        $this->dispatchActuatorAction(
            $actuator->uuid,
            $automation->action,
            $robot->name
        );
        // Notifikasi
        AutomationNotificationService::notifyRobotTriggered(
            $automation->uuid,
            $robot->name,
            $dataDefinition->name ?? $dataDefinition->key,
            $actuator->name,
            $value,
            $automation->condition,
            $automation->constant,
            $automation->action,
            $automation->duration
        );
    }
}
}
}

```

Kode 4.14 Evaluasi dan Eksekusi Otomatisasi Robot

4.1.2 Implementasi *Backend*

Implementasi *backend* dibangun menggunakan kerangka kerja Laravel dengan menerapkan pola desain *Service Layer* untuk memisahkan logika bisnis dari *controller*. Fokus utama pengembangan ini mencakup penanganan fungsionalitas inti seperti manajemen perangkat sensor dan aktuator, serta eksekusi berbagai skenario otomatisasi cerdas. Sistem ini juga mengintegrasikan fitur keamanan menyeluruh yang meliputi otentikasi pengguna, pencatatan *log* aktivitas, dan pembagian hak akses berbasis peran. Rincian lengkap mengenai komponen teknis berupa *class*, *file*, dan *route* yang mendukung seluruh fungsi tersebut disajikan dalam Tabel 4.1.

Tabel 4.2 Implementasi *Backend* Subsistem Otomatisasi Perangkat IoT

No	Fitur	Komponen (Class/File) dan Route	Deskripsi Fungsi
1	Login	<i>AuthenticatedSessionController</i> , <i>LoginRequest</i> , <i>User (Model)</i> , <i>Authenticate (Middleware)</i> , <i>RedirectIfAuthenticated (Middleware)</i> , <i>ActivityLogger (Service)</i>	Menangani proses otentikasi pengguna termasuk validasi kredensial, manajemen sesi, dan pencatatan aktivitas <i>login/logout</i> . <i>Middleware</i> memastikan hanya pengguna terautentikasi yang dapat mengakses subsistem.
		Route	
		<i>GET /auth/login</i>	Menampilkan halaman <i>login</i>
		<i>POST /auth/login</i>	Proses autentikasi kredensial

No	Fitur	Komponen (Class/File) dan Route	Deskripsi Fungsi
		<i>POST /auth/logout</i>	Proses <i>logout</i> dan hapus sesi
2	Perangkat Sensor	<i>SensorController, SensorApiController, SensorService, SensorThresholdService, Sensor (Model), SensorLog (Model), SensorType (Model), SensorUnit (Model), SensorThreshold (Model), SensorResource, SensorLogResource, SensorObserver, SensorLogObserver, SensorValueUpdated (Event), StoreSensorRequest, UpdateSensorRequest, StoreSensorTypeRequest, StoreSensorUnitRequest, UpdateSensorThresholdRequest</i>	Mengelola operasi tambah, ubah, dan hapus sensor, penyimpanan riwayat nilai, konfigurasi tipe dan satuan, serta <i>broadcast</i> nilai terbaru ke <i>frontend</i> melalui <i>WebSocket</i> . Data sensor diperbarui secara <i>realtime</i> saat diterima via MQTT.
		Route	
		<i>GET /dashboard/monitoring</i>	Daftar semua sensor dengan <i>filter</i>
		<i>GET /dashboard/monitoring/create</i>	Form tambah sensor baru
		<i>POST /dashboard/monitoring/create</i>	Simpan sensor baru
		<i>GET /dashboard/monitoring/{uuid}</i>	Detail sensor dan grafik
		<i>GET /dashboard/monitoring/{uuid}/edit</i>	Form edit sensor
		<i>PUT /dashboard/monitoring/{uuid}</i>	Update data sensor
		<i>DELETE /dashboard/monitoring/{uuid}</i>	Hapus sensor
		<i>GET /dashboard/monitoring/{uuid}/logs</i>	Riwayat <i>log</i> sensor
		<i>GET /dashboard/monitoring/{uuid}/logs/download</i>	Unduh <i>log</i> sensor (CSV)
		<i>GET /dashboard/monitoring/list/types</i>	Daftar tipe sensor
		<i>GET /dashboard/monitoring/api/data</i>	API data daftar sensor
		<i>POST /dashboard/monitoring/api/sensor-logs/range</i>	API <i>log</i> sensor dari <i>range</i>
		<i>GET /dashboard/monitoring/api/{uuid}/logs</i>	API data <i>log</i> sensor
		<i>GET /dashboard/monitoring/api/{uuid}/stats</i>	API statistik sensor
		<i>POST /dashboard/monitoring/types</i>	Tambah tipe sensor
		<i>DELETE /dashboard/monitoring/types/{uuid}</i>	Hapus tipe sensor
		<i>POST /dashboard/monitoring/unit</i>	Tambah satuan sensor
		<i>DELETE /dashboard/monitoring/unit/{uuid}</i>	Hapus satuan sensor
3	Perangkat Aktuator	<i>ActuatorController, ActuatorService, TopicGenerator (Service), Actuator (Model), ActuatorLog (Model), ActuatorType (Model), ActuatorResource, ActuatorObserver, ActuatorLogObserver, ActuatorToggled (Event), ActuatorException, StoreActuatorRequest, UpdateActuatorRequest, StoreActuatorTypeRequest, TurnActuatorOn (Job), TurnActuatorOff (Job), TurnActuatorOffDelayed (Job)</i>	Mengelola operasi tambah, ubah, dan hapus aktuator, <i>toggle</i> status ON/OFF secara manual, penyimpanan riwayat perubahan status, dan komunikasi dengan perangkat melalui protokol MQTT. Perubahan status di- <i>broadcast</i> ke <i>frontend</i> via <i>WebSocket</i> .
		Route	
		<i>GET /dashboard/controlling</i>	Daftar semua aktuator
		<i>GET /dashboard/controlling/create</i>	Form tambah aktuator
		<i>POST /dashboard/controlling/create</i>	Simpan aktuator baru
		<i>GET /dashboard/controlling/{uuid}</i>	Detail aktuator
		<i>GET /dashboard/controlling/{uuid}/edit</i>	Form ubah aktuator
		<i>POST /dashboard/controlling/{uuid}/edit</i>	Simpan ubah aktuator
		<i>DELETE /dashboard/controlling/{uuid}</i>	Hapus aktuator
		<i>POST /dashboard/controlling/{uuid}/toggle</i>	<i>Toggle</i> ON/OFF aktuator
		<i>GET /dashboard/controlling/{uuid}/logs</i>	Riwayat <i>log</i> aktuator
		<i>GET /dashboard/controlling/actuator/list/types</i>	Daftar tipe aktuator
		<i>GET /dashboard/controlling/api/data</i>	API data aktuator
		<i>GET /dashboard/controlling/api/{uuid}/logs</i>	API data <i>log</i> aktuator
		<i>GET /dashboard/controlling/api/{uuid}/stats</i>	API statistik aktuator
		<i>POST /dashboard/controlling/types</i>	Tambah tipe aktuator
		<i>DELETE /dashboard/monitoring/types/{uuid}</i>	Hapus tipe aktuator

No	Fitur	Komponen (Class/File) dan Route	Deskripsi Fungsi
4	Otomatisasi Berdasarkan Penjadwalan	<i>AutomationController</i> , <i>AutomationScheduleController</i> , <i>AutomationScheduleService</i> , <i>ActuatorLockManager</i> (Service), <i>AutomationConflictDetector</i> (Service), <i>AutomationNotificationService</i> (Service), <i>AutomationSchedule</i> (Model), <i>AutomationExecutionLog</i> (Model), <i>StoreAutomationScheduleRequest</i> , <i>UpdateAutomationScheduleRequest</i> , <i>RunAutomationSchedules</i> (Command), <i>SendDailyAutomationSummary</i> (Command), <i>RunScheduleIterationsJob</i> (Job), <i>RunActuatorCycle</i> (Job), <i>AutomationUpdated</i> (Event)	Mengelola aturan otomatisasi berbasis waktu dengan konfigurasi durasi, waktu mulai, hari aktif, <i>interval</i> , dan iterasi. <i>Command</i> dijalankan setiap menit oleh <i>cron</i> untuk mengeksekusi jadwal yang sesuai. Sistem <i>lock</i> mencegah konflik antar otomatisasi.
		Route	
		<i>GET /dashboard/automation</i>	Halaman utama otomatisasi
		<i>GET /dashboard/automation/schedules/create</i>	Form tambah jadwal
		<i>POST /dashboard/automation/schedules</i>	Simpan jadwal baru
		<i>GET /dashboard/automation/schedules/{uuid}</i>	Detail jadwal
		<i>GET /dashboard/automation/schedules/{uuid}/edit</i>	Form edit jadwal
		<i>PUT /dashboard/automation/schedules/{uuid}</i>	Ubah jadwal
		<i>DELETE /dashboard/automation/schedules/{uuid}</i>	Hapus jadwal
		<i>POST /dashboard/automation/schedules/{uuid}/toggle</i>	<i>Toggle</i> aktif/nonaktif
		<i>POST /dashboard/automation/schedules/{uuid}/test</i>	Uji coba eksekusi jadwal
5	Otomatisasi Berdasarkan Ambang Batas Sensor	<i>AutomationSensorController</i> , <i>AutomationSensorService</i> , <i>SensorThresholdService</i> (Service), <i>ActuatorLockManager</i> (Service), <i>AutomationSensor</i> (Model), <i>StoreAutomationSensorRequest</i> , <i>UpdateAutomationSensorRequest</i> , <i>MqttSubscribe</i> (Command)	Mengelola aturan otomatisasi berbasis kondisi sensor dengan konfigurasi operator perbandingan (>, <, >=, <=, ==, !=), nilai ambang, dan aksi. Evaluasi dilakukan saat data sensor diterima via MQTT, kemudian aktuator dieksekusi jika kondisi terpenuhi.
		Route	
		<i>GET /dashboard/automation/sensor/create</i>	Form tambah aturan
		<i>POST /dashboard/automation/sensor</i>	Simpan aturan baru
		<i>GET /dashboard/automation/sensor/{uuid}</i>	Detail aturan
		<i>GET /dashboard/automation/sensor/{uuid}/edit</i>	Form edit aturan
		<i>PUT /dashboard/automation/sensor/{uuid}</i>	Simpan perubahan aturan
		<i>DELETE /dashboard/automation/sensor/{uuid}</i>	Hapus aturan
		<i>POST /dashboard/automation/sensor/{uuid}/toggle</i>	<i>Toggle</i> aktif/nonaktif
6	Otomatisasi Data Robot	<i>AutomationRobotController</i> , <i>AutomationRobotService</i> , <i>ActuatorLockManager</i> (Service), <i>AutomationRobot</i> (Model), <i>StoreAutomationRobotRequest</i> , <i>UpdateAutomationRobotRequest</i> , <i>MqttSubscribe</i> (Command), <i>RunRobotIterationsJob</i> (Job), <i>RunRobotActuatorCycle</i> (Job), <i>ProcessRobotAutomationCondition</i> (Job), <i>RobotAutomationTriggered</i> (Event)	Mengelola aturan otomatisasi berbasis data robot dengan konfigurasi definisi data, <i>operator</i> kondisi (termasuk <i>contains</i> , <i>startsWith</i> , <i>endsWith</i> untuk tipe <i>string</i>), nilai, aksi, durasi, <i>interval</i> , dan iterasi. Evaluasi dilakukan saat data robot diterima via MQTT.
		Route	
		<i>GET /dashboard/automation/robot/create</i>	Form tambah otomatisasi robot
		<i>POST /dashboard/automation/robot</i>	Simpan otomatisasi robot baru

No	Fitur	Komponen (Class/File) dan Route	Deskripsi Fungsi
		<i>GET /dashboard/automation/robot/{uuid}</i>	<i>Detail</i> otomatisasi robot
		<i>GET /dashboard/automation/robot/{uuid}/edit</i>	Form ubah otomatisasi robot
		<i>PUT /dashboard/automation/robot/{uuid}</i>	Simpan perubahan otomatisasi robot
		<i>DELETE /dashboard/automation/robot/{uuid}</i>	Hapus otomatisasi robot
		<i>POST /dashboard/automation/robot/{uuid}/toggle</i>	Toggle aktif/nonaktif
7	Notifikasi	<i>NotificationController</i> , <i>PushSubscriptionController</i> , <i>WebPushNotification</i> (Notification), <i>DatabaseNotification</i> (Model), <i>NotificationType</i> (Enum), <i>NewNotificationEvent</i> (Event), <i>DailySummaryService</i> (Service), <i>AutomationNotificationService</i> (Service), <i>SendDailySummary</i> (Command), <i>AutomationExecutionLog</i> (Model)	Mengelola notifikasi sistem termasuk penyimpanan ke basis data, <i>push notification</i> ke perangkat pengguna, dan rangkuman aktivitas harian. Notifikasi otomatis dikirim saat otomatisasi berhasil/gagal dieksekusi.
		Route	
		<i>GET /dashboard/notifications</i>	Daftar notifikasi
		<i>POST /dashboard/notifications/{id}/read</i>	Tandai dibaca
		<i>DELETE /dashboard/notifications/{id}</i>	Hapus notifikasi
		<i>POST /dashboard/notifications/mark-all-read</i>	Tandai semua dibaca
		<i>POST /dashboard/notifications/destroy-all</i>	Hapus semua
		<i>GET /dashboard/notifications/api/notification</i>	API infinite scroll
		<i>POST /dashboard/notifications/push/subscriptions</i>	Daftar <i>push notification</i>
		<i>DELETE /dashboard/notifications/push/subscriptions</i>	Batalkan langganan
		<i>GET /dashboard/notifications/push/vapid-public-key</i>	Ambil <i>VAPID key</i>
8	Log Aktivitas	<i>ActivityLogController</i> , <i>ActivityLog</i> (Model), <i>AutomationExecutionLog</i> (Model), <i>ActivityLogger</i> (Service), <i>DailySummaryService</i> (Service)	Mencatat jejak audit seluruh aktivitas sistem termasuk operasi tambah, ubah, hapus, kontrol aktuator, eksekusi otomatisasi, dan <i>login/logout</i> . Menyediakan filter berdasarkan aksi, entitas, kategori, <i>severity</i> , tanggal, dan pengguna.
		Route	
		<i>GET /dashboard/activity-logs</i>	Daftar log aktivitas lengkap dengan filter
		<i>GET /dashboard/activity-logs/api/insights</i>	
		<i>GET /dashboard/activity-logs/{id}</i>	Detail log aktivitas
		<i>POST /dashboard/activity-logs/cleanup</i>	Hapus log lama
9	Hak Akses	<i>Role</i> (Model), <i>Permission</i> (Model), <i>Permission</i> (Enum), <i>RoleName</i> (Enum), <i>RolePermissionSeeder</i> , <i>PermissionSeeder</i> , <i>PermissionMiddleware</i> (Spatie), <i>RoleMiddleware</i> (Spatie), <i>HasRoles</i> (Trait), <i>HandleInertiaRequests</i> (Middleware)	Implementasi RBAC berbasis <i>Spatie</i> . <i>Permission</i> mengatur akses fitur IoT melalui validasi <i>middleware</i> untuk tiga peran berbeda yaitu Admin, Pengelola, dan Staff

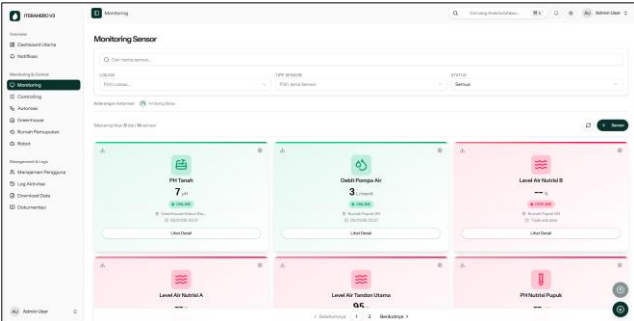
4.1.3 Implementasi Frontend

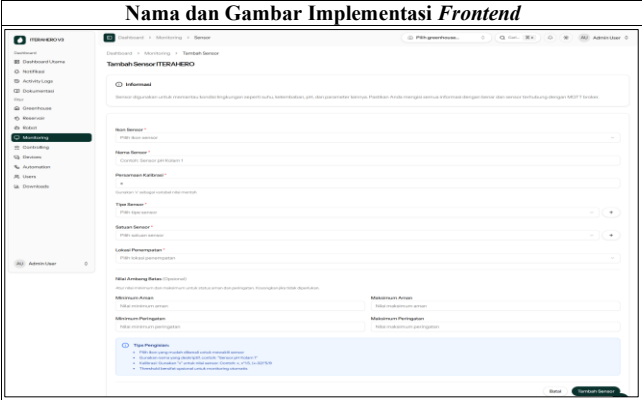
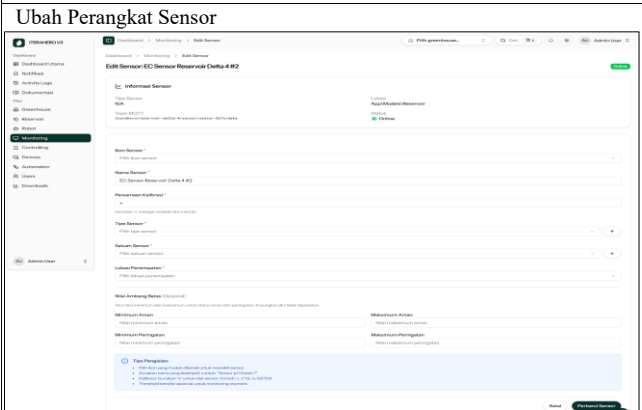
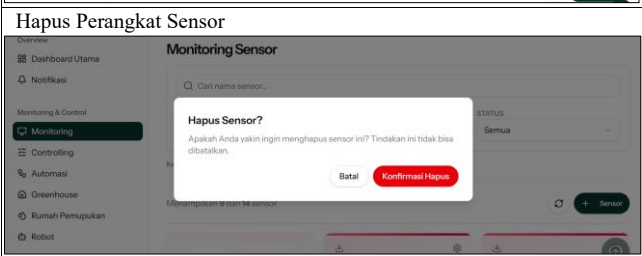
Implementasi *frontend* direalisasikan dengan mengadopsi protokol *Inertia.js* untuk membangun antarmuka yang terintegrasi erat dengan logika *backend* tanpa kompleksitas pengelolaan API terpisah. Dalam arsitektur ini, antarmuka disusun sebagai kumpulan komponen yang menerima

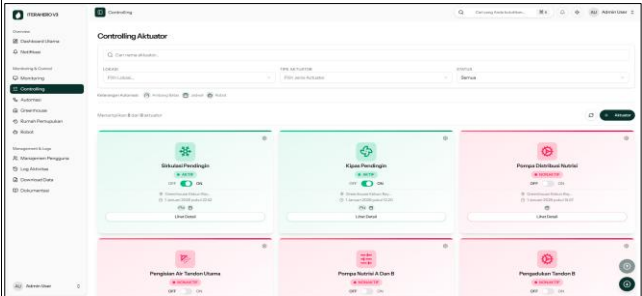
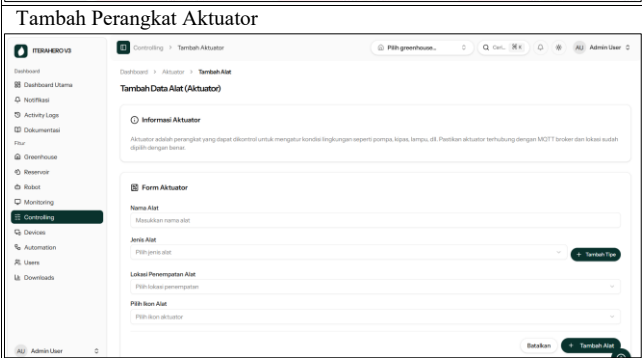
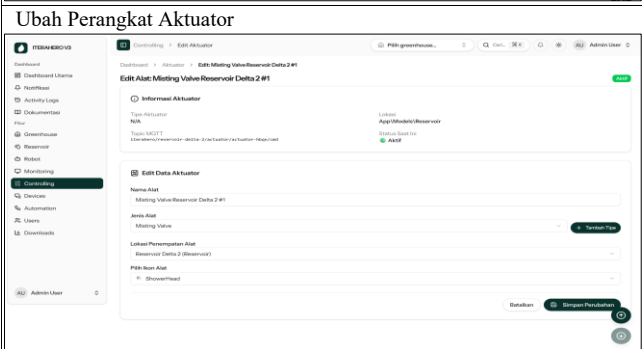
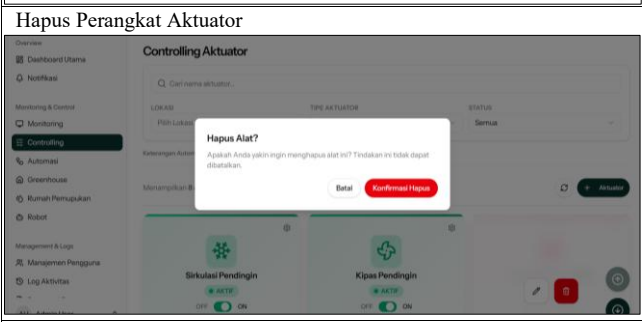
data operasional secara langsung melalui mekanisme injeksi properti (*props*) dari peladen.

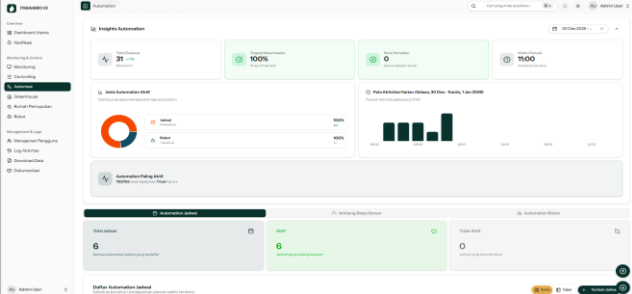
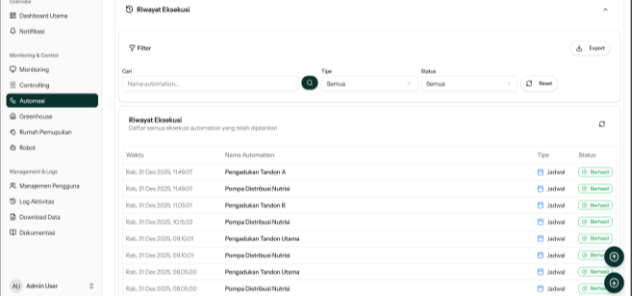
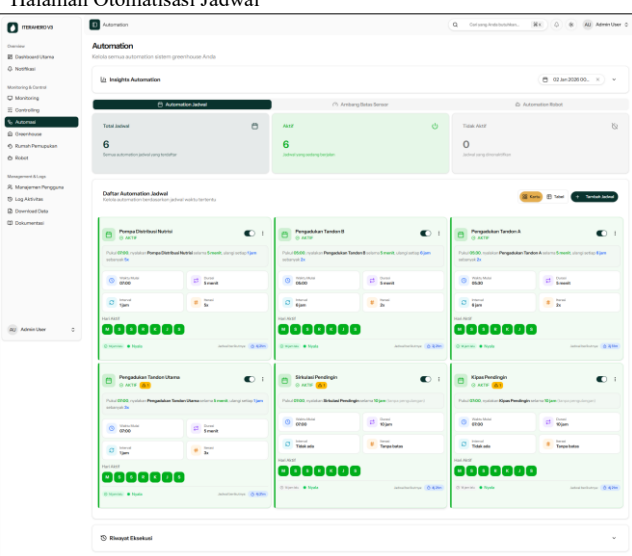
Secara garis besar, implementasi *frontend* mencakup sembilan kelompok fitur utama. Pertama, fitur *login* yang menangani proses otentikasi pengguna berdasarkan peran (Staff, Pengelola, Super Admin). Kedua dan ketiga, fitur manajemen perangkat sensor dan aktuator yang menyediakan operasi Tambah, Edit, dan Hapus lengkap beserta pembaruan nilai secara *real-time* melalui koneksi *WebSocket*. Keempat hingga ketujuh, fitur otomatisasi yang terdiri dari halaman rangkuman, otomatisasi berbasis jadwal, otomatisasi berbasis ambang batas sensor, serta otomatisasi berbasis data rekomendasi robot. Kedelapan, fitur notifikasi yang menampilkan daftar pemberitahuan sistem beserta dukungan *push notification* ke perangkat pengguna. Terakhir, fitur *log* aktivitas yang mencatat jejak audit seluruh tindakan pengguna dan respons sistem untuk keperluan *monitoring* dan evaluasi. Rincian lengkap mengenai komponen fitur yang dihasilkan disajikan pada Tabel 4.2.

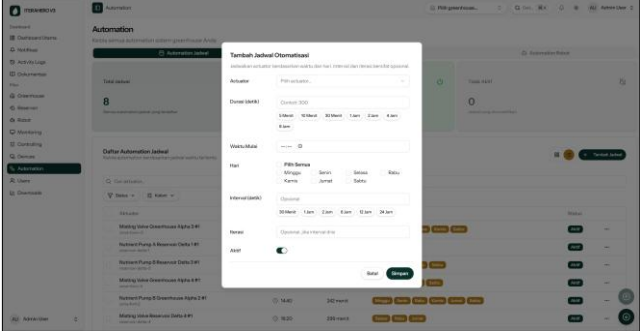
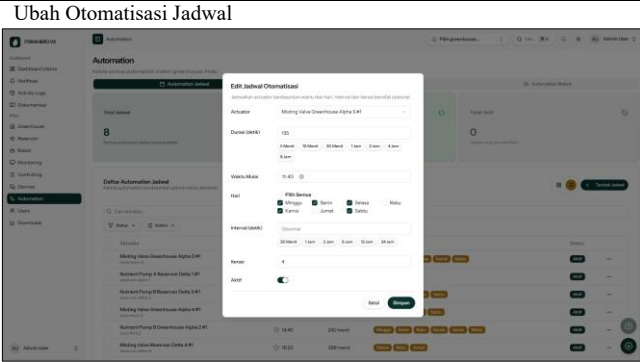
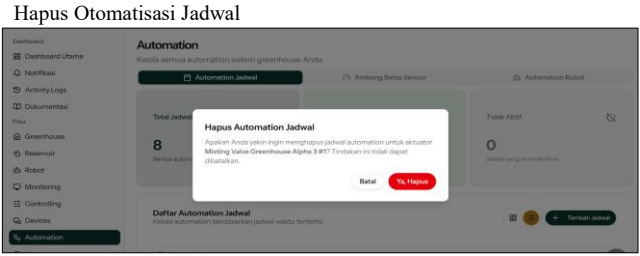
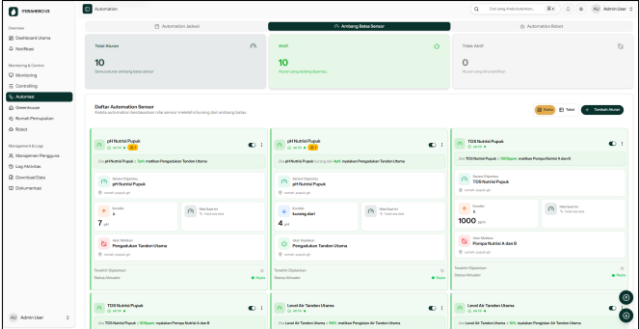
Tabel 4.3 Hasil Implementasi *Frontend*

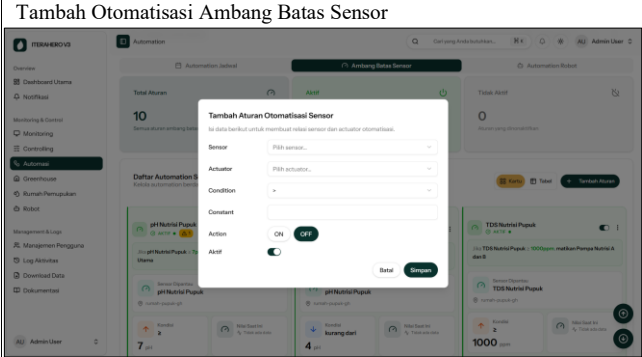
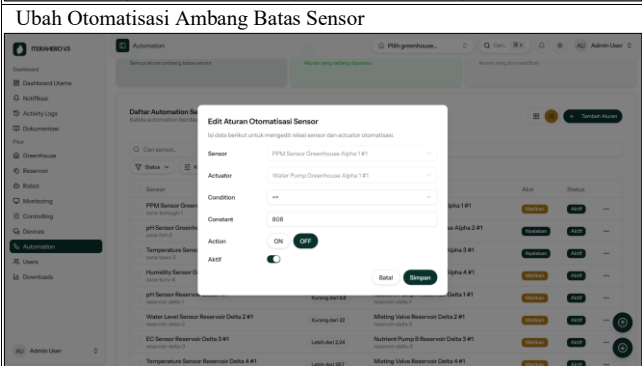
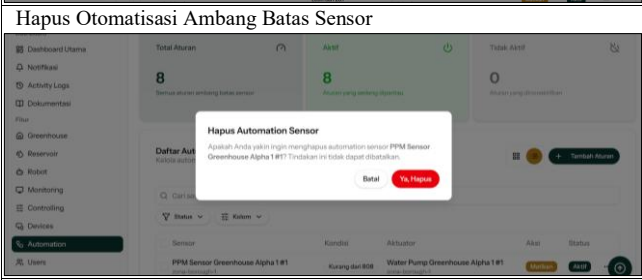
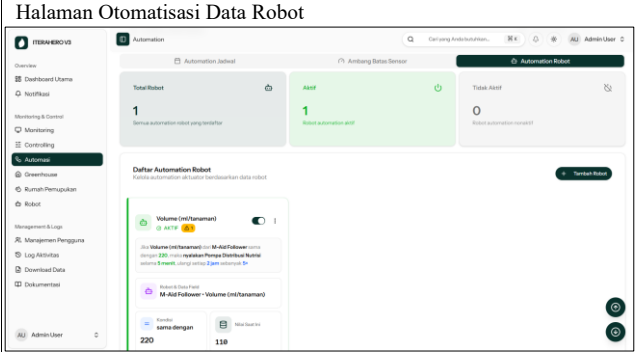
Nama dan Gambar Implementasi <i>Frontend</i>		Deskripsi
Fitur Login		
Halaman Login		Halaman yang digunakan untuk otentikasi guna memverifikasi identitas pengguna (Staff, Pengelola, Super Admin) untuk menjamin hak dan keamanan akses data.
Fitur Perangkat Sensor		
List Halaman Perangkat Sensor		Menampilkan daftar seluruh sensor, <i>filter</i> , paginasi, serta nilai parameter lingkungan terkini yang diperbarui secara <i>real-time</i> .
Tambah Perangkat Sensor		Halaman untuk menambahkan perangkat sensor, dengan mengisi

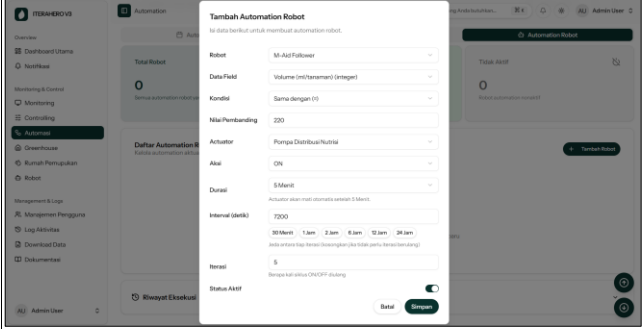
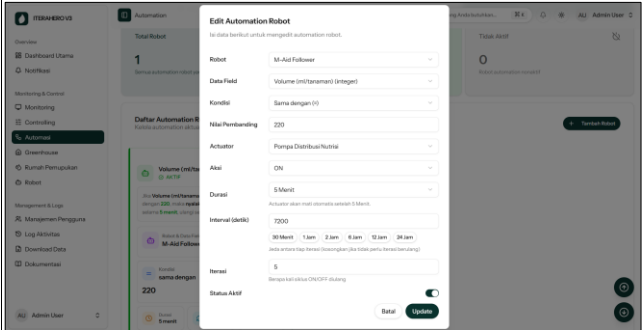
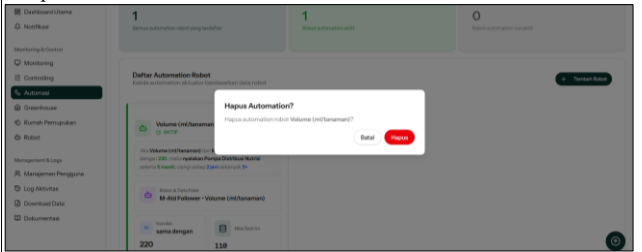
Nama dan Gambar Implementasi <i>Frontend</i> 	Deskripsi icon perangkat, nama sensor, persamaan kalibrasi, tipe sensor, satuan sensor, lokasi, dan juga ambang batas sensor.
Ubah Perangkat Sensor 	Halaman untuk mengedit perangkat sensor, dengan mengisi icon perangkat, nama sensor, persamaan kalibrasi, tipe sensor, satuan sensor, lokasi, dan juga ambang batas sensor yang perlu diubah.
Hapus Perangkat Sensor 	Fitur untuk menghapus perangkat sensor dari data, yang dilengkapi dengan konfirmasi hapus supaya mencegah kasus pengguna salah menekan tombol.
Fitur Perangkat Aktuator	
List Halaman Perangkat Aktuator	Menampilkan daftar seluruh perangkat aktuator, <i>filter</i>, paginasi, serta status perangkat (aktif/nonaktif) yang bisa diperbarui secara <i>real-time</i>.

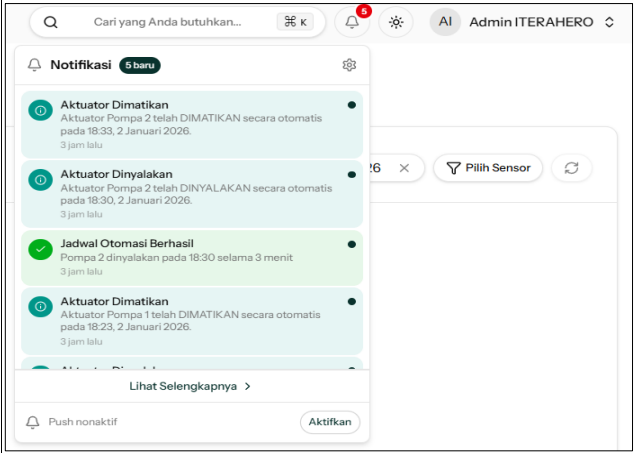
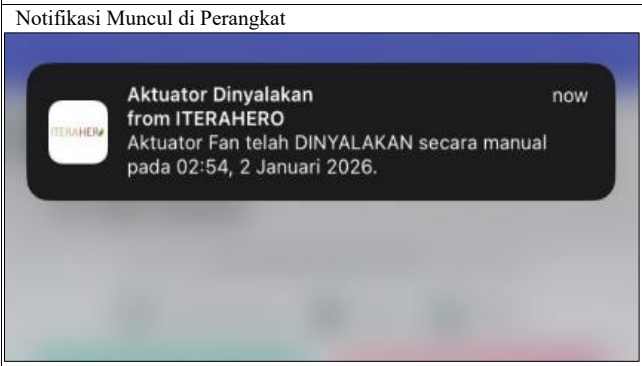
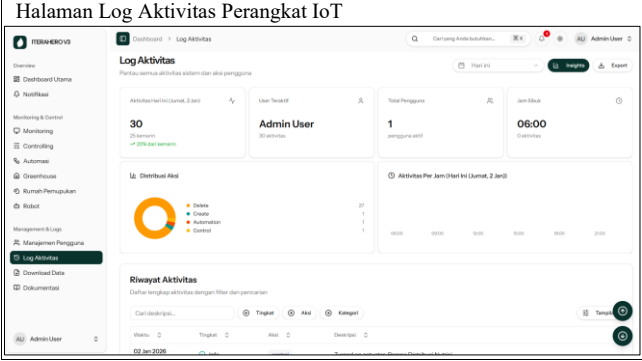
Nama dan Gambar Implementasi <i>Frontend</i>	Deskripsi
	
Tambah Perangkat Aktuator 	Halaman untuk menambahkan perangkat aktuator ke sistem, dengan mengisi nama alat, jenis alat, lokasi, dan icon alat.
Ubah Perangkat Aktuator 	Halaman yang digunakan untuk ubah perangkat aktuator yang sudah ada pada sistem, dengan mengisi nama alat, jenis alat, lokasi, dan icon alat yang perlu diubah.
Hapus Perangkat Aktuator 	Fitur untuk menghapus perangkat aktuator dari data, yang dilengkapi dengan konfirmasi hapus supaya mencegah kasus pengguna salah menekan tombol.
Halaman Otomatisasi	

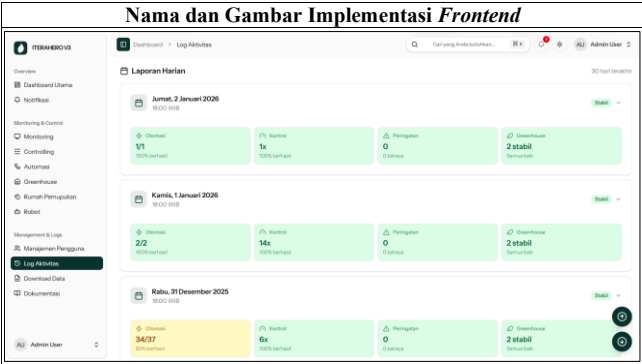
Nama dan Gambar Implementasi <i>Frontend</i> Rangkuman Data Otomatisasi	Deskripsi
	Data rangkuman otomatisasi yang dijalankan oleh sistem, dilengkapi dengan <i>filter</i> tanggal, grafik pola aktivitas perangkat IoT pada otomatisasi.
Riwayat Eksekusi Otomatisasi	
	Riwayat lengkap otomatisasi yang dijalankan oleh sistem mengenai waktu eksekusi perangkat IoT pada otomatisasi berhasil/gagal.
Fitur Otomatisasi Jadwal	
Halaman Otomatisasi Jadwal 	Menampilkan daftar seluruh aturan otomatisasi berbasis penjadwalan harian, dengan jam waktu eksekusinya, <i>filter</i>, paginasi, serta status aturan (aktif/nonaktif). Informasi dibuat dalam cerita supaya pengguna mudah memahami informasi.
Tambah Otomatisasi Jadwal	
	Halaman untuk menambahkan aturan otomatisasi jadwal baru,

Nama dan Gambar Implementasi <i>Frontend</i>	Deskripsi
	dengan memilih aktuator, mengatur durasi, waktu mulai, hari-hari aktif, serta iterasi dan <i>interval</i> pengulangan.
	Halaman untuk mengedit aturan otomatisasi jadwal yang sudah ada, dengan mengubah aktuator, durasi, waktu mulai, hari-hari aktif, atau iterasi dan <i>interval</i> yang perlu diubah.
	Fitur untuk menghapus aturan otomatisasi jadwal dari data, yang dilengkapi dengan konfirmasi hapus supaya mencegah kasus pengguna salah menekan tombol.
Fitur Otomatisasi Ambang Batas Sensor	
	Menampilkan daftar seluruh aturan otomatisasi berbasis ambang batas sensor, dilengkapi <i>filter</i>, paginasi, serta status aturan (aktif/nonaktif).

Nama dan Gambar Implementasi <i>Frontend</i>	Deskripsi
Tambah Otomatisasi Ambang Batas Sensor 	Halaman untuk menambahkan aturan otomatisasi baru, dengan memilih sensor, aktuator, kondisi <i>operator</i>, nilai ambang, dan aksi yang dijalankan.
Ubah Otomatisasi Ambang Batas Sensor 	Halaman untuk mengedit aturan otomatisasi yang sudah ada, dengan mengubah sensor, aktuator, kondisi <i>operator</i>, nilai ambang, atau aksi yang perlu diubah.
Hapus Otomatisasi Ambang Batas Sensor 	Fitur untuk menghapus aturan otomatisasi dari data, yang dilengkapi dengan konfirmasi hapus supaya mencegah kasus pengguna salah menekan tombol.
Fitur Otomatisasi Rekomendasi Robot	
Halaman Otomatisasi Data Robot 	Menampilkan daftar seluruh aturan otomatisasi berbasis data robot, dilengkapi <i>filter</i>, paginasi, serta status aturan (aktif/nonaktif).

Nama dan Gambar Implementasi <i>Frontend</i>	Deskripsi
Tambah Otomatisasi Data Robot 	Halaman untuk menambahkan aturan otomatisasi baru, dengan memilih robot, definisi data, aktuator, kondisi <i>operator</i>, nilai, aksi yang dijalankan, serta mengatur durasi, <i>interval</i>, dan iterasi pengulangan.
Ubah Otomatisasi Data Robot 	Halaman untuk mengedit aturan otomatisasi yang sudah ada, dengan mengubah robot, definisi data, aktuator, kondisi <i>operator</i>, nilai, aksi, durasi, <i>interval</i>, atau iterasi yang perlu diubah.
Hapus Otomatisasi Data Robot 	Fitur untuk menghapus aturan otomatisasi dari data, yang dilengkapi dengan konfirmasi hapus supaya mencegah kasus pengguna salah menekan tombol.
Fitur Notifikasi	
Halaman Notifikasi 	Menampilkan seluruh notifikasi yang diterima pengguna, dilengkapi <i>filter</i> berdasarkan tipe notifikasi dan status baca/belum dibaca.
Dialog Notifikasi 	Jendela <i>pop-up</i> ringkas yang menyajikan detail cepat terkait pesan notifikasi yang

Nama dan Gambar Implementasi <i>Frontend</i>	Deskripsi
	dikirim ke setiap pengguna.
Notifikasi Muncul di Perangkat 	Pesan peringatan instan (<i>push notification</i>) yang muncul di layar perangkat pengguna untuk memberitahu kejadian secara seketika.
Fitur Log Aktivitas Perangkat IoT	
Halaman Log Aktivitas Perangkat IoT 	Rekaman jejak informasi digital yang mencatat seluruh kronologi tindakan pengguna dan respons sistem untuk keperluan monitoring dan evaluasi sistem.

Nama dan Gambar Implementasi <i>Frontend</i>	Deskripsi
	

4.2 Pengujian

Tahap pengujian dilaksanakan secara komprehensif untuk memverifikasi bahwa seluruh komponen subsistem yang telah dibangun mampu beroperasi sesuai dengan spesifikasi kebutuhan fungsional dan non-fungsional yang ditetapkan pada tahap perancangan. Rangkaian evaluasi ini mencakup validasi logika fitur melalui *blackbox testing*, pemeriksaan integritas jalur komunikasi data menggunakan protokol MQTT, serta uji responsivitas antarmuka terhadap berbagai resolusi layar perangkat pengguna. Ringkasan metode dan hasil dari setiap skenario pengujian diuraikan pada sub-bab berikut.

4.2.1 Pengujian *Blackbox*

Pengujian *blackbox testing* difokuskan pada validasi keluaran (*output*) fungsional sistem berdasarkan skenario masukan (*input*) tertentu tanpa meninjau struktur kode internal program. Evaluasi ini mencakup verifikasi terhadap fitur manajemen perangkat, eksekusi logika otomatisasi, serta mekanisme notifikasi untuk memastikan tidak adanya kegagalan fungsi (*bug*) pada operasi utama. Tabel 4.3 menyajikan rekapitulasi tingkat keberhasilan pengujian, sementara rincian lengkap setiap skenario uji dapat dilihat pada Lampiran F.

Tabel 4.4 Rekapitulasi Hasil Pengujian *Blackbox Testing*

No	Kode	Kategori Fitur	Jumlah Skenario	Skenario Berhasil	Skenario Gagal	Persentase Keberhasilan
1	BT-01	Login dan Logout	10	10	0	100%
2	BT-02	Perangkat Sensor	18	18	0	100%
3	BT-03	Perangkat Aktuator	15	15	0	100%
4	BT-04	Halaman Otomatisasi	8	8	0	100%
5	BT-05	Otomatisasi Jadwal	10	10	0	100%
6	BT-06	Otomatisasi Sensor	9	9	0	100%
7	BT-07	Otomatisasi Robot	10	10	0	100%

8	BT-08	Notifikasi	8	8	0	100%
9	BT-09	Log Aktivitas	11	11	0	100%
10	BT-10	Hak Akses	52	52	0	100%
Total		10 Kategori	151	151	0	100%

4.2.2 Pengujian Komunikasi Data dan Otomatisasi

Pengujian komunikasi data dilaksanakan untuk memverifikasi reliabilitas transmisi pesan antara subsistem *server* dan perangkat IoT menggunakan protokol MQTT. Fokus utama pengujian ini adalah validasi pengiriman paket data dengan standar *Quality of Service* (QoS) level 1 (*At least once*), yang menjamin setiap instruksi otomatisasi terkirim secara pasti ke *broker* tanpa risiko kehilangan data (*packet loss*). Pengamatan dilakukan secara *real-time* menggunakan perangkat lunak MQTT Explorer untuk memantau kesesuaian topik (*topic*) dan isi pesan (*payload*) yang diterima oleh perangkat target. Hasil pengujian validasi jalur komunikasi untuk berbagai skenario otomatisasi dirangkum dalam Tabel 4.4.

Tabel 4.5 Rekapitulasi Hasil Pengujian Komunikasi Data dan Otomatisasi Subsistem

No	Kategori Pengujian	Sub-Kategori	Rentang Kode	Jumlah Skenario	Status	Waktu (detik)
1	Komunikasi Data (MQTT)	Sensor (Manual Read)	MQTT-01 s/d MQTT-05	5	Berhasil	<1.00
	Komunikasi Data (MQTT)	Aktuator (Manual Control)	MQTT-06 s/d MQTT-12	7	Berhasil	<1.00
	Komunikasi Data (MQTT)	Data Robot (Simulasi)	MQTT-13 s/d MQTT-16	4	Berhasil	<1.00
2	Otomatisasi Sistem	Logika Ambang Batas (<i>Threshold</i>)	AUTO-01 s/d AUTO-07	7	Berhasil	2.00
	Otomatisasi Sistem	Logika Penjadwalan (<i>Scheduling</i>)	AUTO-08 s/d AUTO-10	3	Berhasil	2.00
	Otomatisasi Sistem	Logika Respon Data Robot	AUTO-11 s/d AUTO-14	4	Berhasil	2.00
Total Skenario Pengujian				30	Berhasil	$\bar{x} = 1.50$

4.2.3 Pengujian Responsif Halaman

Pengujian responsivitas antarmuka bertujuan untuk memastikan tata letak (*layout*) dan elemen visual sistem dapat beradaptasi secara proporsional pada berbagai ukuran *viewport* perangkat pengguna. Validasi dilakukan menggunakan simulasi peramban (*browser*) pada tiga kategori resolusi layar utama, yaitu *mobile* (ponsel), *tablet*, dan *desktop*, guna menjamin aksesibilitas dan kenyamanan penggunaan di lapangan. Hasil pemeriksaan integritas tampilan pada setiap halaman fitur dirangkum dalam Tabel 4.5.

Tabel 4.6 Pengujian Responsif Setiap Halaman

No.	Halaman	mobile $\leq 425px$	768px $\leq$ tablet $< 1024px$	desktop $\geq 1024px$
1.	Halaman Login	✓	✓	✓
2.	Halaman Perangkat Sensor	✓	✓	✓
3.	Halaman Perangkat Aktuator	✓	✓	✓
4.	Halaman Otomatisasi Jadwal	✓	✓	✓
5.	Halaman Otomatisasi berdasarkan Ambang Batas Sensor	✓	✓	✓
6.	Halaman Otomatisasi Rekomendasi Robot	✓	✓	✓
7.	Halaman Notifikasi	✓	✓	✓

Catatan: ✓ = *responsive*, X = tidak *responsive*

4.3 Deploy

Tahapan *deployment* merupakan proses pemindahan sistem dari lingkungan pengembangan lokal menuju peladen (*server*) produksi untuk memastikan aplikasi dapat diakses secara publik melalui jaringan internet. Konfigurasi infrastruktur dan instalasi dependensi perangkat lunak dilakukan agar seluruh fitur otomatisasi dan notifikasi *real-time* dapat berjalan stabil saat digunakan oleh pengguna akhir. Proses *deployment* dilakukan menggunakan teknologi kontainerisasi Docker untuk menjamin konsistensi lingkungan antara tahap pengembangan dan produksi. Aplikasi dijalankan menggunakan Laravel Octane dengan server aplikasi FrankenPHP yang terintegrasi dengan Caddy Web Server, memberikan performa tinggi untuk menangani koneksi *concurrent* yang dibutuhkan oleh fitur *WebSockets*. Rincian spesifikasi lingkungan produksi yang digunakan serta status ketersediaan akses sistem disajikan dalam Tabel 4.6.

Tabel 4.7 Spesifikasi Lingkungan Produksi dan Status Akses

Komponen	Spesifikasi/Keterangan	Status
Server OS	Ubuntu 22.04 LTS (Linux)	Terkonfigurasi✓
Web Server	Caddy (via FrankenPHP)	Aktif (HTTPS/HTTP3)✓
Database	PostgreSQL 17	Terhubung✓
Cache & Queue	Redis (Alpine)	Berjalan✓
MQTT Broker	Eclipse Mosquitto 2.0.21	Aktif (PORT 1883) ✓
Runtime	PHP 8.4 (Laravel Octane)	Optimal✓
Domain Akses	www.iterahero.com	Dapat Diakses✓

4.4 Review

Tahap tinjauan (*review*) dilaksanakan melibatkan penanggung jawab Greenhouse Kebun Raya ITERA, Ibu Zunanik Mufidah, S.TP., M.Si., guna memvalidasi kesesuaian fungsi sistem dengan kebutuhan operasional yang telah disepakati pada tahap inisiasi. Proses ini bertujuan untuk memastikan

bahwa skenario otomatisasi, logika penjadwalan, dan mekanisme peringatan dini telah berjalan akurat sebelum sistem diserahkan kepada pengguna yang lebih luas. Validasi dilakukan dengan mendemonstrasikan skenario penggunaan langsung di lapangan, mulai dari pengaturan jadwal penyiraman hingga simulasi kondisi kritis untuk memicu notifikasi. Hasil validasi dan catatan perbaikan dari tahap ini dirangkum dalam Tabel 4.7.

Tabel 4.8 Tahapan Review Hasil Implementasi

No	Fitur Skenario	Hasil Validasi	Catatan/Tindak Lanjut
1.	Login/Logout	Sesuai/Tidak Sesuai	
2.	Perangkat Sensor	Sesuai/Tidak Sesuai	
3.	Perangkat Aktuator	Sesuai/Tidak Sesuai	
4.	Otomatisasi Berdasarkan Penjadwalan	Sesuai/Tidak Sesuai	
5.	Otomatisasi Berdasarkan Ambang Batas Sensor	Sesuai/Tidak Sesuai	
6.	Otomatisasi Data Robot	Sesuai/Tidak Sesuai	
7.	Notifikasi	Sesuai/Tidak Sesuai	
8.	Log Aktivitas	Sesuai/Tidak Sesuai	

4.5 Pengujian *Usability*

Pengujian *usability* dilakukan untuk mengukur kualitas interaksi pengguna terhadap subsistem dengan melibatkan 32 responden yang terdiri dari staf, pengelola, dan administrator sistem. Evaluasi ini menerapkan kuesioner *System Usability Scale* (SUS) untuk mendapatkan skor kelayakan sistem secara kuantitatif. Rekapitulasi penilaian subjektif responden terhadap sepuluh indikator SUS disajikan secara rinci pada Tabel 4.8.

Tabel 4.9 Hasil Pengujian SUS

R	Pertanyaan										Jumlah	Nilai SUS
	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10		
R1	4	2	3	1	4	4	3	4	3	1	29	72.5
R2	4	1	3	4	3	3	3	4	4	1	30	75
R3	2	3	4	2	4	3	3	3	4	3	31	77.5
R4	4	3	3	3	4	3	4	3	4	3	34	85
R5	4	1	3	4	3	2	4	3	3	1	28	70
R6	4	4	4	4	4	4	4	4	4	4	40	100
R7	4	4	4	4	4	4	4	4	4	4	40	100
R8	4	3	4	4	4	3	4	4	4	3	37	92.5
R9	4	3	4	2	4	3	4	4	4	3	35	87.5
R10	4	2	4	0	4	2	4	4	4	2	30	75
R11	4	2	4	3	3	3	4	4	4	1	32	80
R12	3	3	4	2	3	2	3	4	3	0	27	67.5
R13	1	2	2	4	0	2	1	1	1	3	17	42.5
R14	3	3	3	1	3	3	2	3	3	3	27	67.5
R15	4	0	4	4	4	0	4	4	4	0	28	70
R16	3	3	3	3	4	3	2	2	3	3	29	72.5

R	Pertanyaan										Jumlah	Nilai SUS
	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10		
R17												
R18												
R19												
R20												
R21												
R22												
R23												
R24												
R25												
R26												
R27												
R28												
R29												
R30												
R31												
R32												
$\bar{x}$	4	2	4	3	3	3	3	3	4	2	Skor SUS	77.1875

Keterangan: R=Responden

4.6 Analisis

Hasil evaluasi terhadap seluruh pengujian menunjukkan bahwa subsistem otomatisasi beroperasi sesuai dengan spesifikasi fungsional dan kebutuhan pengguna yang telah ditetapkan. Hal ini dibuktikan melalui capaian tingkat keberhasilan 100% pada pengujian *blackbox*, yang memvalidasi bahwa logika otomatisasi berjalan dengan tepat tanpa kendala fungsional. Kinerja sistem dari sisi komunikasi data juga terkonfirmasi melalui penerapan protokol MQTT dengan *Quality of Service* (QoS) level 1, yang memastikan integritas pengiriman instruksi kendali tanpa kehilangan paket data (*packet loss*) sekaligus menggantikan mekanisme *polling* sebelumnya. Selain aspek teknis, pengukuran tingkat penerimaan pengguna menggunakan metode *System Usability Scale* (SUS) menghasilkan skor rata-rata 90,5 (*Grade A, Excellent*), mengindikasikan bahwa antarmuka sistem dapat digunakan secara efektif oleh para staf dan pengelola operasional di lapangan.

Hasil evaluasi *System Usability Scale* (SUS) menunjukkan skor rata-rata 77,19 yang menempatkan sistem pada kategori *acceptable* dengan *grade B*, didorong oleh tingginya preferensi penggunaan kembali (Q1) dan kemudahan pengoperasian (Q3) serta kepercayaan diri pengguna (Q9) yang dominan mencapai skor 4; namun, terdapat kesenjangan pada persepsi kompleksitas (Q2) dan kebutuhan pembelajaran awal (Q10) yang hanya mencapai skor 2, menyiratkan bahwa meskipun sistem dinilai konsisten (Q6) dan minim kebutuhan *technical support* (Q4), pengguna baru tetap menghadapi hambatan *learnability* akibat fitur yang dirasa rumit, sehingga

penyederhanaan antarmuka menjadi krusial untuk meningkatkan *usability* secara menyeluruh.

BAB V

KESIMPULAN

5.1 Kesimpulan

Berdasarkan seluruh tahapan penelitian yang telah dilaksanakan, mulai dari analisis kebutuhan hingga pengujian akhir terhadap subsistem otomatisasi, dapat ditarik kesimpulan sebagai berikut:

1. Implementasi metode *Agile Kanban* pada pengembangan subsistem otomatisasi berhasil menghasilkan arsitektur *Event-Driven* yang responsif, di mana integrasi protokol MQTT dan teknologi *WebSocket* terbukti mampu menggantikan mekanisme *polling* konvensional untuk manajemen jadwal, ambang batas sensor, dan rekomendasi robot secara *real-time*.
2. Evaluasi teknis melalui *Blackbox Testing* memvalidasi tingkat keberhasilan fungsional sebesar 100%, sedangkan pengujian usability menghasilkan skor *System Usability Scale* (SUS) 82,5 yang diklasifikasikan sebagai *Grade A* dengan predikat *Excellent*, menunjukkan tingkat penerimaan dan kepuasan pengguna yang sangat tinggi terhadap sistem.

5.2 Saran

Optimalisasi dan pengembangan keberlanjutan subsistem untuk mendukung operasional *greenhouse* di masa mendatang memerlukan implementasi dari beberapa rekomendasi strategis berikut:

1. Transformasi platform menuju aplikasi seluler (*mobile apps*) secara *native* sangat direkomendasikan untuk menjamin reliabilitas fitur notifikasi *push* serta memberikan fleksibilitas akses yang lebih ergonomis bagi petugas lapangan.
2. Peningkatan presisi algoritma kecerdasan buatan pada sisi robot diperlukan agar data rekomendasi otomatisasi yang dikirimkan ke subsistem mampu mengakomodasi analisis fase tumbuh dan kondisi fisiologis tanaman secara lebih spesifik.
3. Eskalasi infrastruktur perangkat keras disarankan melalui integrasi variasi sensor dan aktuator yang lebih kompleks untuk melengkapi mekanisme otomatisasi yang telah terbangun agar adaptif terhadap berbagai parameter budidaya.
4. Penerapan sistem orkestrasi alur kerja (*workflow orchestration engine*) seperti n8n direkomendasikan untuk memfasilitasi skenario otomatisasi bertingkat yang kompleks serta terintegrasi dengan layanan *Generative AI*, sehingga mekanisme kendali perangkat dapat dieksekusi secara sekuensial dan adaptif berdasarkan analisis data cerdas.

DAFTAR PUSTAKA

- [1] Presiden Republik Indonesia, “Peraturan Presiden (PERPRES) Nomor 83 Tahun 2023 tentang Penyelenggaraan Kebun Raya,” 2023. [Online]. Available: <https://peraturan.bpk.go.id/Details/274235/perpres-no-83-tahun-2023>
- [2] Pemerintah Pusat, “Undang-undang (UU) Nomor 11 Tahun 2019 tentang Sistem Nasional Ilmu Pengetahuan dan Teknologi,” 2019. [Online]. Available: <https://peraturan.bpk.go.id/Details/117023/uu-no-11-tahun-2019>
- [3] Institut Teknologi Sumatera, “Rencana Strategis (Renstra) Institut Teknologi Sumatera 2025-2029,” 2025, *Lampung Selatan*.
- [4] Rudiysanyah, “Golden Melon Tumbuh Subur di Smart Farming System ITERA,” Jan. 2023. [Online]. Available: <https://www.itera.ac.id/golden-melon-tumbuh-subur-di-smart-farming-system-itera/>
- [5] D. Y. Setyawan, W. Warsito, R. Marjunus, N. Nurfiana, and R. Syahputri, “A Systematic Literature Review: Internet of Things on Smart Greenhouse,” *Int J Adv Comput Sci Appl*, vol. 13, no. 12, p. 672, 2022, [Online]. Available: www.ijacsa.thesai.org
- [6] F. Abou-Mehdi-Hassani, A. Zaguia, H. Ait Bouh, and A. Mkhida, “Systematic literature review of smart greenhouse monitoring,” *SN Comput Sci*, vol. 6, no. 1, p. 95, 2025, doi: 10.1007/s42979-024-03640-4.
- [7] V. Jagatha, A. Khamesipour, and S. Chung, “Cross-Platform App Development: A Comparative Study of PWAs and React Native Mobile Apps,” in *2023 Proceedings of the ISCAP Conference*, Albuquerque, NM: ISCAP (Information Systems and Computing Academic Professionals), 2023. [Online]. Available: <https://iscap.us/proceedings/>
- [8] R. A. Nugraha, “Sistem Monitoring Smart Greenhouse Berbasis IoT Dengan Menggunakan Metode Agile Kanban (Studi Kasus Greenhouse Melon ITERA),” Institut Teknologi Sumatera, Lampung Selatan, 2023.
- [9] R. Appelqvist and O. Örmmyr, “Performance comparison of XHR polling, Long polling, Server sent events and Websockets,” Blekinge Institute of Technology, Faculty of Computing, Karlskrona, Sweden, 2017.
- [10] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O’Reilly Media, 2017.
- [11] Suhandi, S. Purnama, R. Ahsanitaqwm, and S. Nurm, “The Role of Automation and IoT in Enhancing Operational Efficiency: Evidence from PLS-SEM Analysis,” *APTISI Transactions on Management (ATM)*, vol. 9, no. 1, pp. 72–81, 2025, doi: 10.33050.
- [12] S. A. Khajeh, M. Saberikamarposhti, and A. M. Rahmani, “Real-Time Scheduling in IoT Applications: A Systematic Review,” *Sensors*, vol. 23, no. 1, p. 232, 2023, doi: 10.3390/s23010232.
- [13] M. I. Hossain, “Software Development Life Cycle (SDLC) Methodologies for Information Systems Project Management,” *International Journal for Multidisciplinary Research (IJFMR)*, vol. 5, no. 5, 2023, doi: 10.36948/ijfmr.2023.v05i05.6228.
- [14] B. D. Magar, “Comparative Study of Agile Methodologies: Scrum, Kanban, and Extreme Programming (XP),” Vaasan Ammattikorkeakoulu University of Applied Sciences, 2025.
- [15] E. Brechner, *Agile Project Management with Kanban*. Redmond, Washington: Microsoft Press, 2015.

- [16] Tim Penyusun, “Pedoman Tugas Akhir Capstone Program Studi Teknik Informatika,” 2024, *Lampung Selatan*.
- [17] M. S. Tuloli, R. Patalangi, and R. Takdir, “Pengukuran Tingkat Usability Sistem Aplikasi e-Rapor Menggunakan Metode Usability Testing dan SUS,” *Jambura Journal of Informatics*, vol. 4, no. 1, pp. 14–26, 2022, doi: 10.37905/jji.v4i1.13411.
- [18] “ISO 9241-11:2018 Ergonomics of human-system interaction – Part 11: Usability: Definitions and concepts,” 2018, *Geneva, Switzerland*.
- [19] M. A. Memon, R. Thurasamy, H. Ting, and J.-H. Cheah, “Purposive Sampling: A Review and Guidelines for Quantitative Research,” *Journal of Applied Structural Equation Modeling*, vol. 9, no. 1, pp. 1–23, 2025, doi: 10.47263/JASEM.9(1)01.
- [20] I. Etikan, S. A. Musa, and R. S. Alkassim, “Comparison of Convenience Sampling and Purposive Sampling,” *American Journal of Theoretical and Applied Statistics*, vol. 5, no. 1, pp. 1–4, 2016, doi: 10.11648/j.ajtas.20160501.11.
- [21] A. Morchid, R. Jebabra, H. Qjidaa, R. El Alami, and M. O. Jamil, “Agri-tech innovations for sustainability: A fire detection system based on MQTT broker and IoT to improve environmental risk management,” *Results in Engineering*, vol. 24, Dec. 2024, doi: 10.1016/j.rineng.2024.103683.
- [22] B. Hardika *et al.*, “Pengujian Blackbox Testing Website Garuda Farm Menggunakan Teknik Equivalence Partitioning.”
- [23] S. Sari and O. C. R. Rachmawati, “The Implementation of Agile Kanban in the Development of an IoT-Based Sugarcane Growth Monitoring System,” *G-Tech: Jurnal Teknologi Terapan*, vol. 9, no. 4, pp. 1858–1867, 2025, doi: 10.70609/g-tech.v9i4.7845.
- [24] H. Y. Riskiawan *et al.*, “Artificial Intelligence Enabled Smart Monitoring and Controlling of IoT-Green House,” *Arab J Sci Eng*, 2023, doi: 10.1007/s13369-023-07887-6.
- [25] Maryamah, L. Hasibuan, K. Anwar, and A. F. R. Ahmad, “Sistem Informasi Pendidikan,” *Milenial: Journal for Teachers and Learning*, vol. 2, no. 1, pp. 1–11, 2021, [Online]. Available: <https://ejournal.anotero.org/index.php/milenial>
- [26] A. Hidayat and A. Buana, “SISTEM INFORMASI PERPUSTAKAAN DIGITAL BERBASIS WEB MENGGUNAKAN FRAMEWORK SLIM CENDANA,” *Jurnal Manajemen Informatika (JUMIKA)*, vol. 5, no. 1, 2018, [Online]. Available: <http://jurnal.stmik-dci.ac.id/index.php/jumika/>
- [27] C. Chastro and E. Darmawan H., “Perbandingan Pengembangan Front End Menggunakan Blade Template dan Vue Js,” *Jurnal Strategi*, vol. 2, no. 2, Nov. 2020.
- [28] D. N. Rizkiani, A. Sumadyo, and A. Marlina, “Greenhouse Sebagai Wadah Penelitian Hortikultura Pada Balai Penelitian Dan Pengembangan Tanaman Pangan Di Pematang,” *SENTHONG: Jurnal Ilmiah Mahasiswa Arsitektur*, vol. 3, no. 2, pp. 461–470, Jul. 2020, [Online]. Available: <https://jurnal.ft.uns.ac.id/index.php/senthong/index>
- [29] S. Megawati and A. Lawi, “Pengembangan Sistem Teknologi Internet of Things Yang Perlu Dikembangkan Negara Indonesia,” *JEET: Journal Information Engineering and Educational Technology*, vol. 5, no. 1, 2021.
- [30] Y. Efendi, “INTERNET OF THINGS (IOT) SISTEM PENGENDALIAN LAMPU MENGGUNAKAN RASPBERRY PI BERBASIS MOBILE,” *Jurnal Ilmiah Ilmu Komputer*, vol. 4, no. 1, 2018.

- [31] S. N. Malini and E. Gusmira M.Si, "LITERATUR REVIEW: PENGGUNAAN INTERNET OF THINGS (IOT) DALAM PEMANTAUAN SUHU DAN KELEMBAPAN MENGGUNAKAN SENSOR DHT11," *JSSIT: Jurnal Sains dan Sains Terapan*, vol. 2, no. 2, Nov. 2024.
- [32] F. P. E. Putra, F. Muslim, N. Hasanah, Holipah, R. Paradina, and R. Alim, "Analisis Komparasi Protokol Websocket dan MQTT Dalam Proses Push Notification," *Jurnal Sistim Informasi dan Teknologi*, vol. 5, no. 4, pp. 63–72, Jan. 2023, doi: 10.60083/jsisfotek.v5i4.325.
- [33] M. Diono, A. D. Putri, H. Azwar, and W. Khabzli, "Sistem Monitoring Jaringan Sensor Node Berbasis Protokol MQTT," *Jurnal ELEMENTER*, vol. 7, no. 2, pp. 120–126, Nov. 2021, [Online]. Available: <https://jurnal.pcr.ac.id/index.php/elementer>
- [34] A. Rahman and A. N. Salim, "Sistem Kendali pH dan Kekeruhan Air Aquascape Menggunakan Wemos D1 Mini ESP8266 Berbasis IoT," *Jurnal Teknologi Terpadu*, vol. 8, no. 1, pp. 22–30, 2022.
- [35] G. Yudha Saputra, A. Denhas Afrizal, F. Khusnu Reza Mahfud, F. Angga Pribadi, and F. Jati Pamungkas, "PENERAPAN PROTOKOL MQTT PADA TEKNOLOGI WAN (STUDI KASUS SISTEM PARKIR UNIVERISTAS BRAWIJAYA)," 2017.
- [36] P. Sethi and S. R. Sarangi, "Internet of Things: Architectures, protocols, and applications," *Journal of Electrical and Computer Engineering*, vol. 2017, 2017, doi: 10.1155/2017/9324035.
- [37] M. Javaid, A. Haleem, R. P. Singh, and S. Rab, "Significance of sensors for Industry 4.0: Roles, capabilities, and applications," *Sensors International*, vol. 2, p. 100110, 2021, doi: 10.1016/j.sintl.2021.100110.
- [38] J. Katare, "Agile Marketing as a Key Driver to Increasing Operational Efficiencies and Speed to Market," *International Journal of Business Administration*, vol. 13, no. 2, 2022.
- [39] GeeksforGeeks, "Kanban Agile Methodology," 2024. [Online]. Available: <https://www.geeksforgeeks.org/software-engineering/kanban-agile-methodology/>
- [40] S. AKHBARONA, "PENGEMBANGAN APLIKASI INVENTORY MANAGEMENT BERBASIS WEBSITE DENGAN BARCODE MENGGUNAKAN MODEL PERSONAL EXTREME PROGRAMMING (STUDI KASUS: KEBUN RAYA ITERA)," Institut Teknologi Sumatera (ITERA), Lampung, 2023. [Online]. Available: <https://repo.itera.ac.id/depan/submission/SB2505280052>
- [41] S. J. Putri, D. G. P. Putri, and W. H. N. Putra, "Analisis Komparasi pada Teknik Black Box Testing (Studi Kasus: Website Lars)," *Journal of Internet and Software Engineering (JISE)*, vol. 5, no. 1, p. 23, May 2024.
- [42] J. Brooke, "SUS: A Retrospective," *J Usability Stud*, vol. 8, no. 2, pp. 29–40, Feb. 2013.
- [43] J. Sauro, "5 Ways to Interpret a SUS Score," 2018. [Online]. Available: <https://measuringu.com/interpret-sus-score/>
- [44] A. Aurellia, S. Nooriansyah, and Y. Amrozi, "PEMANFAATAN UML DALAM PERANCANGAN SISTEM INFORMASI PRODUK KREATIF DAUR ULANG SAMPAH BERBASIS WEB," *JITET (Jurnal Informatika dan Teknik Elektro Terapan)*, vol. 13, no. 3S1, pp. 1739–1748, 2025, doi: 10.23960/jitet.v13i3S1.8073.

- [45] dheaprianiblog, “Entity Relationship Model,” Nov. 2015. [Online]. Available: <https://dheaprianiblog.wordpress.com/2015/11/04/entity-relationship-model/>
- [46] R. Appelqvist and O. Örnmyr, “Performance comparison of XHR polling, Long polling, Server sent events and Websockets,” Blekinge Institute of Technology, Faculty of Computing, Karlskrona, Sweden, 2017.
- [47] Laravel, “Laravel Introduction,” 2025. [Online]. Available: <https://laravel.com/docs/12.x/installation>
- [48] Laravel, “Laravel Reverb,” 2025. [Online]. Available: <https://laravel.com/docs/12.x/reverb>
- [49] Laravel, “Laravel Horizon,” 2025. [Online]. Available: <https://laravel.com/docs/12.x/horizon>
- [50] Laravel, “Laravel Notifications,” 2025. [Online]. Available: <https://laravel.com/docs/12.x/notifications>
- [51] A. I. Sanka, M. H. Chowdhury, and R. C. C. Cheung, “Efficient High-Performance FPGA-Redis Hybrid NoSQL Caching System for Blockchain Scalability,” *Comput Commun*, vol. 169, pp. 81–91, 2021, doi: 10.1016/j.comcom.2021.01.017.
- [52] B. Michelson, “Event-Driven Architecture Overview,” Boston, Massachusetts, Feb. 2006. doi: 10.1571/bda2-2-06cc.

LAMPIRAN

A. Surat Tugas Ibu Zunanik Mufidah, S.TP., M.Si. Sebagai Ketua Tim Penelitian *Greenhouse* Kebun Raya ITERA



KEMENTERIAN PENDIDIKAN TINGGI, SAINS,
DAN TEKNOLOGI
INSTITUT TEKNOLOGI SUMATERA
LEMBAGA PENELITIAN DAN PENGABDIAN KEPADA MASYARAKAT
Jalan Terusan Ryacudu Way Hui, Kecamatan Jati Agung, Lampung Selatan 35365
Telepon: (0721) 8030188
Email: ippm@itera.ac.id, Website : <http://ippm.itera.ac.id>

SURAT TUGAS

Nomor: 4809/IT9.2.1/PT.01.05/2025

Kepala Lembaga Penelitian dan Pengabdian kepada Masyarakat Institut Teknologi Sumatera dengan ini memberi tugas kepada:

No	Nama	NIDN/NIM	Program Studi	Keterangan
1.	Zunanik Mufidah, M.Si.	0015059206	Teknik Biosistem	Ketua
2.	Kisna Pertiwi, S.Si, M.Si.	0001059402	Rekayasa Instrumentasi dan Automasi	Anggota
3.	Winda Yulita, M.Cs.	0027079304	Teknik Informatika	Anggota
4.	Gerry Romora Tarigan	122310072	Teknik Biosistem	Mahasiswa
5.	Istifa Zuriyanti	122310002	Teknik Biosistem	Mahasiswa
6.	Amas Muda Luckyto	122310055	Teknik Biosistem	Mahasiswa
7.	Iqbal Darwis	122310019	Teknik Biosistem	Mahasiswa
8.	Cornelius Linux	122140079	Teknik Informatika	Mahasiswa
9.	Muhammad Yusuf	122140193	Teknik Informatika	Mahasiswa
10.	Ade Muharom	122490029	Rekayasa Instrumentasi dan Automasi	Mahasiswa
11.	Muslimin	122490019	Rekayasa Instrumentasi dan Automasi	Mahasiswa

Untuk melaksanakan kegiatan "Penelitian" skema Hilirisasi Riset-Pengujian Model Dan Prototype Dana BOPTN Direktorat Penelitian dan Pengabdian kepada Masyarakat Tahun Anggaran 2025 yang berjudul "Transformasi Digital Pemupukan: Implementasi IoT dalam Sistem Peracik Pupuk Otomatis di Lahan Pertanian", pada tanggal 23 Juli s.d. 9 Desember 2025 di Institut Teknologi Sumatera dan Lampung Timur.

Demikian surat tugas ini dibuat untuk dilaksanakan dengan penuh tanggung jawab.

7 November 2025

Kepala


Muhamad Fatikhul Arif
NIP. 198111152020121002

Tembusan:

1. Wakil Rektor Bidang Akademik dan Kemahasiswaan;
2. Dekan Fakultas Teknologi Industri;
3. Koordinator Prodi Teknik Biosistem;
4. Koordinator Prodi Rekayasa Instrumentasi dan Automasi;
5. Koordinator Prodi Teknik Informatika;
6. Arsip.

B. Format Instrumen dan Hasil Wawancara

1.	Bagaimana kendala yang Ibu alami selama mengoperasikan sistem monitoring dan pengendalian greenhouse dalam dua tahun terakhir? Jawaban Ibu Zuna: Iya dek, sebenarnya sistem otomatisasi yang lama itu sudah cukup membantu kami, cuma di lapangan sering terasa agak lambat dan kadang tidak langsung bereaksi begitu kondisi di greenhouse berubah. Bagian-bagian seperti pengaturan suhu, penyiraman, dan pemupukan itu juga masih berjalan sendiri-sendiri, jadi kami harus cek satu-satu, dan itu bikin pekerjaan kurang efisien. Notifikasinya pun sering telat datang atau infonya tidak lengkap, jadinya kami tidak cepat tahu apakah proses otomatisasinya benar-benar berhasil atau malah gagal. Untuk tampilan juga kami berharap bisa dibuat lebih modern dan konsisten, biar data pentingnya langsung terlihat dan lebih mudah dipahami staf di lapangan. Selain itu dek, ke depannya kami ada rencana pakai robot keliling untuk cek kondisi tanaman, jadi akan sangat membantu kalau subsistem yang baru ini sudah disiapkan supaya bisa tersambung dengan robot itu dan bisa pakai rekomendasinya untuk mengatur perangkat di greenhouse/rumah pemupukan.
2.	Dalam kondisi apa perangkat yang ada di greenhouse seharusnya menyala atau mati secara otomatis? Misalnya ketika suhu terlalu tinggi atau air habis. Jawaban Ibu Zuna: Kalau suhu sudah melewati batas tertentu, misalnya lebih dari 31 derajat, kipas dan cooling pad itu harus menyala sendiri. Kalau suhu sudah kembali normal, ya harus mati sendiri juga. Begitu juga dengan kelembapan, kalau rendah harus otomatis dinaikkan.
3.	Apakah ada kegiatan rutin seperti penyiraman, pengadukan nutrisi, atau pendinginan yang sebaiknya berjalan otomatis berdasarkan waktu tertentu? Kapan saja jadwal yang Ibu perlukan? Jawaban Ibu Zuna: Beberapa alat itu perlu dijalankan berdasarkan jadwal. Misalnya pompa untuk penyiraman pagi dan sore. Atau pengadukan tandon setiap hari jam tertentu. Saya ingin pengaturannya mudah. Saya tidak mau harus minta programmer setiap kali ada perubahan jadwal.

4.	Apakah sistem yang sebelumnya sudah mendukung otomatisasi jadwal atau hanya monitoring? Apa kelemahan yang paling terasa? Jawaban Ibu Zuna: Kadang jadwal penyiraman telat atau tidak jalan, terutama kalau server lagi berat. Jadi sistemnya tidak bisa diandalkan kalau waktunya pas-pasan. Ini sangat merugikan karena tanaman butuh konsistensi.
5.	Saya dengar greenhouse berencana menggunakan robot pemantau tanaman. Informasi apa saja yang perlu dikirim robot ke sistem agar bisa membantu otomatisasi, misalnya rekomendasi nutrisi atau kondisi tanaman? Jawaban Ibu Zuna: Kami juga ada rencana pakai robot keliling. Kalau robot melihat ada tanaman yang kuning atau kurang nutrisi, sistem harus bisa menerima informasi itu dan menyesuaikan dosis nutrisi otomatis. Saya ingin sistem ini bisa terhubung dengan robot, bukan robot berjalan sendiri tanpa koneksi. Untuk rencana data yang dikumpulkan itu adalah kategori tanaman (sehat, kurang sehat, dan sakit), lalu fase dari tanaman tersebut masuk ke vegetatif atau generatif, lalu ada nilai konsentrasi nutrisi ppm, dan rekomendasi volume tanaman si dek.
6.	Informasi apa saja yang Ibu ingin dapatkan sebagai notifikasi? Apakah perlu ringkasan otomatisasi harian atau peringatan kalau ada kegagalan? Jawaban Ibu Zuna: Saya ingin ada notifikasi kalau otomatisasi berhasil atau gagal. Kalau pagi ini penyiraman tidak jalan, saya harus tahu supaya bisa segera diperbaiki. Notifikasi juga harus muncul kalau nilai sensor keluar batas, atau stok nutrisi mau habis.
7.	Oke Ibu, apakah ada tambahan lainnya? Jawaban Ibu Zuna:

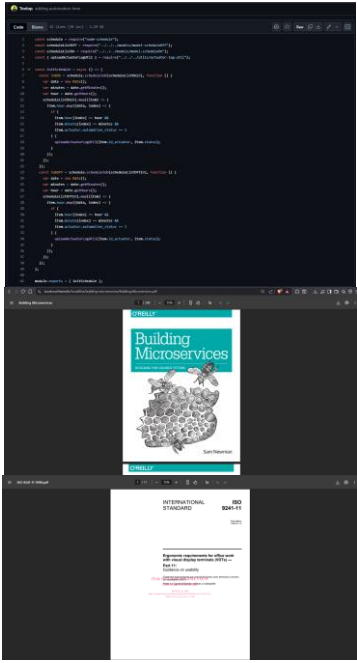
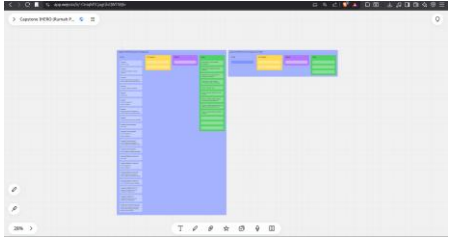
	Oiya dek, 1 lagi saya ingin tahu perangkat mana yang nyala, kapan nyala, dan siapa yang mengubah aturannya. Ini penting untuk memastikan semuanya berjalan benar. Sama tampilan si paling dek, kami sering memantau lewat HP sambil keliling. Tampilan di HP harus enak dilihat, jangan besar-besar dan jangan keluar layar bisa discroll seperti yang lama.
--	--

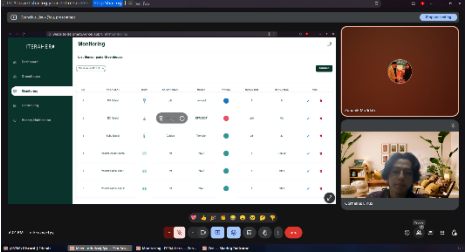
Tertanda Tangan,
Lampung Selatan, Rabu, 19-11-2025



Zunanik Mufidah, S.TP., M.Si.

C. Daftar Bukti Dokumentasi Penelitian

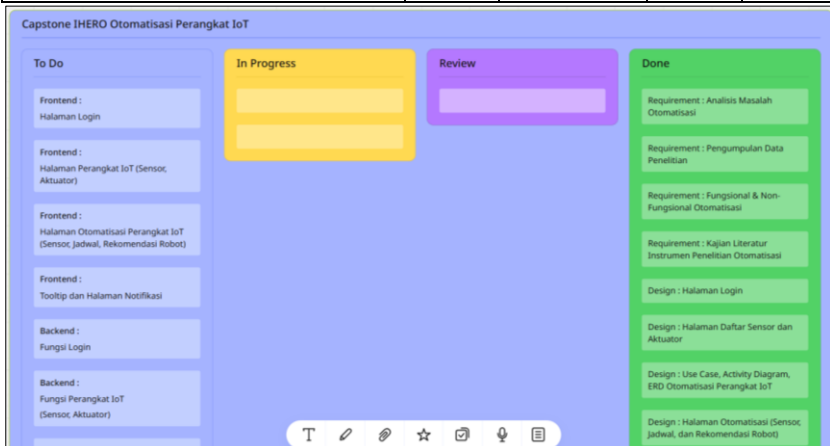
Bukti Foto	Detail Informasi
	Wawancara dengan Zunanik Mufidah, S.TP., M.Si. selaku penanggung jawab greenhouse Kebun Raya ITERA Kegiatan wawancara terbuka untuk menggali permasalahan operasional dan kebutuhan otomatisasi perangkat IoT di greenhouse. Timestamp GKU1 ITERA 18-11-2025 Pukul 17:00-18:00
	Kajian literatur mengenai permasalahan dan temuan masalah pada sumber kode lama Kegiatan peninjauan referensi dan analisis kode sumber sebelumnya untuk mengidentifikasi batasan teknis dan peluang peningkatan subsistem. Timestamp 18-11-2025 Pukul 20:00 sd. 23:00
	Perancangan Tugas Agile Kanban Kegiatan penyusunan backlog dan visualisasi alur kerja menggunakan pendekatan Agile Kanban sebagai dasar perencanaan pengembangan subsistem. Timestamp 20-11-2025 Pukul 7:00 – 15:00

Bukti Foto	Detail Informasi
	Tahap Penunjukan Masalah Inisialisasi Bagian Requirement & Review Design Rancangan (Use Case, Activity Diagram, ERD Design <i>Wireframe</i>) Kegiatan diskusi untuk memvalidasi kebutuhan sistem serta melakukan peninjauan desain awal seperti Use Case, Activity Diagram, ERD, dan Wireframe. Timestamp 20-11-2025 Pukul 17:00 – 18:00
	Pengambilan observasi tambahan & tanya jawab mengenai sensor, aktuator, dan instrumen yang diperlukan untuk otomatisasi Kegiatan observasi lapangan untuk memastikan kesesuaian rancangan subsistem dengan perangkat fisik yang digunakan di greenhouse. Timestamp 23 November pukul 17:30 – 18:00
	Judul Detail Kegiatan ke N Deskripsi Timestamp

D. Rincian Proses Agile Kanban

No.	Nama Fitur dan Proses Penelitian	To Do	In Progress	Review	Done
1.	Requirement : Analisis Masalah Otomatisasi	✓	✓	✓	✓
2.	Requirement : Pengumpulan Data Penelitian	✓	✓	✓	✓
3.	Requirement : Kajian Literatur Instrumen Penelitian Otomatisasi Perangkat IoT	✓	✓	✓	✓
4.	Requirement : Fungsional & Non-Fungsional Otomatisasi Perangkat IoT	✓	✓	✓	✓
5.	Design : Use Case, Activity Diagram, ERD Otomatisasi Perangkat IoT	✓	✓	✓	✓
6.	Design : Wireframe Halaman Login	✓	✓	✓	✓
7.	Design : Wireframe Halaman Perangkat IoT (Sensor dan Aktuator)	✓	✓	✓	✓
8.	Design : Wireframe Halaman Otomatisasi Perangkat IoT (Sensor, Jadwal, Rekomendasi Robot)	✓	✓	✓	✓
9.	Design : Wireframe Tooltip dan Halaman Notifikasi	✓	✓	✓	✓
10.	Frontend : Halaman Login	✓	✓	✓	✓
11.	Frontend : Halaman Perangkat IoT (Sensor, Aktuator)	✓	✓	✓	✓
12.	Frontend : Halaman Otomatisasi Perangkat IoT (Sensor, Jadwal, Rekomendasi Robot)	✓	✓	✓	✓
13.	Frontend : Tooltip dan Halaman Notifikasi	✓	✓	✓	✓
14.	Backend : Fungsi Login	✓	✓	✓	✓
15.	Backend : Fungsi Perangkat IoT (Sensor, Aktuator)	✓	✓	✓	✓
16.	Backend : Fungsi Otomatisasi Perangkat IoT (Sensor, Jadwal, Rekomendasi Robot)	✓	✓	✓	✓
17.	Backend : Fungsi Tooltip dan Halaman Notifikasi	✓	✓	✓	✓
18.	Integrasi Frontend Backend : Fitur Login	✓	✓	✓	✓

No.	Nama Fitur dan Proses Penelitian	To Do	In Progress	Review	Done
19.	Integrasi Frontend Backend : Fitur Perangkat IoT (Sensor, Aktuator)	✓	✓	✓	✓
20.	Integrasi Frontend Backend : Fitur Otomatisasi Perangkat IoT (Sensor, Jadwal, Rekomendasi Robot)	✓	✓	✓	✓
21.	Integrasi Frontend Backend : Fitur Tooltip dan Halaman Notifikasi	✓	✓	✓	✓
22.	Pengujian Blackbox & Responsif : Fitur Login	✓	✓	✓	✓
23.	Pengujian Blackbox & Responsif : Fitur Perangkat IoT (Sensor, Aktuator)	✓	✓	✓	✓
24.	Pengujian Blackbox & Responsif : Fitur Otomatisasi Perangkat IoT (Sensor, Jadwal, Rekomendasi Robot)	✓	✓	✓	✓
25.	Pengujian Blackbox & Responsif : Fitur Tooltip dan Halaman Notifikasi	✓	✓	✓	✓
26.	Pengujian Usability SUS : Pengujian menyeluruh ke fitur Otomatisasi Perangkat IoT	✓	✓	✓	
27.	Analisis dan Penarikan Kesimpulan : Analisis dan Penarikan Kesimpulan Seluruh Hasil Pengujian Blackbox, Responsif, dan Usability	✓	✓	✓	
Total Fitur dan Proses Penelitian (%)		100%	32%	32%	32%



Link Kanban Board : <https://s.id/XvO8t>

E. *Stakeholder yang memiliki akses*

Nama	Status/Jabatan	Program Studi	Hak Akses Sistem
Alawiyah, S.P., M. Hut.	Kepala UPT Konservasi Flora Sumatera Kebun Raya ITERA	-	Pengelola
Ananda Poppy Tirana Putri, S.Ars.	Bidang Arsitektural dan Visual Grafis	-	Pengelola
Eryka Merdiana, S.P.	Pranata Lab Pendidikan	-	Pengelola
Agung Permada Yusuf, S.Hut.	Bidang Eksplorasi dan Koleksi Tumbuhan	-	Pengelola
Anggita Marchetiara Putri, S.Hut.	Pranata Lab Pendidikan	-	Pengelola
Dr. Suratun Nafisah, M.Sc.	Dosen	Teknik Elektro	Pengelola
Zunanik Mufidah, S.TP., M.Si.	Dosen	Teknik Biosistem	Super Admin
Kisna Pertiwi, S.Si., M.Si.	Dosen	Teknik Biosistem	Pengelola
Raizummi Fil'aini, S.TP., M.Si.	Dosen	Teknik Biosistem	Pengelola
M Gilang Indra Mardika, M.T.	Dosen	Teknik Sipil	Pengelola
Alif Renaldi Pratama	Mahasiswa	Teknik Elektro	Pengelola
Alfin Mahmud	Mahasiswa	Teknik Elektro	Pengelola
Gerry Romora Tarigan	Mahasiswa	Teknik Biosistem	Pengelola
Amas Muda Luckyto Tamba	Mahasiswa	Teknik Biosistem	Pengelola
Iqbal Darwis	Mahasiswa	Teknik Biosistem	Pengelola
Istifa Zuriyanti	Mahasiswa	Teknik Biosistem	Pengelola
Cornelius Linux	Mahasiswa	Teknik Informatika	Super Admin
Muslimin	Mahasiswa	Rekayasa Instrumentasi Automasi	Pengelola
Ade Muharom	Mahasiswa	Rekayasa Instrumentasi Automasi	Pengelola
Muhammad Farhan Yusuf Almajid	Mahasiswa	Teknik Elektro	Staff
Rolas Agusta Nugrahadi	Mahasiswa	Teknik Elektro	Staff
Akbar Tri Agung	Mahasiswa	Teknik Elektro	Staff
Nadya Shafwah	Mahasiswa	Teknik Informatika	Staff
Louis Hutabarat	Mahasiswa	Teknik Informatika	Staff
Martino Kelvin	Mahasiswa	Teknik Informatika	Staff
Samuel Cornelius Evaldo	Mahasiswa	Rekayasa Instrumentasi Automasi	Staff
Dwifachruzi Al Jariah	Mahasiswa	Rekayasa Instrumentasi Automasi	Staff

Nama	Status/Jabatan	Program Studi	Hak Akses Sistem
Muhammad Zaki Irfandi	Mahasiswa	Rekayasa Instrumentasi Automasi	Staff
Sherly Anggraeni	Mahasiswa	Teknik Biosistem	Staff
Isnaton Kurniawati	Mahasiswa	Teknik Biosistem	Staff
Syahra Nurhafizah	Mahasiswa	Teknik Biosistem	Staff
Gimin	Pekerja	Pengelola Kebun	Pengelola
Total Staff			12
Total Pengelola			18
Total Super Admin			2
Total Pengguna Memiliki Akses			32

F. Rincian Pengujian *Blackbox Testing*

Perangkat Sensor			
No	Kode Perangkat	Nama	Deskripsi Singkat
1	S-01	Arus Listrik GH	Sensor ini mengukur arus listrik yang digunakan perangkat di <i>greenhouse</i> .
2	S-02	Suhu Ruang	Sensor ini mengukur suhu udara di dalam <i>greenhouse</i> .
3	S-03	Kelembapan Ruang	Sensor ini mengukur tingkat kelembapan udara di dalam <i>greenhouse</i> .
4	S-04	Suhu Tanah	Sensor ini mengukur suhu pada media tanam.
5	S-05	pH Tanah	Sensor ini mengukur tingkat keasaman tanah untuk menilai kondisi media tanam.
6	S-06	Nutrisi Tanah (EC)	Sensor ini mengukur tingkat kepekatan nutrisi dalam tanah melalui nilai EC.
7	S-07	Kalium Tanah (K2)	Sensor ini mengukur kadar kalium dalam tanah untuk mendukung pertumbuhan tanaman.
8	S-08	Nitrogen Tanah (N2)	Sensor ini mengukur kadar nitrogen dalam tanah untuk mendukung pertumbuhan tanaman.
9	S-09	TDS Nutrisi Pupuk	Sensor ini mengukur total zat terlarut pada larutan nutrisi pupuk.
10	S-10	pH Nutrisi Pupuk	Sensor ini mengukur tingkat keasaman larutan nutrisi pupuk.
11	S-11	Level Air Tandon Utama	Sensor ini mengukur ketinggian air di tandon utama untuk memastikan ketersediaan air.
12	S-12	Level Air Nutrisi A	Sensor ini mengukur ketinggian larutan nutrisi A untuk memastikan persediaan tetap aman.
13	S-13	Level Air Nutrisi B	Sensor ini mengukur ketinggian larutan nutrisi B untuk memastikan persediaan tetap aman.
14	S-14	Debit Pompa Air	Sensor ini mengukur laju aliran air yang keluar dari pompa air.
Perangkat Aktuator			
No	Kode Perangkat	Nama	Deskripsi Singkat

Perangkat Sensor			
No	Kode Perangkat	Nama	Deskripsi Singkat
15	A-01	Kipas Pendingin	Aktuator ini mengalirkan udara untuk menurunkan suhu ruang greenhouse.
16	A-02	Sirkulasi Pendingin	Aktuator ini menjalankan sistem sirkulasi untuk membantu proses pendinginan.
17	A-03	Pengadukan Tandon Utama	Aktuator ini mengaduk isi tandon utama agar campuran tetap merata.
18	A-04	Pengadukan Tandon A	Aktuator ini mengaduk larutan nutrisi A agar konsentrasi tetap stabil.
19	A-05	Pengadukan Tandon B	Aktuator ini mengaduk larutan nutrisi B agar konsentrasi tetap stabil.
20	A-06	Pompa Nutrisi A dan B	Aktuator ini memompa nutrisi A dan B menuju proses pencampuran ke Tandon Utama.
21	A-07	Pengisian Air Tandon Utama	Aktuator ini membuka aliran air untuk mengisi tandon utama secara terkontrol.
22	A-08	Pompa Distribusi Nutrisi	Aktuator ini menyalurkan larutan nutrisi ke area tanaman sesuai kebutuhan sistem.
Data Rekomendasi Robot			
No	Kode Data	Nama	Deskripsi Singkat
23	DR-01	Kategori Tanaman	(Sehat, Kurang Sehat, Sakit)
24	DR-02	Fase Tanaman	(Vegetatif, Generatif)
25	DR-03	Rekomendasi Nutrisi (PPM)	(480-2000)
26	DR-04	Volume Air (ml/tanaman)	(120-380)

Bagaimana jika otomatisasi robot ini jadwalnya menabrak?

Daftar Otomatisasi Perangkat IoT(Penjadwalan, Ambang Batas Sensor, Data Rekomendasi Robot) Otomatisasi maksudnya adalah menyalakan aktuator apa

Notifikasi (Bagaimana mendesain supaya notifikasi ini modular seiring bertambahnya perangkat sensor, aktuator, robot) ? tapi dalam lingkup otomatisasi Perangkat IoT

Activity-Log (Untuk melihat seluruh aktivitas perangkat IoT)

No	Fitur	Jumlah File Code	Jumlah Route
1	Login	6	3
2	Manajemen Perangkat Sensor	19	19
3	Manajemen Perangkat Aktuator	17	14
4	Otomatisasi Berdasarkan Penjadwalan	15	9
5	Otomatisasi Berdasarkan Ambang Batas Sensor	8	7
6	Otomatisasi Berdasarkan Data Robot	11	7
7	Notifikasi	10	9
8	Log Aktivitas	5	4
9	Hak Akses	14	6
Total	9 Fitur	105	78